

INTERNATIONAL UNIVERSITY OF APPLIED SCIENCES

Frequent Batch Auction Engine

A Design Analysis and Simulation-Based Comparison against Continuous Trading

Bachelor Thesis

submitted for the degree of Bachelor of Science

Author: Alejandro Moral Aranda
Matriculation number: 92132071
Programme: B.Sc. Data Science
Supervisor: Prof. Dr. rer. nat. Hartwig Mayer
Submission date: November 5, 2026

Abstract

Continuous limit order books process orders serially, in the sequence in which they arrive. This single architectural property creates a mechanical incentive to compete on speed rather than on price, and the resulting latency arms race imposes a measurable cost on liquidity demanders. The frequent batch auction (FBA) has been proposed as a market design response: orders are collected over a short discrete interval and cleared simultaneously at a single uniform price, so that being faster within the interval no longer buys a better price.

This thesis studies the FBA from two directions. The theoretical part decomposes an FBA matching engine into five design blocks: the batch interval, the information disclosure regime, order flow segregation, the clearing and rationing rule, and settlement and residual handling. For each block it sets out the available options and derives, from auction theory and market microstructure theory, the strategic behaviour each option induces. The analysis shows that batching does not by itself make speed worthless. Price-time rationing inside a batch still rewards being early, and the price improvement that uniform clearing produces is a transfer between buyers and sellers rather than a gain. When the close of a batch is predictable, the trader who submits last can steer that transfer, so the race to be first becomes a race to be last.

The empirical part implements a continuous double auction and a frequent batch auction with one-second batches, and replays through both the same order-by-order data of the Solana (SOL) perpetual futures contract on the Hyperliquid exchange for December 2025. Because both engines receive identical orders and are compared second by second, any difference between them comes from the matching mechanism alone.

The main finding is that, at one second, batching largely reproduces the continuous market with a short delay, and where the two differ the continuous market comes out slightly ahead. Price formation does not differ: both engines trade statistically indistinguishable volumes at closely matching prices, and the same net order flow moves both prices by the same amount within five seconds. The continuous market is slightly more liquid, a few percent deeper near the price, and slightly less volatile, and it executes orders faster: in the batch market orders wait 3.52 s longer for their first fill. Batching is meant to add liquidity by gathering buy and sell orders that arrive at different moments into one auction. In this market, however, trades occur in only about a quarter of all seconds, so a one-second batch usually contains just the few orders that would have traded at once in the continuous market, and there is nothing extra to combine. In the continuous market, resting orders pay a systematic adverse-selection cost; whether batching lowers it cannot be measured in the replay, because trades in a batch auction have no direction. These results describe the mechanical effect of batching on fixed historical orders, not how traders would adapt to it.

Keywords: market microstructure; frequent batch auction; matching engine; latency arbitrage; uniform-price clearing; Level 4 order book data.

Declaration of Authorship

I hereby declare that this thesis is my own work and that I have used no sources or aids other than those indicated. All passages taken literally or in substance from published or unpublished sources are identified as such. This thesis has not been submitted, in the same or a similar form, to any other examination authority.

Place, Date

Signature

Contents

Abstract i

Declaration of Authorship iii

List of Figures vii

List of Tables viii

List of Abbreviations ix

1 Introduction 1

1.1 Motivation . . . 1
1.2 The Design Problem of a Frequent Batch Auction Engine . . . 2
1.3 Problem Statement and Research Gap . . . 2
1.4 Research Questions . . . 3
1.5 Contribution . . . 3
1.6 Structure of the Thesis . . . 4

2 Background 5

2.1 Basic Terminology and Notation . . . 5
2.2 The Continuous Double Auction . . . 8
2.3 The Frequent Batch Auction . . . 9
2.4 Measurement Framework . . . 14
2.5 Metrics . . . 14

3 Theoretical Foundations and Literature Review 21

3.1 Discrete-Time Market Design . . . 21
3.2 Information Asymmetry and Adverse Selection . . . 23
3.3 Market Transparency Theory . . . 24
3.4 Auction Theory and Uniform-Price Clearing . . . 25
3.5 Measurement of Market Quality: Motivating the Metrics . . . 26
3.6 From Theory to Design and Measurement . . . 27

4 Design of a Frequent Batch Auction Engine 28

4.1 The Engine as a Set of Design Blocks . . . 28

4.2 Block 1: The Batch Interval . . . 28
4.3 Block 2: Information Disclosure . . . 33
4.4 Block 3: Order Flow Segregation . . . 34
4.5 Block 4: Matching, Clearing and Rationing . . . 36
4.6 Block 5: Settlement and Residual Handling . . . 37
4.7 Summary: Design Options and Their Consequences . . . 38

5 Simulation Design and Data **41**
5.1 Simulation Environment . . . 41
5.2 Venue and Data . . . 43
5.3 Experimental Design . . . 44
5.4 Metrics . . . 45
5.5 Validation of the Simulator . . . 46

6 Results: Comparing CDA and FBA **47**
6.1 Sample and Descriptive Statistics . . . 48
6.2 Code Validation Results . . . 49
6.3 RQ2.1: Liquidity and Transaction Costs . . . 49
6.4 RQ2.2: Price Discovery and Market Quality . . . 52
6.5 RQ2.3: Execution . . . 55
6.6 Conclusions from the Simulation . . . 56
6.7 Limits of the Comparison . . . 60

7 Conclusion **63**
7.1 Answers to the Research Questions . . . 63
7.2 Limitations . . . 64
7.3 Future Work . . . 65

Bibliography **66**

A Batch Clearing Algorithm **69**

B Engine Implementation and Configuration **71**
B.1 Orders and Trades . . . 71
B.2 The Replay Loop . . . 72
B.3 The Continuous Engine . . . 72
B.4 The Batch Engine . . . 73
B.5 Fixed Configuration of the Run . . . 74

C Reproducibility **75**
C.1 Getting the Code . . . 75
C.2 Running It Without Downloading Any Data . . . 76
C.3 Running the Full Month . . . 76

C.4 Downloading the Exact Simulated Output Used Here 78

D Testing and Verification **79**

D.1 The Input Data 79

D.2 The Engines 80

D.3 The Metrics and the Result Files 81

D.4 Determinism 81

D.5 Does the Replay Match the Real Venue? 82

D.6 Was the Month One Continuous Run? 82

D.7 What Was Not Checked 83

E Comparability of the FBA's Unmatched Orders and the CDA Book **84**

E.1 The Question 84

E.2 Pre-Clearing: Settlement Mismatch and Inflated Depth 84

E.3 Post-Clearing: Lopsidedness and a Different Kind of "Still There" 85

E.4 What the Metric Formulas Actually Use 85

List of Figures

2.1 FBA clearing as a supply/demand diagram 10

4.1 The batch cycle and its design blocks 29

6.1 Spread decomposition 50
6.2 Depth comparison 51
6.3 Daily effective spread, full month 51
6.4 Matched-timestamp volatility by day 53
6.5 Kyle's lambda 53
6.6 Price paths, both mechanisms 54
6.7 Price difference, FBA minus CDA 54
6.8 Engine-control: compute latency 56

C.1 The simulator's interactive prompt 77

D.1 Simulated price against the real price 83

List of Tables

2.1 Notation 6
2.2 Overview of the metrics 15

3.1 From theory to design and measurement 27

4.1 Disclosure regimes and induced behaviour 34
4.2 Design options and their consequences 38

5.1 FBA design choices 42

6.1 Sample description 48
6.2 RQ2.1 results 49
6.3 RQ2.2 results 52
6.4 RQ2.3 results 55
6.5 Findings and their reliability 59
6.6 How far each metric can be relied on 62

B.1 Engine configuration 74

D.1 Independent record counts 79
D.2 Rust engines against an independent reference 80

List of Abbreviations

bps	Basis points (10^{-4})
CDA	Continuous Double Auction
CME	Chicago Mercantile Exchange
DFBA	Dual Flow Batch Auction
FBA	Frequent Batch Auction
HAC	Heteroskedasticity- and Autocorrelation-Consistent (standard errors)
IOC	Immediate-or-Cancel
RMS	Root Mean Square
RQ	Research Question
SOL	Solana (token)
USD	United States Dollar
VWAP	Volume-Weighted Average Price

Chapter 1

Introduction

Motivation

Almost every major securities and derivatives exchange organises trading as a continuous double auction (CDA), implemented as a continuous limit order book. Orders are processed one at a time in the order the exchange received them, and whenever an incoming order crosses the opposite side of the book a trade is executed immediately at the resting limit price. The rule is simple, easy to explain, and a trader willing to pay the current best price never has to wait.

The same rule has a far less benign structural consequence. Because execution is serial and instantaneous, the value of a trading opportunity accrues entirely to whoever reaches the matching engine first. When public information changes the fair value of an asset, the stale quotes resting in the book are mispriced for a short interval, and the first participant to act either takes a stale quote or cancels it. Nothing about the informational content of the event determines who wins; only relative speed does. The continuous auction is meant to give each trade to the participant willing to pay the best price; in practice it partly gives it to the participant who has spent the most on being fast (Budish et al. 2015).

The competition is not costless. Microwave and hollow-core fibre links, co-location and custom hardware absorb real resources without improving the informational efficiency of prices in any first-order way, and for participants outside the race the risk of being picked off is priced into quotes: a market maker who expects to lose a fraction of its fills to stale quotes must widen its spread to break even on the remainder. Aquilina et al. (2022) measure this cost directly from London Stock Exchange message-level data.

The frequent batch auction (FBA) is the market design response proposed by Budish et al. (2015). Instead of processing orders as they arrive, the engine collects all orders arriving within a short interval τ and clears them simultaneously at a single uniform price. Arrival order inside the interval no longer matters, and all trades settle at the same price, so a participant cannot obtain a better price by being earlier. Speed retains value only across intervals, the marginal return to shaving another nanosecond off a network path collapses to approximately zero, and competition is redirected from latency to price. As Chapter 4 shows, this holds only under particular design choices: a predictable batch close and time priority inside the batch each give speed value again.

The idea is not exotic: most equity exchanges already run uniform-price call auctions at the open and close. What is new is the frequency — an auction every few hundred milliseconds or faster, so that the delay is small for ordinary traders yet long enough to swamp the speed differences that fund the arms

race. In decentralised finance the argument arrives from a different direction but converges on the same mechanism: settlement is already discrete, ordering inside a block is economically contestable, and uniform-price batch clearing removes the ordering rent otherwise extracted by block producers (Daian et al. 2020; Gong et al. 2023; Walther 2021; Dutta and Team 2025).

The Design Problem of a Frequent Batch Auction Engine

Stating that orders should be batched and cleared at a uniform price does not yet specify a matching engine; it specifies a family of them. An operator implementing one must decide how long the interval is and whether its end is predictable, what participants see while orders accumulate, whether all flow meets in one pool, and, when the two sides do not match exactly at the clearing price, which rationing rule decides who is filled. None of these questions has a single correct answer; each is a trade-off that changes the strategic problem participants face. The theoretical part of this thesis makes those trade-offs explicit and derives, from auction theory and market microstructure theory, which behaviour each option should be expected to induce.

Problem Statement and Research Gap

The existing literature leaves two gaps that this thesis addresses. First, the literature on frequent batch auctions is largely about whether batching improves on continuous trading, not about how a batching engine is built. Budish et al. (2015) establish the equilibrium argument, Aquilina et al. (2022) quantify what the alternative costs, Zoican et al. (2021) and Eibelshäuser and Smetak (2022) study how information and speed interact inside auctions, and Aldrich and López Vargas (2020) compare the two institutions in the laboratory, and Budish et al. (2024) explain why exchanges, which earn rents from selling speed, have little private incentive to adopt batching at all. Individual design dimensions appear in isolation: rationing rules in the derivatives microstructure literature, disclosure in the transparency literature (Madhavan 1996), and clearing formulations in the decentralised exchange literature (Walther 2021; Ramsay et al. 2023). What is missing is a consolidated treatment that decomposes the engine into its architectural blocks, sets out the options for each, and traces each option to the strategic behaviour it induces.

Secondly, field comparisons cannot hold order flow constant. The empirical literature comparing batch and continuous trading must compare different venues, assets or time periods. Lee et al. (2026) exploit the Taiwan Stock Exchange's transition from intraday call auctions to continuous trading, about as clean a natural experiment as this area offers, yet the participant population, submission strategies and information environment all adjust together with the mechanism. This is a property of the object, not a shortcoming of the studies: an order stream exists only under the mechanism that produced it, so a strictly like-for-like comparison in which one and the same stream is executed once continuously and once in batches cannot be obtained from field data. Simulation with two interchangeable engines is the only design that holds the flow fixed and varies the mechanism.

Research Questions

The thesis is organised around two research questions, one for each part.

> **RQ1 (theoretical).** How is a frequent batch auction engine designed, and which design options exist for each of its architectural components?

RQ1 is answered by decomposing the engine into five design blocks — the batch interval, the information disclosure regime, order flow segregation, the matching and rationing rule, and settlement and residual handling — enumerating the options within each, and deriving the strategic behaviour each induces.

> **RQ2 (empirical).** How do a continuous double auction and a frequent batch auction differ when both engines are fed with the same order flow?

RQ2 is decomposed into three measurable dimensions, each operationalised by a set of metrics defined in Section 2.5:

- **RQ2.1 Liquidity and transaction costs.** Which mechanism lets participants trade at a lower cost and with more depth available?
- **RQ2.2 Price discovery and market quality.** Which mechanism produces a more stable, less dispersed price series?
- **RQ2.3 Execution.** How much volume is executed, how long do orders wait for their first fill, and at what price relative to their own limit do they trade? How does batching change these outcomes for the same order flow?

How participants would time their orders against a batch close cannot be measured this way: the replayed orders were submitted to a continuous market and never saw a batch boundary (Section 7.2).

Contribution

The thesis makes three contributions.

1. **A map of the design space.** It decomposes a frequent batch auction engine into five design blocks, sets out the options within each, and links every option to the behaviour theory predicts it will induce and to its main cost (Chapter 4, summarised in Table 4.2).
2. **The boundary repricing race.** It shows that the price improvement uniform clearing gives on one side of a batch is a transfer from the other side, not a gain, and that under a predictable close and a visible batch the participant who submits last can steer that transfer. Batching with a fixed close therefore moves the speed race to the batch boundary instead of removing it (Section 4.2.3). The same chapter proposes four ways to redefine the maker–taker distinction once arrival order no longer defines it (Section 4.4).

3. **A same-flow comparison.** It implements a continuous and a batch matching engine on one shared order and trade representation and feeds both with the same month of Level 4 order flow from the Hyperliquid exchange (Albers et al. 2026), at nanosecond resolution. The mechanism is the only treatment variable, and quantities that field data never show, such as every submitted price, the exact rationing outcome and which batches clear no trade, are observed directly (Chapters 5 and 6).

All three are bounded by what a replay of fixed historical order flow can show. The design decomposition is normative, not a report of which option was tested; the empirical comparison isolates the mechanism's mechanical effect on one dataset, not how participants would have behaved had they known they were trading under it (Sections 6.7 and 7.2 set out exactly where that line falls).

Structure of the Thesis

The thesis is organised as follows.

1. **Introduction** (this chapter): motivation, research gaps, research questions and contributions.
2. **Background** (Chapter 2): terminology and notation, precise definitions of the continuous double auction and the frequent batch auction, and the formal definition of every metric used in the comparison.
3. **Theoretical Foundations and Literature Review** (Chapter 3): the theory and literature the thesis builds on, each strand linked to a design block and to the metrics that measure it.
4. **Design of a Frequent Batch Auction Engine** (Chapter 4): answers RQ1 by decomposing the engine into its five design blocks, derives the boundary repricing race that a predictable batch close produces, and summarises the consequences of every design option in one table.
5. **Simulation Design and Data** (Chapter 5): the simulation, the data and the experimental design.
6. **Results** (Chapter 6): answers RQ2 with the paired comparison of the two engines.
7. **Conclusion** (Chapter 7): the answers to both research questions, the limitations and future work.

The appendices document the clearing algorithm, the engine configuration, reproducibility, testing, and the comparability of the FBA's unmatched orders with a CDA book.

Chapter 2

Background

This chapter fixes the vocabulary and the notation that the rest of the thesis uses. It defines the basic objects of a trading venue — orders, the limit order book, trades — and then gives a precise description of the two matching mechanisms compared in this thesis, the continuous double auction (CDA) and the frequent batch auction (FBA). It closes with the formal definition of every metric used in the empirical comparison of Chapter 6. Why these mechanisms and these metrics matter is the subject of Chapter 3; this chapter only says precisely what they are.

Basic Terminology and Notation

Orders

The instrument studied here is the SOL *perpetual futures contract* traded on Hyperliquid (SOL-PERP), a futures contract on the price of the Solana token without an expiry date (Hyperliquid 2026). For this analysis only its order book matters; one contract unit is one SOL. Prices p are quoted in US dollars per SOL (written USD throughout) and quantities q in SOL. Section 5.2.1 describes the venue. An *order* k is an instruction to buy or sell and is described by

$$k = (s_k,\, q_k,\, \pi_k,\, t_k), \tag{2.1}$$

where $s_k \in \{\text{buy}, \text{sell}\}$ is its side, $q_k > 0$ its quantity in units of the base asset (SOL), π_k its limit price in the quote currency (USD per SOL) and t_k its submission time. A buy order is also called a *bid* and a sell order an *ask*; the sets of bids and asks in a given collection of orders are written $\mathbb{B}$ and $\mathbb{A}$.

A *limit order* trades only at its limit price or better: a bid at π_k or lower, an ask at π_k or higher. A *market order* has no limit and accepts any price; it is treated as a bid with no upper price limit or an ask with no lower price limit. A *cancellation* removes a previously submitted order that has not yet been filled. An order's *time-in-force* states how long it may wait to be filled (for example until cancelled, or immediately or never); Section 5.1 states which of these rules the simulated engines enforce.

The Limit Order Book

A *limit order book* is the set of limit orders that have been submitted, not yet filled and not cancelled. These orders are said to *rest* in the book. Orders with the same limit price form a *price level*. At time t the *best bid* b_t is the highest resting bid price and the *best ask* a_t the lowest resting ask price. Their

midpoint and the quoted spread are

$$m_t = \frac{a_t + b_t}{2}, \qquad \text{quoted spread}_t = a_t - b_t. \tag{2.2}$$

The quantity resting at a price level is the *depth* at that level; the quantity at b_t and a_t is the depth *at the best*, or at the *touch*. Relative quantities are expressed in *basis points* (bps), $1\,\text{bps} = 10^{-4}$, so that a price p lies $|p-m|/m \cdot 10^4$ bps away from a reference price m.

Trades, Makers and Takers

A *trade* is an exchange of quantity between one buy order and one sell order at one price. When an arriving order is matched against a resting order, the resting order is the *maker* (it supplied liquidity) and the arriving order the *taker* or *aggressor* (it demanded liquidity). A trade is indexed like an order: trade k has price p_k, quantity q_k and time t_k, and whether k denotes an order or a trade is always clear from context. The direction of trade k is $\varepsilon_k = +1$ if the aggressor was a buyer and $\varepsilon_k = -1$ if it was a seller. The signed quantity $\varepsilon_k q_k$, summed over a set of trades, is their net *order flow*: positive when buying pressure dominates. A participant who quotes on both sides of the book to earn the spread is a *market maker* or *liquidity provider*.

Table 2.1 collects every symbol used in the thesis, grouped by topic, with the section in which each is defined.

Table 2.1: Notation used throughout the thesis.

Symbol	Meaning	Defined in
Orders, book and trades		
$p,\ q$	a price (USD per SOL) and a quantity (SOL)	§2.1
k	index of an order, or of a trade	§2.1
s_k, q_k, π_k, t_k	side, quantity, limit price and submission time of order k	§2.1
j	a resting order that an arriving order trades against (CDA)	§2.2
$\mathbb{B}, \mathbb{A}$	sets of bids (buy orders) and asks (sell orders) of a batch or a book	§2.1
b_t, a_t	best bid and best ask of the CDA book at time t	§2.1
m_t	midpoint $(a_t + b_t)/2$ of the CDA book at time t	(2.2)
m	a reference price against which a distance in bps is measured	§2.1
p_k, q_k, t_k	price, quantity and time of trade k	§2.1
ε_k	direction of trade k: $+1$ buyer-initiated, -1 seller-initiated	§2.1
Batch clearing		
τ	batch interval of the FBA; also the width of a measurement bucket ($\tau = 1$ s in the run)	§2.3
$\mathbf{1}\{\cdot\}$	indicator function: 1 if the condition holds, 0 otherwise	(2.3)
$D(p), S(p)$	cumulative demand and supply of a batch at price p	(2.3)

Table 2.1, continued

Symbol	Meaning	Defined in
$V(p)$	executable volume $\min\{D(p), S(p)\}$ at price p	§2.3
$\mathscr{P}^*$	set of volume-maximising prices	§2.3
$\underline{p}, \overline{p}$	lowest and highest price in $\mathscr{P}^*$	§2.3
V^*	maximal executable volume of a batch	§2.3
σ	tie-break rule selecting one price from $\mathscr{P}^*$	(2.6)
p^*	clearing price of a batch, $p^* = \sigma(\mathscr{P}^*)$	(2.6)
p_{prev}	clearing price of the previous batch that traded (tie-break step 3)	§2.3
x_k	quantity of order k executed in a batch	§2.3
Surplus		
$W^{\mathbb{B}}(p), W^{\mathbb{A}}(p)$	buyer and seller surplus (price improvement against own limits) of a batch at price p	(2.7)
W	total surplus of a batch, $W = W^{\mathbb{B}}(p^*) + W^{\mathbb{A}}(p^*)$; independent of the price chosen in $\mathscr{P}^*$	(2.7), (4.1)
Measurement grid		
t_0	timestamp of the first record of the run	(2.8)
$\text{bucket}(t)$	the one-second bucket containing time t	(2.8)
e	engine, $e \in \{\text{CDA}, \text{FBA}\}$	§2.4
M_t^e	value of metric M for engine e in bucket t	§2.4
d_t	paired difference $M_t^{\text{FBA}} - M_t^{\text{CDA}}$	§2.4
n	number of buckets (or days) with a paired value	§5.3
Metrics		
h	distance threshold of depth within h bps, $h \in \{10, 50, 100\}$	§2.5.1
m_k	pre-trade reference price of trade k	§2.5.1
Δ	markout horizon: time after a trade at which the reference price is read again, $\Delta \in \{1, 5, 30\}$ s	§2.5.1
$m_{k+\Delta}$	first reference price at or after $t_k + \Delta$	§2.5.1
S_k^{eff}	effective spread of trade k	(2.9)
$S_k^{\text{real}}(\Delta)$	realized spread of trade k at horizon Δ	(2.10)
$\text{PI}_k(\Delta)$	price impact of trade k at horizon Δ	(2.11)
P_t^{c}, R_t	last reference price in bucket t, and its return from the previous bucket (Amihud)	(2.12)
r_i	return between two successive reference prices within a bucket	§2.5.2
$\mathscr{T}$	set of buckets in which both engines traded	§2.5.2
VWAP_t^e	volume-weighted average price of engine e in bucket t	§2.5.2

Table 2.1, continued

Symbol	Meaning	Defined in
r_i^e	log return of VWAP^e between consecutive buckets of $\mathscr{T}$	(2.13)
$\mathrm{vol}_{\mathrm{day}}^e, N_{\mathrm{day}}$	matched-timestamp volatility of engine e on one day, and the number of returns it uses	(2.13)
λ	Kyle's lambda, price change per unit of net order flow	§2.5.2
f_i	venue net taker flow in second i (1,000 SOL), the regressor of Kyle's lambda	(2.14)
$y_i^e(h)$	price change (bps) of engine e from second $i-1$ to $i+h$	(2.15)
$\lambda^e(h), u_i^e$	Kyle's lambda of engine e at horizon h, and the regression residual	(2.16)
$\theta(h), v_i$	difference $\lambda^{\mathrm{FBA}}(h) - \lambda^{\mathrm{CDA}}(h)$, and its regression residual	(2.17)
μ	prior expected value of the asset in the model of Kyle (1985)	§3.2
Design analysis		
δ	difference in message latency between two participants	§4.2
$\tau_{\min}, \tau_{\max}$	bounds of a randomised closing time	§4.2
i	a participant (in sums: an observation index)	§4.2.3
ℓ_i	message latency of participant i	(4.3)
κ	share of the marginal volume allocated pro rata in a hybrid rationing rule	§4.5.3

The Continuous Double Auction

The CDA processes orders one at a time, in the order the venue receives them, against a limit order book. Resting orders are ranked by *price-time priority*: a better price first, and among orders at the same price, the earlier one first. When a buy order k arrives, it is matched as follows (a sell order is symmetric, with bids in place of asks):

1. While k has quantity left and the book holds an ask with $a_t \le \pi_k$ (always, for a market order), k trades against the first-ranked order j at the best ask. The trade price is the *resting* order's price, $p = a_t$, and the trade quantity is the smaller of the two orders' remaining quantities; both are reduced by it, and a fully filled resting order leaves the book.

2. Any quantity of a limit order that is left when no further ask crosses rests in the book at π_k, behind the orders already resting at that price. The unfilled remainder of a market order does not rest.

Three properties of this rule matter for the comparison. Execution is immediate: an order that can trade does so the instant it arrives. A single order that sweeps several price levels trades at several

prices. And the maker–taker distinction is defined by arrival order, so every CDA trade has a known aggressor and direction ε_k.

The Frequent Batch Auction

A frequent batch auction collects orders over a half-open window $[0, \tau)$ of length τ and, at the end of the window, considers all of them together instead of one at a time. Let $\mathbb{B}$ and $\mathbb{A}$ denote the sets of bids and asks in the batch. A market bid is treated as having no upper price limit and a market ask as having no lower price limit, so it never falls out of either schedule below on grounds of price.

Demand and Supply

The batch is cleared by lining up how much would be bought against how much would be sold at every possible price, and finding where the two balance. For any price p, the bid side gives the cumulative *demand* at that price and the ask side gives the cumulative *supply*. Demand is the quantity buyers are willing to take at p or less, so it can only fall as p rises: a bid counts toward it once its limit price is at least p, and drops out as soon as p climbs past that limit. Supply is the mirror image: the quantity sellers are willing to give up at p or more, which can only rise as p rises, since an ask counts once its limit price is at most p. Written out, with the market-order convention above,

$$D(p) = \sum_{k \in \mathbb{B}} q_k \mathbf{1}\{\pi_k \geq p\}, \qquad S(p) = \sum_{k \in \mathbb{A}} q_k \mathbf{1}\{\pi_k \leq p\}. \tag{2.3}$$

Plotted against price, D and S are *step* curves rather than smooth ones, because each order adds its whole quantity at once, the moment its limit price is reached, rather than a little at a time: D steps down as price rises, S steps up. Figure 2.1 shows exactly this picture for a small worked example. At any price p, only the smaller of the two sides can trade, so the volume executable at that price is

$$V(p) = \min\{D(p), S(p)\}. \tag{2.4}$$

Finding the Clearing Price

The two curves do not have to cross at one exact price: they can run side by side, both holding the same height, across several price ticks at once, which happens often when the book is thin. Every price in that stretch executes the same maximum volume, so volume by itself cannot say which one is used. The clearing rule therefore works in two stages. First, it finds every price that reaches the greatest executable volume, the volume-maximising set

$$\mathscr{P}^* = \arg\max_{p \in \mathbb{R}_+} V(p), \qquad V^* = \max_p V(p), \tag{2.5}$$

which, since D is non-increasing and S is non-decreasing, is a single closed interval $[\underline{p}, \overline{p}]$ whenever the book crosses at all ($V^* > 0$). When that interval holds more than one price, a second, tie-breaking

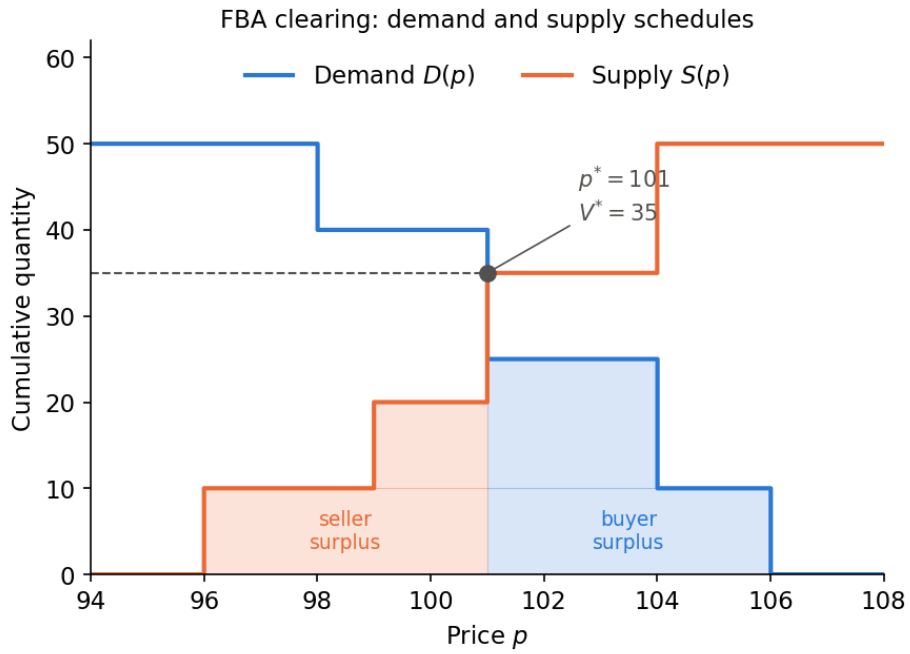


Figure 2.1: The FBA clearing mechanism drawn as a classical demand/supply diagram: $D(p)$ and $S(p)$ as step functions of price. Reading it: pick any price on the horizontal axis; the height of the demand step there is how much would be bought at that price or cheaper, the height of the supply step is how much would be sold at that price or dearer. The two curves cross at the clearing price $p^* = 101$, where the executable volume $V(p)$ is largest ($V^* = 35$). Drawn from a small illustrative batch of eight orders, four buys (limits 106, 104, 101, 98) and four sells (limits 96, 99, 101, 104), chosen because it clears without a tie and so needs no tie-break cascade (below). At p^* demand exceeds supply by 5, so the buy order at 101 is only partly filled. The shaded areas are the buyers' surplus (right of p^*, under D) and the sellers' surplus (left of p^*, under S), built from one rectangle per order that trades at a better price than its limit. A real batch from the simulated month has dozens of orders and does not render legibly at this scale.

stage is needed to pick exactly one of them:

$$p^* = \sigma\left(\mathscr{P}^*\right) \in \mathscr{P}^*. \tag{2.6}$$

Real auctions resolve this indeterminacy with a fixed cascade of tie-breaking steps, applied in order until one price remains (Haynes and Onur 2020), and the engine used in this thesis follows exactly this cascade (Chapter B):

1. choose the price (or prices) that maximise $V(p)$ — this is $\mathscr{P}^*$ itself;
2. if $\mathscr{P}^*$ is not a singleton, choose the price that minimises the residual imbalance $|D(p) - S(p)|$;
3. if still tied, choose the price closest to the auction's own previous clearing price p_{prev};
4. if still tied, choose the lowest such price.

For example, take a tick of 0.01 USD and a batch holding a buy order for 10 SOL with limit 100.03, a buy order for 5 SOL with limit 100.01, and a sell order for 10 SOL with limit 100.00. Every price from 100.00 to 100.03 executes the full 10 SOL, so $\mathscr{P}^* = [100.00, 100.03]$: over this stretch the executable volume is a horizontal line. At 100.00 and 100.01 demand is 15 SOL against a supply of 10, an imbalance of 5 SOL; at 100.02 and 100.03 the 5 SOL order no longer buys and demand equals supply. Step 2 therefore keeps 100.02 and 100.03, the prices where the demand and supply curves finally cross, and steps 3 and 4 choose between those two.

Allocation, Rationing and the Residual

Let $x_k \geq 0$ denote the executed quantity of order k under a volume-maximising allocation, so that $\sum_{k\in\mathbb{B}} x_k = \sum_{k\in\mathbb{A}} x_k = V^*$. Every trade of the batch is at the single price p^*. At p^* the two sides are usually not equal, $D(p^*) \neq S(p^*)$, so the orders on the longer side cannot all be filled and a *rationing rule* decides which are. The engine used here ranks the eligible orders of each side by *price-time priority within the batch*: market orders first, then the better limit price, then the earlier submission, and fills them in that order until the shorter side is used up (Section 4.5.3 discusses the alternatives). The volume left unmatched, $|D(p^*) - S(p^*)|$ on the longer side together with every order that was not eligible at p^*, is the *residual*; in this thesis it rolls over into the next batch with its limit price and submission time unchanged. An unfilled market order rolls over as a market order, still without a limit, and keeps its priority in the next batch. A common convention on exchanges is instead to convert it into a limit order at the auction price; the engine used here does not do this.

Price Improvement and Surplus

Uniform pricing gives every executed order a price at least as good as the one it asked for, and every inframarginal order a strictly better one; Figure 2.1 shows exactly how much better. A buyer with limit price π_k above p^* does not pay π_k; the whole batch pays p^*, so this order keeps the gap $\pi_k - p^*$ for every unit it trades. In Figure 2.1 each such order is one shaded rectangle: its width is how far that order's limit sits above the clearing price, its height is how much of it trades, and its area is what that

order gains by not having to pay what it was willing to. Sellers gain the mirror amount, $p^* - \pi_k$, the gap between the clearing price and their own limit. Summed over every executed order, these gaps are the familiar consumer- and producer-surplus areas of a supply/demand diagram, read off a single discrete batch instead of a continuous market. With price on the horizontal axis, as in Figure 2.1, the buyers' share is the area to the right of p^* under the demand curve, and the sellers' share is the area to the left of p^* under the supply curve.

Formally, each side's surplus at a candidate price p is

$$W^{\mathbb{B}}(p) = \sum_{k \in \mathbb{B}} (\pi_k - p) x_k, \qquad W^{\mathbb{A}}(p) = \sum_{k \in \mathbb{A}} (p - \pi_k) x_k, \tag{2.7}$$

and total surplus at the batch's own clearing price is $W = W^{\mathbb{B}}(p^*) + W^{\mathbb{A}}(p^*)$. Every term in both sums is non-negative for an executed order — a buy only has $x_k > 0$ if $\pi_k \geq p^*$, and a sell only if $\pi_k \leq p^*$, so no order ever executes at a loss relative to its own limit. This is what Chapter 6 measures as *trader surplus*. The worked example also previews a point Section 4.2.3 returns to in full: the two sides need not split W evenly. Here the buyers gain $5 \times 10 + 3 \times 15 = 95$ and the sellers $5 \times 10 + 2 \times 10 = 70$, while the buy and the sell with limit 101 trade exactly at their limit and gain nothing. In this example $\mathscr{P}^*$ holds the single price 101, so the split is pinned down; when $\mathscr{P}^*$ holds several prices, choosing a different one would split W differently without changing who trades or how much.

What Differs from the CDA

Set against Section 2.2, the FBA differs in three respects. Orders wait up to τ for their first chance to trade, instead of trading on arrival, and longer if they are rolled over. All trades of a batch share one price p^*, instead of each trading at the resting order's price. And because the orders of a batch trade simultaneously, no order crosses another's quote: there is no maker and no taker, and a trade has no observable direction ε_k. This point is one reason several metrics below need an adaptation for the FBA. The other reason is that the FBA has no order book that could be observed in the way the CDA's can.

Why the FBA's unmatched orders are not an order book. At first sight the FBA seems to offer a ready-made substitute for the CDA's book. At any moment it holds a set of orders that have not traded yet, each with a side, a quantity and a limit price. Sorted by price, this set would give a best bid, a best ask, depth at every level and a book imbalance, and every book-based statistic could then be computed exactly as for the CDA. This does not work, because the two sets are produced by different processes and therefore have different meanings.

A CDA book is what remains *after* matching. Every order that can trade does so the instant it arrives (Section 2.2), so an order sits in the book only because no opposite order has been willing to trade with it. The book is therefore never crossed ($b_t < a_t$ at all times), and its depth is liquidity that is available: a new order can trade against it immediately, at the prices shown.

The FBA's pending orders are what is waiting *for* matching: the residual of the last batch plus everything submitted since. None of these orders has yet been matched against any other, which has three consequences.

- **The set can be crossed.** Whenever a batch is going to trade, its best buy is at or above its best sell, so its "quoted spread" is zero or negative and its depth includes quantity that is about to execute.
- **Its content depends on when it is read.** Just after a clear it holds only the residual; just before the next clear it holds the residual plus a full interval of new orders. A CDA book has no such cycle.
- **Nobody can trade against it at the prices shown.** A pending order offers to trade only at the next clearing price, and only if that batch's demand and supply reach its limit. It is not standing liquidity at its own limit price in the moment it is observed.

Statistics computed on the pending set as if it were a book would measure the batching delay rather than the liquidity of the market. For the same reason, Kyle's lambda is not estimated from the pending set: both engines are measured against one common measure of order flow, the venue's own net taker flow (Section 2.5.2).

When the FBA is read: before the close, after the clear, or at the CDA's times. Any book-like FBA statistic must be read at some point in the batch cycle, and that choice decides which way the comparison is biased.

- **Just before the close** the pending set is at its fullest: it holds the whole interval's orders, including those that are about to execute. Depth read here is overstated, so the bias *favours* the FBA. Both depth measures in this thesis are read at this point (Section 2.5.1).
- **Just after the clear** the residual is, like a CDA book, no longer crossed, because everything that could trade at p^* has traded. But this is the thinnest moment of the cycle. The clear has just removed exactly the orders closest to p^*, and the next interval's orders have not arrived yet, so the residual is often one-sided near p^*. A spread or depth read here is biased *against* the FBA. The FBA quoted spread is read at this point, which is one reason it is treated as weaker evidence (Section 6.7.2).
- **At the CDA's snapshot times.** One could instead read the pending set at the same instants as the CDA's book snapshots, so that both engines are observed "at the same time". The same clock time does not make the two sets the same kind of object. At any instant inside a batch, the pending set is the residual plus whatever has arrived so far. It is still unmatched and possibly crossed, and its content depends on how far into the batch the snapshot falls. Averaging such readings over the interval only mixes the two biases above, in proportions set by when orders happen to arrive.

The sampling also differs. A CDA bucket value is the mean of all book snapshots taken within the second (Section 2.5.1), which describes the typical state of the book. An FBA value is a single reading per batch, taken at one end of the cycle.

For these reasons the comparison rests mainly on *trades*. A trade exists only once matching is complete, in both mechanisms: in the CDA at the instant two orders cross, and in the FBA at the instant the batch clears. Executed volume, VWAP and trader surplus are therefore read from the same kind of object in both engines, whenever they are read. The effective and realized spread and the price impact are trade-based too, but they need the direction of each trade, which the FBA does not have, so their FBA values cannot be compared with the CDA's (Section 6.7.1). The book-like FBA metrics are kept, but each is read at a stated point in the cycle, and its direction of bias is taken into account when the results are interpreted. Chapter E sets out the argument in more detail.

Measurement Framework

Both engines are measured on one common time grid. Every observation is assigned to a fixed-width time *bucket*,

$$\text{bucket}(t) = t_0 + \left\lfloor \frac{t - t_0}{\tau} \right\rfloor \tau, \tag{2.8}$$

where t_0 is the timestamp of the first record of the run and the width equals the batch interval, $\tau = 1$ second throughout this thesis. Every metric below is computed once per bucket and per engine, so each bucket yields one CDA value and one FBA value from the same order flow. Buckets are contiguous: a bucket with no qualifying observation still exists, with the metric marked missing. A missing value ("no qualifying observation in this bucket") and a value of zero ("there were observations and the value is zero") are kept distinct.

For a metric M, the comparison is the *paired difference* $d_t = M_t^{\text{FBA}} - M_t^{\text{CDA}}$ over the buckets in which both values exist; Section 5.3 describes how its standard error is estimated. Two metrics are built differently: matched-timestamp volatility is summarised and paired by calendar day, and Kyle's lambda is a slope estimated over all seconds together, compared through a regression on the difference (Section 2.5.2).

Metrics

Every metric below is computed for both engines. The metrics are grouped by the part of RQ2 (Section 1.4) they answer. Table 2.2 gives an overview; Chapter 3 explains why each is used.

Liquidity and Transaction Costs (RQ2.1)

This group measures how much quantity is available near the price and what a trade costs. The first three metrics describe the book; the spread decomposition and Amihud illiquidity describe the trades that executed. Book-based CDA values are averaged over *book snapshots*: the state of the CDA book

Table 2.2: Overview of the metrics, by research question. Every metric is computed for both engines.

RQ	Metric	What it measures
2.1	Quoted spread	cost of crossing the book now
	Depth at best / within h bps	quantity available near the price
	Effective spread	cost an actual trade paid
	Realized spread	part of that cost the maker keeps
	Price impact	part lost to informed traders
	Amihud illiquidity	price move per dollar traded
2.2	Matched-timestamp volatility	price variability on a common grid
	Intra-interval dispersion	price variation within one second
	Kyle's lambda	price move per unit of the venue's net order flow
2.3	Executed volume, trade count	how much trades
	Time to execution	how long orders wait
	Trader surplus	price improvement against the limit
—	Throughput, clearing latency	computing speed (control)

recorded after every record the engine processes, so a busy second contributes many snapshots and a quiet one few.

Quoted percentage spread. For the CDA, every book snapshot in the bucket with both sides present contributes $(a_t - b_t)/m_t \cdot 10^4$ bps, with m_t the midpoint of (2.2), and the bucket value is the mean over snapshots. The FBA has no book while a batch accumulates; its analogue is the gap between the best unfilled sell and the best unfilled buy among the orders left after a clear (including those rolled over from earlier batches), relative to p^* and in bps, averaged over the batches in which both exist.

Depth at best. For the CDA, depth at best is the mean over snapshots of (best-bid size + best-ask size)/2; only the touch is counted. For the FBA, it is the mean over batches of $(D(p^*) + S(p^*))/2$, the quantity eligible to trade at the clearing price. The FBA has no best bid and best ask in the CDA's sense. The natural candidate, the size at the best unfilled buy and sell among the orders left after a clear, is read at the thinnest moment of the batch cycle and is often one-sided near p^* (Section 2.3.5). The schedule at p^* was preferred because it exists in every batch; it counts every order priced at or better than p^*, not one price level, so the two engines' values are reported but not compared (Section 6.7.2). A batch that does not cross still has a price: the engine applies the same tie-break cascade to the submitted limit prices even when no volume can execute, and the price it selects serves as p^* for the FBA's book-like metrics in that batch, so depth and quoted spread exist in almost every second, not only in those with a trade.

Depth within h bps, $h \in \{10, 50, 100\}$. The quantity of all orders priced within h bps of a reference price, half the sum of both sides, cumulative across thresholds. The orders counted are those in the book (CDA, reference m_t) or pending in the batch just before it clears (FBA, rolled-over orders included, reference p^*).

Effective, realized spread and price impact. Let trade k have price p_k, direction ε_k and pre-trade reference price m_k (for the CDA, the midpoint of (2.2); the FBA's reference price is given below). The *markout horizon* Δ is the time after the trade at which the reference price is read again, and $m_{k+\Delta}$ is that later reference price. In proportional form (Hasbrouck 1993; Hendershott et al. 2011),

$$S_k^{\text{eff}} = \frac{2\varepsilon_k(p_k - m_k)}{m_k}, \tag{2.9}$$

$$S_k^{\text{real}}(\Delta) = \frac{2\varepsilon_k(p_k - m_{k+\Delta})}{m_k}, \tag{2.10}$$

$$\text{PI}_k(\Delta) = \frac{2\varepsilon_k(m_{k+\Delta} - m_k)}{m_k}, \tag{2.11}$$

Each is converted to basis points by multiplying by 10^4. By construction the effective spread is the sum of the other two, $S_k^{\text{eff}} = S_k^{\text{real}}(\Delta) + \text{PI}_k(\Delta)$, for every single trade and therefore also for any quantity-weighted average of trades. The effective spread is the trade's slippage against the pre-trade midpoint; in practice slippage also depends on the horizon over which it is measured, on trade size and on volatility (Section 3.2.3). The bucket value is the quantity-weighted mean over the bucket's trades, for $\Delta \in \{1, 5, 30\}$ s. For the CDA, m_k is the book midpoint just before the trade (the best price of the one side present, if the other side is empty) and $m_{k+\Delta}$ the first midpoint at or after $t_k + \Delta$. The FBA needs two adaptations. Its reference price m_k is the clearing price of the last earlier batch that traded, p_{prev}, since there is no midpoint while a batch accumulates, and $m_{k+\Delta}$ is the first FBA clearing price at or after $t_k + \Delta$. And its trades have no direction ε_k (Section 2.3), so the simulator records only the unsigned deviation $2|p_k - m_k|/m_k$.

Signing the FBA side. Each bucket in which both engines traded is given a sign $\hat{\varepsilon}_k$ by a zero-tick rule (Lee and Ready 1991) applied to the *CDA's* prices: taking these buckets in time order, the sign is $+1$ if the CDA's VWAP rose from the previous such bucket, -1 if it fell, and the last nonzero tick if it is unchanged. The reported FBA effective and realized spread are $\hat{\varepsilon}_k$ times the unsigned values, and the FBA price impact is their difference. Because the sign multiplies an *unsigned* deviation, not the signed one of (2.9), these values are close to zero by construction and are not compared with the CDA's (Section 6.7.1).

Amihud illiquidity. Let P_t^{c} be the last reference price observed in bucket t, carried forward across empty buckets, and $R_t = (P_t^{\text{c}} - P_{t-1}^{\text{c}})/P_{t-1}^{\text{c}}$ its return from the previous bucket. Then

$$\text{Amihud}_t = \frac{|R_t|}{\text{dollar volume}_t} \cdot 10^6 \tag{2.12}$$

for every bucket that traded a nonzero volume (Amihud 2002); it is missing otherwise, so a price move on zero volume is not recorded as infinite illiquidity.

Price Discovery and Market Quality (RQ2.2)

This group measures how each engine's price moves: how variable it is, how much it varies within one second, and how strongly it responds to order flow. Volatility is grouped here with price discovery rather than with the spread; other authors treat the spread as the main market-quality measure, and in this thesis it belongs to RQ2.1.

Realized volatility. Realized volatility in a bucket is the square root of the sum of squared returns, $\sqrt{\sum_i r_i^2}$, where r_i is the return between two successive observations of the reference price within the bucket. These observations are not equally spaced: for the CDA they follow the records the engine processes, and for the FBA they are successive clearing prices, one interval τ apart. The measure needs at least two prices in one bucket, which the FBA, clearing at most once per second, almost never has; it is therefore not used for the comparison, and is replaced by the next metric.

Matched-timestamp volatility. Let $\mathscr{T}$ be the set of buckets in which both engines traded, with VWAP_t^e the volume-weighted average price of engine e in bucket t. Over consecutive $t_i, t_{i+1} \in \mathscr{T}$ the two engines' log returns $r_i^e = \ln(\mathrm{VWAP}_{t_{i+1}}^e / \mathrm{VWAP}_{t_i}^e)$ are computed over the *same* instants for both. They are grouped by the calendar day of t_{i+1} and, with N_{day} the number of returns on that day, summarised as

$$\mathrm{vol}_{\mathrm{day}}^e = \sqrt{\frac{1}{N_{\mathrm{day}}} \sum_{i \in \mathrm{day}} (r_i^e)^2} \cdot 10^4 \text{ bps}, \tag{2.13}$$

a root *mean* square, so that a busier day is not mistaken for a more volatile one. A day with fewer than 5 matched returns is excluded for both engines. Because $\mathscr{T}$ needs a trade on both sides, the measure compares the engines on common ground; it is not a whole-month volatility estimate for either.

Intra-interval price dispersion. The coefficient of variation of the trade prices within a bucket, $\mathrm{std}(\{p_k\}) / \mathrm{mean}(\{p_k\}) \cdot 10^4$ bps (population standard deviation). It is zero for the FBA by construction whenever the bucket holds a single batch clear, since every trade of a batch is at p^*, so it serves as a check of the batch rule rather than as a comparison between the engines.

Kyle's lambda. Kyle's lambda measures how far the price moves per unit of net order flow (Kyle 1985). It is estimated as the slope of a regression of price change y on net signed flow f; the regressor, the response and the regression itself are set out in turn below.

The FBA poses a problem for the regressor f. Net order flow is normally read from the trades themselves, as buyer-initiated minus seller-initiated quantity, $\sum_k \varepsilon_k q_k$ (Section 2.1). In the CDA every trade has an aggressor, so this is possible. In the FBA it is not: all orders of a batch trade at once, no trade has an aggressor, and ε_k does not exist (Section 2.3.5). The FBA therefore has no net order flow of its own to put on the right-hand side of the regression.

A possible solution, and the one used here, is to measure the flow *outside* both engines and use that one series for both. The raw Hyperliquid records have no aggressor field, but they allow the ag-

gressor to be identified. Every fill on the real venue writes a record with status *filled*, the order's side and its remaining size, and each record carries the time elapsed since the order was submitted (`timestampDiff`, in milliseconds). A fill recorded at the order's own submission time, with an elapsed time of zero, means the order traded on arrival: it took liquidity and was the taker. A fill recorded later belongs to an order that had been resting, a maker. The filled quantity is the drop in the order's remaining size. This classification describes the venue's own continuous market; inside the FBA there are still no makers or takers, but each second of its order flow now has an external direction. Summing the taker fills by side gives the venue's net taker flow in each second i,

$$f_i = \frac{\text{taker buy quantity}_i - \text{taker sell quantity}_i}{1{,}000 \text{ SOL}}, \tag{2.14}$$

positive when buyers were the aggressors on balance. Three properties make it the right regressor. It is identical for both engines, because both replay the same orders in the same second. Its direction comes from fill records, not from a price; the only error left is a maker order filled within the first millisecond after it was posted, which the record's millisecond resolution would count as a taker. And it comes from neither of the two engines being compared: the simulator never reads these records, so the FBA is not measured against a quantity produced by its competitor, and the CDA's inflated simulated volume (Section D.5) does not enter.

With f_i fixed, each engine is regressed on it separately. The response is that engine's price change,

$$y_i^e(h) = \frac{P_{i+h}^e - P_{i-1}^e}{P_{i-1}^e} \cdot 10^4, \qquad h \in \{0,5\}\,\text{s}, \tag{2.15}$$

where P_i^e is engine e's VWAP in second i, carried forward over seconds in which it did not trade. Lambda is then the slope of a linear regression of this price change on the flow, estimated separately for each engine e and each horizon h:

$$y_i^e(h) = \lambda^e(h)\, f_i + u_i^e, \qquad \hat{\lambda}^e(h) = \frac{\sum_i f_i y_i^e(h)}{\sum_i f_i^2}, \tag{2.16}$$

where u_i^e is the part of the price change not explained by the flow. The regression is estimated by ordinary least squares *without an intercept*, that is, with $\mu = 0$ in Kyle's price rule $p = \mu + \lambda f$ (Section 3.2). The rule says that with no net flow the price should not move, so the line is forced through the origin, and the estimator is the ratio given in (2.16). The question each slope answers is the same for both engines: when the real market saw this much net buying in a second, how far did this mechanism's price move? The sample is every second with nonzero venue flow in which both y_i^{CDA} and y_i^{FBA} exist, pooled over the whole month, so both regressions use exactly the same observations and λ is in bps per 1,000 SOL. Because prices are carried forward, an engine need not trade in second i for y_i^e to exist, so this sample is larger than the set of seconds in which both engines traded that the other paired metrics use. In a second in which an engine did not trade, its same-second change ($h = 0$) is zero by construction.

To test whether the two slopes differ, a third regression is run on the difference between the engines' price changes,

$$y_i^{\mathrm{FBA}}(h) - y_i^{\mathrm{CDA}}(h) = \theta(h)\, f_i + v_i, \tag{2.17}$$

again through the origin. Because both engines share the same f_i and the same seconds, its slope is exactly the difference of the two lambdas, $\hat{\theta}(h) = \hat{\lambda}^{\mathrm{FBA}}(h) - \hat{\lambda}^{\mathrm{CDA}}(h)$. Consecutive seconds are correlated, so the standard error of $\hat{\theta}$ is a Newey–West error, with the lag length chosen by the same rule as for every other metric (Section 5.3); a $\hat{\theta}$ that is not significantly different from zero means the two prices respond to the same flow by the same amount.

For the CDA alone, a per-sweep estimate is also reported: within each second, the midpoint move 5 s after each sweep is regressed through the origin on the sweep's net executed quantity, in bps per SOL, and the per-second slopes are averaged over the month (Section 6.4). An average of per-second slopes weights a second with a few small sweeps as much as one with large sweeps, so it is not on the scale of the pooled slopes above and is not compared with them.

Execution (RQ2.3)

This group measures what happens to the orders themselves: how much volume is traded, how long an order waits for its first fill, and how much better than its own limit price it trades.

Executed volume, notional, VWAP and trade count. The executed volume of a bucket is the sum of q over its trades, and its notional is the sum of $p \cdot q$. Their ratio is the volume-weighted average price (VWAP), which is missing when no volume traded. The trade count is the number of trades in the bucket. A bucket without trades has volume, notional and trade count zero, so averages over the month include the seconds in which nothing traded.

Average time to execution. For every order that receives a fill, partial or complete, the time from its submission to its first fill is measured in seconds. The bucket value is the average over the orders submitted in that bucket.

Trader surplus. Trader surplus is the realized price improvement against each side's own limit, summed over the bucket's trades: the buy side of a trade at p_k contributes $\max(0, \pi - p_k)\, q_k$ and the sell side $\max(0, p_k - \pi)\, q_k$, where π is that side's limit price; a market order has no limit, so its side contributes nothing. For the FBA this is W of (2.7); for the CDA the same quantity measured at the resting order's price. For a trade between two limit orders the two contributions add up to $(\pi_{\mathrm{buy}} - \pi_{\mathrm{sell}})\, q_k$ whatever the trade price, so a difference between the engines in this metric comes from which orders were matched, or from trades involving a market order, not from the price as such (Section 4.2.3).

Engine Controls

Throughput and clearing latency. Throughput is the number of messages in the bucket divided by the computing time spent on them; clearing latency is that computing time divided by the number of timed operations, which is one per order for the CDA but one per *batch* for the FBA. Both are wall-clock measurements, so they are the only values that differ between repeated runs (Section D.4). Neither describes market quality. Because the replay runs on the timestamps stored in the data, not on the computer's clock (Section 5.1), computing speed cannot change any other metric; the two controls only show whether each engine would be fast enough to run in real time.

Chapter 3

Theoretical Foundations and Literature Review

This chapter sets out the bodies of theory the thesis relies on — latency arbitrage and discrete-time market design, adverse selection, market transparency, uniform-price auction theory and the measurement of market quality. The thesis's own argument that a predictable batch close turns price improvement into a boundary repricing race builds on these strands, but is a consequence of a design choice and is therefore developed in the design chapter (Section 4.2.3). Each strand is used later in one of two ways: it shapes a design block of Chapter 4, or it motivates a metric of the empirical comparison and predicts its sign. Each section states that link explicitly, and Section 3.6 collects them in one table. Notation and the formal definition of every metric are in Chapter 2.

Discrete-Time Market Design

The Mechanics of Latency Arbitrage

The argument of Budish et al. (2015) rests on a single structural observation, not on irrationality or market power: in a continuous market, a public information event creates a one-off, risk-free arbitrage opportunity, and that opportunity goes to whoever is fastest.

Consider a liquidity provider quoting a two-sided market in an asset whose fair value is tracked by a correlated, continuously observable signal. When the signal jumps, every participant receives the same information at essentially the same instant: the liquidity provider wants to cancel its stale quote, and every other fast participant wants to trade against it. The engine processes both messages in arrival order, so whether the quote is cancelled or picked off depends on network path length, hardware and code efficiency, not on anything informational. Even if every participant had identical hardware and network paths, the outcome would only become a lottery: with N participants racing for the same stale quote, the liquidity provider's cancellation arrives first with a probability of about $1/N$, so the quote is still picked off in roughly $(N-1)/N$ of such events.

Two consequences follow. The opportunity is mechanically recurrent, appearing every time the public signal moves — thousands of times a day in a liquid instrument. And the value of being marginally faster does not decline as everybody invests in speed: since only the winner is paid, the equilibrium is a rent-dissipating race in which expenditure on latency is bounded by the value of the opportunities, which persist however fast the market becomes. Speed is a purely relative good.

The cost is not borne by the racers. A liquidity provider that expects to lose a fraction of its volume to picked-off quotes must recover that loss from the trades it does not lose, shifting the burden onto liquidity demanders as a wider spread — a tax on liquidity in the words of Budish et al. (2015).

Empirical Magnitude

Aquilina et al. (2022) measure this directly, using London Stock Exchange message-level data (including messages never executed) to identify latency races as clusters of near-simultaneous attempts to take or cancel the same quote. Races are extremely frequent and short — far shorter than any batch interval measured in milliseconds; a large share of the volume at the exact moment of a price change is race volume; and the aggregate transfer, small relative to total volume but large in absolute terms, concentrates in the effective spread paid by liquidity demanders. Removing the race therefore removes a component of the spread. This is why the effective spread, and its split into realized spread and price impact (Section 2.5.1), is the natural cost measure of RQ2.1; it needs a trade direction, which only the CDA provides (Section 6.7.1).

Batching as the Response

Frequent batch auctions change two rules at once, and the two do different work. *Discrete time* removes the informational value of within-interval arrival order: if orders are collected over $[t, t+\tau)$ and matched only at $t+\tau$, any two orders inside the window are symmetric with respect to time as far as the price is concerned (whether they stay symmetric in allocation depends on the rationing rule, Section 4.5.3). *Uniform pricing* removes the value of queue position within the matched set: if all executions occur at one price p^*, no participant obtains a better price by being earlier. Either rule alone is insufficient — discrete time with discriminatory pricing would still reward earlier submission, and uniform pricing in continuous time is not well defined. Together they imply that speed pays only for crossing an interval boundary, getting into the current batch rather than the next: the value of a marginal nanosecond falls from "wins the race" to "matters only in the vanishingly rare case that it changes which batch an order lands in". In the model of Budish et al. (2015), liquidity providers then no longer need to price sniping risk into their quotes and spreads narrow. Whether such sniping occurs in the SOL perpetual studied here cannot be measured with the data used, which contain no identifiable latency races; Hyperliquid's own processing of cancellations before new orders already limits it (Section 5.2.1). The residual cost is delay, an average of $\tau/2$ if orders arrive uniformly, so the choice of τ trades speed neutralisation against delay imposed on everybody. The benefit is not free of side effects: Baldauf and Mollner (2020) show that competition among fast traders affects both liquidity and the incentive to acquire information, so a measure that curbs the race can lower spreads and still change how much information prices contain. This conclusion assumes that waiting inside the window gains nothing; Section 4.2.3 shows that with a predictable close and a visible batch it does, and that speed then regains value near the boundary.

Both halves of this trade-off are followed up later. The choice of τ and of its closing rule is design Block 1 (Section 4.2). The benefit, a lower adverse-selection cost, would be measured by the spread decomposition, which the replay can build only for the CDA, and indirectly by Amihud illiquidity

and Kyle's lambda (RQ2.1, RQ2.2); the cost, delay, is measured by the time to execution (RQ2.3, Section 2.5.3).

Information Asymmetry and Adverse Selection

Latency arbitrage is one specific mechanism through which a liquidity provider loses money to better-informed counterparties; the general theory of that loss predates high-frequency trading by three decades and is what makes the spread a meaningful economic object.

The Spread as an Information Premium

Glosten and Milgrom (1985) model a competitive, risk-neutral market maker facing a mixed population of informed and uninformed traders it cannot tell apart. Because a buy order is more likely to come from an informed trader when the asset is undervalued, every order carries information, so quotes must be conditionally fair given the direction of the order expected — a bid–ask spread that exists purely as an information premium, even with no inventory risk or processing cost. The spread widens with the proportion of informed traders and the precision of their information, and narrows when the market maker can identify a subset of flow as uninformed. This is the theoretical foundation of order flow segregation: separating benign flow lets a liquidity provider quote tighter against it without cross-subsidising losses on the rest. It underlies design Block 3 (Section 4.4). The simulated engine clears a single pool (Table 5.1), so segregation is analysed in the design part but not measured.

Strategic Informed Trading and Market Depth

Kyle (1985) approaches the same problem from the informed trader's side. A single informed agent chooses how much to trade, knowing that trading more reveals more of its information. Noise traders provide cover for it, and a competitive market maker sets the price as the expected value of the asset given the total order flow. The equilibrium price function is linear, $p = \mu + \lambda f$, where μ is the prior expected value, f is aggregate order flow and λ measures price impact per unit of flow; its reciprocal, market depth, is the volume required to move the price by one unit. Kyle's model is a batch model — the market maker observes aggregate flow and sets a single price, the structure of a uniform-price call auction — so the FBA literature can be read as asking what happens when a real market is organised the way Kyle already assumes. λ also supplies the empirical bridge: price impact is the operational measure of adverse selection. The thesis estimates λ for each engine as the slope of price change on order flow (Section 2.5.2); a lower λ would mean prices react less to one-sided flow.

Decomposing the Spread

The adverse selection premium is not directly observable; only the total cost of a trade is. The standard strategy is to decompose the effective spread into a realized spread and a price impact component (Hasbrouck 1993; Hendershott et al. 2011), as defined in (2.9)–(2.11), with $S_k^{\text{eff}} = S_k^{\text{real}}(\Delta) + \text{PI}_k(\Delta)$, where Δ is the markout horizon, the time after the trade at which the midpoint is read again. The effective spread is what the liquidity demander paid relative to the midpoint. The realized spread is

what the provider still holds once the market has revised its valuation, the gross compensation for supplying liquidity. The price impact is the permanent revision of the midpoint in the direction of the trade, the part lost to the information the order carried.

From the liquidity demander's side, the effective spread is the trade's slippage against the midpoint. Slippage is not a fixed property of a trade: it depends on the horizon Δ at which it is measured, on the trade's size and on volatility. A large order faces a trade-off between executing quickly, with a larger price impact, and executing slowly in smaller pieces, with a smaller impact but more exposure to price moves in the meantime. The replay keeps every order's size and timing as submitted, so this trade-off is not studied here.

When a public signal moves, discrete uniform-price clearing removes the latency-arbitrage mechanism, since the aggressive order and the maker's cancellation are processed together and execution occurs at a price that already reflects orders submitted in response to the signal. It does not remove adverse selection in the theoretical sense — a genuinely informed trader can still price aggressively into a batch (Eibelshäuser and Smetak 2022) — so the prediction is a reduction in price impact, not its elimination. A fall in effective spread driven by price impact is therefore the result theory predicts; a fall driven by realized spread would mean providers earn less with the informational component untouched, and a fall in price impact with no change in effective spread would mean the benefit is captured by providers rather than passed on. Telling these three readings apart needs all three components for both mechanisms. The replay supplies them only for the CDA, because FBA trades have no direction (Section 6.7.1), so for the FBA the question stays open.

Market Transparency Theory

Madhavan (1996) defines transparency as participants' ability to observe information about the trading process, and shows more of it is not unambiguously better: greater pre-trade transparency helps prices aggregate information and reduces uncertainty about market state, but also exposes the trading intent of participants transacting in size, and can increase volatility and reduce liquidity in some configurations. Consolidating with O'Hara (1995), disclosure design is a genuine trade-off, not a monotone improvement.

In an FBA the question is narrower: what is disclosed during the accumulation phase, before the auction clears? Moving toward opacity protects submitted orders from being exploited but degrades the information available for pricing, and classical auction theory predicts that participants respond to uncertainty about the clearing price by shading their bids (Klemperer 1999). In a uniform-price setting shading is not merely a transfer: shaded limit prices are less likely to cross, so it can reduce the volume that clears.

The trade-off assumes the operator can choose where to sit among these options. Decentralised venues that settle through a public mempool are the limiting case where it cannot: a transaction waiting there is visible to everyone before it executes, so such markets are fully open by default without the benefit that regime is supposed to buy. The value that can be extracted by reordering, inserting or excluding

pending transactions is known as miner, now maximal, extractable value (MEV) (Daian et al. 2020). Not every blockchain venue is in this position: hybrid designs match orders off chain and settle on chain, and Hyperliquid orders transactions in its own consensus layer (Section 5.2.1), so the exposure depends on the architecture and not on the use of a blockchain as such. Disclosure normally exposes the trading intent of the order submitter (Madhavan 1996), and the informational disadvantage this creates normally falls on the market maker posting a quote (Glosten and Milgrom 1985). A mempool flips both of these. What is exposed is not a resting quote but an instruction that has not executed yet, and the party exposed by it is not the quoter but the one demanding liquidity, the submitter itself. An observer can buy ahead of a pending swap and sell into the price it moves, sandwiching the submitter (Daian et al. 2020). Batch clearing removes this by construction: if every order in the interval settles at one uniform price, position within the batch carries no value for the price an order obtains.

This strand shapes design Block 2 (Section 4.3). It cannot be tested in the replay: the replayed orders never see what the engine publishes, so the disclosure regime cannot change any result (Table 5.1), and bid shading is listed among the behavioural predictions out of reach (Section 7.2).

Auction Theory and Uniform-Price Clearing

In a uniform-price auction all winning participants transact at the same market-clearing price, independent of the prices they submitted. A participant's own bid determines *whether* it trades but, except for the marginal order, not *at what price*. Misreporting has no upside: bidding below one's valuation only risks losing an acceptable trade, bidding above only risks winning an unacceptable one, so for a price-taking participant with single-unit demand truthful revelation is approximately optimal, a property inherited from Vickrey (1961). This is the formal basis for the claim that batching substitutes competition in price for competition in speed: if submitted limits are approximately truthful, the clearing price aggregates valuations rather than reaction times. For a price-taking participant, visibility of the accumulated batch does not disturb this — a participant who can anticipate the clearing price pays it regardless of the limit it submits above it — but it does change *when* orders are submitted, since observing the indicative price gives a participant reason to withhold until shortly before the batch closes.

The qualification carried by "approximately" depends jointly on pivotality and size. A participant too small to move the crossing point controls only its own inclusion, so shading saves nothing and truthful bidding is optimal without qualification. A pivotal participant with single-unit demand does pin the clearing price with its own limit, but the marginal unit carries almost no surplus, so the incentive to shade is negligible. The case that matters is the pivotal participant with multi-unit demand: its last unit sets the price for every unit it buys, so shading the marginal bid reduces the cost of the entire inframarginal block while risking only the one unit barely worth trading. The saving scales with size, the loss does not, and the incentive to misreport is restored; Klemperer (1999) surveys this literature. Pivotality is relative to the liquidity accumulated within one batch, not the market as a whole, so the strength of the truthfulness property is itself a function of interval length: shorter intervals produce

thinner batches, where an order that would be atomistic in a long window can be price-setting in a short one.

For every executed non-marginal order, uniform pricing also produces price improvement relative to the submitted limit: a buyer willing to pay π_k who clears at $p^* < \pi_k$ captures $(\pi_k - p^*)x_k$ of surplus that, under continuous discriminatory execution at the resting price, would in general have accrued differently. Aggregating this across executed orders gives the realized trader surplus metric (Section 2.5.3). Section 4.2.3 shows that for a given set of executed orders the total of this surplus does not depend on the price: the price only decides how it is split between buyers and sellers, so price improvement on one side is a transfer from the other, not a gain.

These results inform design Block 4, the clearing and rationing rule (Section 4.5). The link between interval length and pivotality also has an empirical counterpart: thin batches show up as batches that do not cross at all (Section 6.5).

Measurement of Market Quality: Motivating the Metrics

The empirical comparison relies on the standard microstructure toolkit, and each metric of Section 2.5 is chosen because one of the theories above makes a prediction about it.

Liquidity and transaction costs (RQ2.1). Quoted spread and depth describe the cost a hypothetical trade would face: if batching pools liquidity, as its proponents argue, the FBA should be at least as deep. Effective spread describes what an actual trade cost, and realized spread and price impact decompose that cost into the part retained by the liquidity provider and the part lost to adverse selection (Hasbrouck 1993); Section 3.2.3 gives the prediction for each. Amihud (2002) provides a low-frequency summary of price impact per unit of volume, which can be computed for both mechanisms because it needs neither a book nor a trade direction; it does need a reference price, which differs between the two engines (Section 6.7.2).

Price discovery (RQ2.2). Batching aggregates orders before setting a price, so it should change how much prices move and how they respond to order flow. Kyle's lambda (Section 3.2) measures the price response to flow directly, intra-interval dispersion checks the mechanical promise of uniform pricing, and volatility measures how much the price varies. Hasbrouck (1995) supplies a framework for attributing price discovery across venues, but it needs venues that trade the same asset simultaneously and influence each other; the two engines here never interact, so the thesis compares their price series on a common time grid instead (matched-timestamp volatility).

Execution and timing (RQ2.3). The cost of batching is delay (Section 3.1), measured by the time to execution. Uniform pricing promises price improvement (Section 3.4), measured by trader surplus. Short intervals produce thin batches, which show up as batches that clear no trade at all; their share is reported as a descriptive statistic (Section 6.1). The boundary argument of the design chapter (Section 4.2.3) predicts that, under a fixed and visible close, orders bunch near the batch close; testing it needs participants who can see that close, which a replay of orders submitted to a continuous market cannot provide (Section 7.2). Executed volume and trade count have no theoretical prediction of their

own: they check that the two engines trade the same flow, so that any other difference can be attributed to the mechanism.

Three of these metrics need adapting for a batch mechanism (Section 2.5.1). Measures that reference the midpoint at the moment of the trade use the previous clearing price instead, which is at least one interval old. The quoted spread is read from the orders left after a clear. And the spread decomposition needs the direction ε_k of each trade, which an FBA trade does not have (Section 2.3); this is the one adaptation that fails (Section 6.7.1).

Finally, the experimental convention is that of Gode and Sunder (1993): hold the induced order flow constant and vary only the mechanism. Their demonstration that institutions can produce allocative efficiency largely independently of trader rationality also licenses replay-based simulation, since much of what a mechanism does is mechanical rather than behavioural; LeBaron (2006) surveys the agent-based alternative in which behaviour is modelled and allowed to adapt, and Wah and Wellman (2013) apply it to latency arbitrage, comparing continuous and periodic call markets with simulated traders.

From Theory to Design and Measurement

Table 3.1 summarises how each strand of this chapter, and the boundary argument of the design chapter that builds on them, is used later: the design block it shapes in Chapter 4, the metric that measures it in Chapter 6, and what this thesis can test. Two strands, transparency and the boundary repricing race, cannot be tested, because both need participants who react to the mechanism and the replay holds behaviour fixed. Segregation (Block 3) is analysed in the design part but not measured, because the simulated engine clears a single pool.

Table 3.1: Each theoretical strand, its prediction for an FBA, the design block it shapes and the metric that measures it.

Strand	Prediction for the FBA	Design block	Metric (RQ)
Latency arbitrage (3.1)	no speed race within τ; lower adverse-selection cost; delay of about $\tau/2$	1 Interval	spread decomposition (2.1, CDA only); time to execution (2.3)
Adverse selection (3.2)	lower price impact, not zero; price reacts less to flow	3 Segregation	price impact (2.1, CDA only); Kyle's lambda (2.2)
Transparency (3.3)	disclosure trades exploitation against bid shading	2 Disclosure	not testable in a replay
Auction theory (3.4)	near-truthful limits; price improvement; thin batches at short τ	4 Clearing	trader surplus (2.3); share of batches with no trade (descriptive)
Market quality (3.5)	batching pools liquidity and dampens price moves	—	depth, Amihud (2.1); volatility (2.2)
Boundary repricing race (4.2.3)	with a fixed, visible close, submissions bunch at the close; speed regains value there	1, 2, 4	not testable in a replay

Chapter 4

Design of a Frequent Batch Auction Engine

This chapter breaks a batch auction engine into five design blocks, lists the options available in each block, and explains from theory how participants should be expected to behave under each option. The most consequential of these behaviours, the boundary repricing race that a predictable close produces, is developed in full as part of Block 1 (Section 4.2.3), and Section 4.7 closes the chapter with a summary of every option and its consequences. The chapter uses the notation of Chapter 2, in particular the clearing mechanism defined in Section 2.3.

The Engine as a Set of Design Blocks

A matching engine takes a stream of timestamped messages and produces trades. A continuous engine handles each message completely before reading the next. A batch engine first collects messages for a while, and then processes the whole set at once. This one change forces five decisions that a continuous engine never has to make, because serial processing settles them automatically:

1. how long the collection period τ lasts, and whether its end can be predicted;
2. what participants can see while orders are being collected;
3. whether all orders compete in one pool;
4. how the trading price is computed and how the orders on the longer side are served;
5. how the trades are settled and what happens to orders that did not trade.

Figure 4.1 shows how these blocks fit together in one batch cycle.

Block 1: The Batch Interval

The Length of τ

The interval length decides how much of the speed advantage disappears. A speed difference of δ between two participants only matters if it changes which batch their orders land in. If orders arrive at random times, this happens with probability δ/τ. With δ of a few microseconds — the time scale on which the races of Aquilina et al. (2022) are decided — and τ around 100 milliseconds, the probability is only 10^{-4} to 10^{-5}. Speed is not banned; it stops paying.

The price is delay. Every participant waits on average $\tau/2$ before its order can trade, and up to τ in the worst case. This is harmless for an investor thinking in minutes, but it hurts a market maker holding

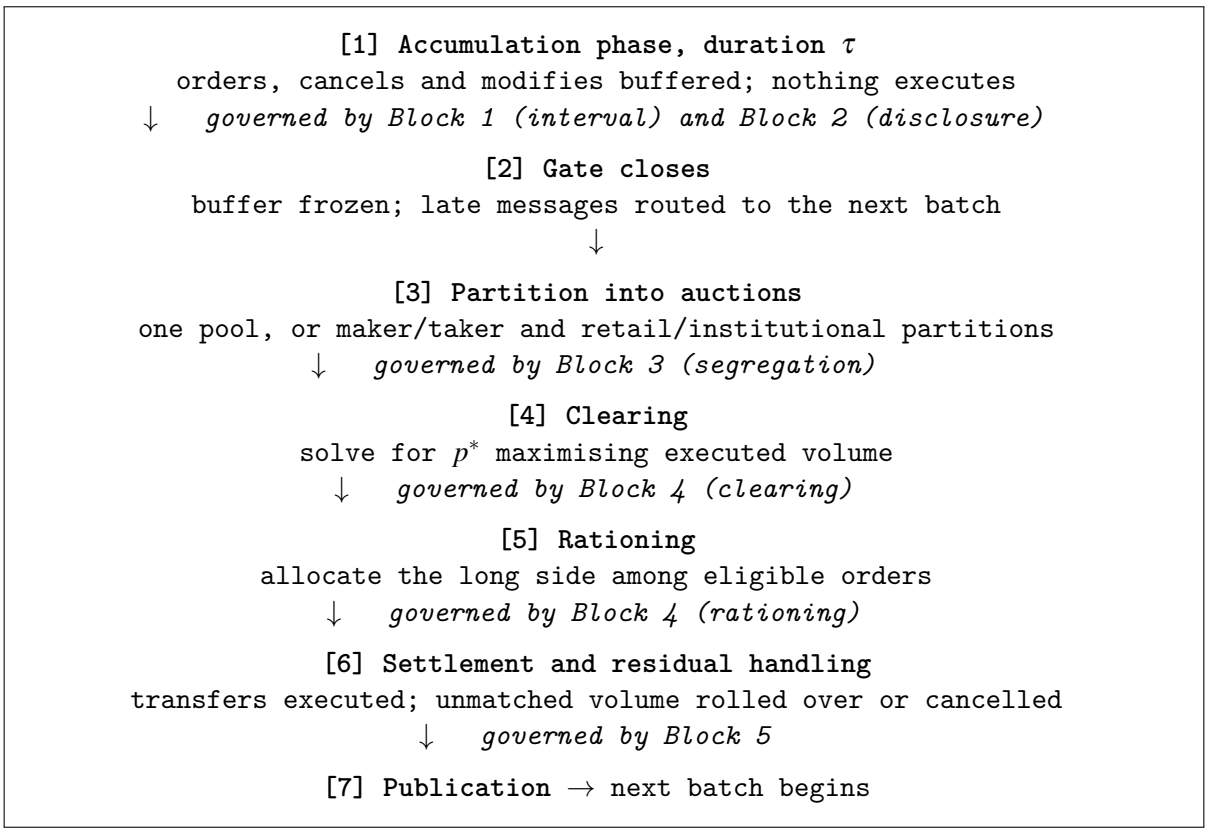


Figure 4.1: One batch cycle of a frequent batch auction engine, annotated with the design block that governs each stage.

inventory, because the price can move during the interval and the order cannot be withdrawn in time. The interval should therefore be long enough to remove speed differences and short enough to keep prices current. The right value depends on the instrument:

- **Lower bound:** τ must be clearly longer than the differences in network delay between participants. Otherwise faster participants still reach the current batch more often and a weaker speed race remains.
- **Upper bound:** τ must be short enough for new information to reach prices quickly. Zoican et al. (2021) show that more frequent auctions do not always improve market quality, and that very long intervals let orders become stale.
- **Liquidity:** in a thinly traded instrument a short interval may collect no matching orders at all, leaving batches empty, so the market behaves like a continuous one with an added delay.

Fixed versus Randomised Closing

If everyone knows when the batch closes, that moment becomes a target. A fast participant can wait until the last possible moment, see as much information as possible, and only then submit, reprice or cancel. This is the *boundary repricing race* (sometimes called boundary sniping). It is the same speed race as in continuous trading, moved to the end of the interval; what is won is a better share of the

surplus of a batch whose contents are now partly known, rather than a single stale quote. Section 4.2.3 derives it formally; the options below differ in how exposed they are to it.

- **Fixed closing time.** Simple, easy to audit, and compatible with clock-aligned reporting, but easy for participants to plan around and fully exposed to the boundary repricing race. This is the option the simulated engine implements (Table 5.1).
- **Random closing time.** The batch closes at a time drawn from a publicly announced range, for example uniformly between $\tau_{\min}$ and $\tau_{\max}$. Waiting until the last moment now risks missing the batch, so it becomes a gamble instead of the best strategy. The drawbacks are that participants cannot time their orders precisely and that the random draw must be verifiable, which is hard in a decentralised setting.
- **Closing triggered by volume or events.** The batch closes when enough volume has accumulated or a market event occurs. This adapts the interval to activity, but because the closing depends on what participants do, it can be manipulated: a participant can trigger the close by submitting volume.
- **Random end after a minimum duration.** The batch runs for a fixed minimum time and then ends at a random moment within a short window. Xetra's closing auction works this way: after a minimum call phase it ends at a random time, precisely to make the price harder to manipulate (Deutsche Börse 2026). The schedule stays predictable to within that short window, while the exact final moment, the one a late submitter would target, is not.

Consequence of the Time Window: Price Improvement and the Boundary Repricing Race

Choosing a time window, and in particular a window whose end everyone can predict, has a consequence that the option list above only names. This subsection derives it. The argument is, to the author's knowledge, not stated in this form in the existing literature. Its components are individually familiar: the residual value of low latency near a batch boundary is established by Schlegel (2023), the tendency of sophisticated traders to submit late in transparent call auctions is documented by Daures et al. (2026), and the strategic consequences of the clearing-price selection rule in uniform-price auctions are studied by Burkett and Woodward (2020). What has not been made is the connection between them. The frequent batch auction literature treats the price improvement that uniform clearing confers on inframarginal orders as a benefit of the design; this subsection argues that the treatment rests on a category error. Price improvement measured on one side of the book is not a welfare quantity but a transfer, and the rule that determines its size is manipulable by the timing of order submission. A batch with a predictable close therefore does not remove the incentive to compete on speed: it converts a race for priority into a race for proximity to the batch boundary, independently of the informational motives usually invoked to explain late submission.

The argument considers a single batch and uses the objects of Section 2.3: the schedules $D(p)$ and $S(p)$, the volume-maximising set $\mathscr{P}^* = [\underline{p}, \overline{p}]$, the clearing price $p^* = \sigma(\mathscr{P}^*)$, the executed quantities x_k with $\sum_{k\in\mathbb{B}} x_k = \sum_{k\in\mathbb{A}} x_k = V^*$, and the buyer and seller surplus $W^{\mathbb{B}}(p)$ and $W^{\mathbb{A}}(p)$ of (2.7).

The first step is that total surplus does not depend on which clearing price is chosen. For any $p \in \mathscr{P}^*$ with allocation x held fixed,

$$W = W^{\mathbb{B}}(p) + W^{\mathbb{A}}(p) = \sum_{k\in\mathbb{B}} \pi_k x_k - \sum_{k\in\mathbb{A}} \pi_k x_k, \tag{4.1}$$

which does not depend on p.

Substituting the definitions of $W^{\mathbb{B}}$ and $W^{\mathbb{A}}$ from (2.7) into this sum, the terms in p enter as $p\,(\sum_{\mathbb{A}} x_k - \sum_{\mathbb{B}} x_k)$, which vanishes by flow conservation. Two consequences follow immediately. First, $\partial W^{\mathbb{B}}/\partial p = -V^*$ and $\partial W^{\mathbb{A}}/\partial p = +V^*$: every unit of price improvement conferred on one side is subtracted from the other. Second, if one mechanism shows more buy-side price improvement than another, this alone does not show that it created more value: total gains from trade are fixed by the allocation, and the clearing price only divides them. The metric must therefore be read as a distributive statistic, reported for both sides jointly. If $\mathscr{P}^*$ were a singleton, the division would be pinned down by the submitted limits and there would be nothing to contest; because it is frequently an interval, especially in thin batches, the surplus split depends on a rule, and any participant able to influence the position of $\mathscr{P}^*$ or of σ within it has an incentive to do so.

The Last-Mover Advantage

Suppose the engine operates under the fully open disclosure regime of Block 2 (Table 4.1), so that a participant submitting at time t observes the schedules formed by all orders with $t_k < t$.

Late submission weakly dominates. Suppose a participant can see the orders already submitted by others, and hence the residual supply or demand it faces, and that the selection rule σ responds monotonically to submitted limit prices, so that a participant can shift the clearing price within $\mathscr{P}^*$ by adjusting its own limit. The participant's best response is then to set its limit so that the clearing price moves to the edge of $\mathscr{P}^*$ most favourable to it,

$$p^* \to \underline{p} \text{ for a buyer}, \qquad p^* \to \overline{p} \text{ for a seller}, \tag{4.2}$$

while still keeping its order executed. Because this manoeuvre requires knowing the other orders, a participant's surplus can only grow with the information it has when it submits. Submitting later is therefore never worse than submitting earlier, and it is strictly better whenever the extra information changes the participant's optimal limit.

The argument is the standard one for sequential-move bargaining over a fixed surplus, applied to the division established in (4.1). A seller who observes a resting buy limit of $\pi_{\mathbb{B}}$ knows that any ask weakly below it will execute, and therefore submits just below $\pi_{\mathbb{B}}$ rather than at its own valuation, capturing the interval. Truthfulness (Section 3.4) is not violated in its own terms, since it is stated for a participant that cannot move the clearing price; what disclosure does is make the boundaries of $\mathscr{P}^*$

computable, and hence make deviation actionable for any participant whose order is large enough to reach them.

This identifies the source of the incentive to wait. When several prices clear the same maximum volume, the choice among them is not neutral: it transfers surplus between buyers and sellers. Under a monotone σ, the participant who moves last can capture this transfer by pushing the price to the boundary of $\mathscr{P}^*$ in its own favour. The prize goes to whoever submits closest to the deadline with the fullest view of the book: a reward for speed and timing rather than for providing liquidity or information.

A simple example shows the mechanism. A batch holds one buy order for 1 SOL with a limit of 100.10 USD. A seller values its 1 SOL at 100.00 USD and submits last, seeing the resting buy order. If it submits at its own valuation, every price from 100.00 to 100.10 clears the trade, and the selection rule decides how the 0.10 USD of surplus is split, for example 100.05 for an even split. If it instead submits at 100.10, the volume-maximising set shrinks to that single price: the trade and the total surplus are unchanged, but the seller now takes all 0.10 USD and the buyer's price improvement falls to zero. Only the seller's position as the last mover with a view of the book made this possible.

Two conditions expose a participant to being the donating side of the transfer: submitting without observing the accumulated batch, as an uninformed participant does, and submitting early without repricing. Since the transfer is zero-sum by (4.1), every participant exposed in this way is matched by one with an incentive to exploit it. An order resting in an open batch is thus a free option written to every subsequent submitter in the window. This is the same option Copeland and Galai (1983) describe for a resting limit order in a continuous book: there, it is written to traders reacting to news; here, the same option is written to traders reacting to the accumulating batch itself.

Cancellation, Repricing and the Deadline

The natural defence against the last-mover advantage (4.2) is to withdraw or reprice an order that has become adversely positioned. This is where the speed race re-enters.

Latency monotonicity. Let participant i face message latency $\ell_i > 0$ and let the batch close deterministically at τ, so that the window is $[0, \tau)$ as in Section 2.3. A message dispatched at time t is included in the batch if and only if

$$t + \ell_i < \tau, \tag{4.3}$$

so the latest observable state on which i can act is that prevailing just before $\tau - \ell_i$. The information set available to i at the moment of its final decision is therefore ordered by ℓ_i, and expected surplus is non-increasing in ℓ_i.

The structure of the advantage differs from the continuous case but its source does not. In a continuous book, lower latency wins a contest for priority; in a batch, priority within the window has no effect on the price by construction (and affects the allocation only if the rationing rule rewards it, Section 4.5.3), and lower latency instead buys a later decision point. The fast trader observes more of the batch before committing, can reprice on more recent information, and can place its final message

closer to the boundary while retaining the same probability of inclusion. Speed is not neutralised; its payoff is relocated from ordering to deadline precision. That some speed advantage survives batching is consistent with the experimental evidence of Aldrich and López Vargas (2020), who find that investment in speed falls but does not disappear under batch clearing, and with the model of Zoican et al. (2021).

Three remarks bound the mechanism. Prohibiting cancellation within the batch does not close the channel: a participant with two-sided access can still neutralise an adverse position with an offsetting order, a synthetic cancellation that needs no cancel message at all (true enforcement would require account-level self-match prevention). Banning cancellation outright would, if anything, strengthen the last-mover advantage (4.2), by raising the cost of correcting an early submission instead of removing it. And the channel is disclosure-conditional: the informational component of the last-mover advantage requires an observable accumulating batch and is absent under a sealed-bid regime.

Equilibrium Implication and Design Responses

If late submission weakly dominates and lateness is bounded only by ℓ_i, submissions concentrate near the boundary and are ordered by latency. In the limit the accumulation phase carries no orders and therefore no information, and the auction degenerates into a sealed tender conducted in the final microseconds of the window: the batch retains its length nominally while losing the property the length was chosen to deliver.

Three design responses follow, one in this block and two in later ones. Randomising the close time (the random closing option above) makes waiting costly, since an order dispatched at t is included only if the drawn close exceeds $t + \ell_i$; the participant then solves an interior optimum trading the value of information against the risk of exclusion, which justifies randomisation independently of the sniping argument usually advanced for it. Restricting disclosure to an aggregate imbalance or to nothing (Block 2, Section 4.3) removes the informational component of the last-mover advantage while leaving bid shading, so the choice is between two costs rather than between one cost and none. Finally, and least discussed in the literature, σ can be made insensitive to the marginal submitted limits (Block 4, Section 4.5), for instance by choosing the point of $\mathscr{P}^*$ closest to an external reference price. This removes the lever without reducing gains from trade, since by (4.1) the choice of price within $\mathscr{P}^*$ does not change total surplus for a given allocation.

The prediction that submissions bunch near the close needs participants who can see the close and choose when to submit. It cannot be tested in the replay of this thesis, because the replayed orders were submitted to a continuous market and never timed against a batch boundary (Section 5.4.1).

Block 2: Information Disclosure

The disclosure regime decides what participants can see when they submit. There are four regimes, from publishing everything to publishing nothing (Table 4.1). More secrecy protects submitted orders from being exploited, but gives participants less price information and encourages *bid shading*, meaning deliberately submitting a less aggressive price than one's true valuation.

Table 4.1: Disclosure regimes during the accumulation phase and their predicted behavioural consequences.

Regime	What is published during accumulation	Predicted behaviour
Fully open book	All accumulated orders, continuously updated	Strong within-batch price discovery; early submitters expose intent and are exploited by later ones; incentive to submit as late as possible, which compounds the boundary repricing race (Section 4.2.3)
Indicative clearing price	The price at which the batch would clear if it closed now	Coordination on a common price estimate without exposing individual orders; some intent leakage through price movement; the regime of most equity opening and closing auctions
Aggregate imbalance	Net excess demand or supply, no price schedule	Signals direction of pressure and attracts offsetting liquidity without revealing individual limits; weaker price anchoring
Sealed bid	Nothing until the auction clears	No exploitation of submitted orders and no incentive to delay for informational reasons; clearing price uncertainty induces bid shading, which can reduce cleared volume

Disclosure and interval design are linked. An open book gives participants a reason to submit late, which makes a random closing time more valuable. A sealed-bid design removes most of that reason and makes a fixed closing time easier to defend. Secrecy also does not remove information leakage; it only moves it. Under sealed bidding, the price and volume of one batch become public before the next one closes, and as τ becomes shorter this sequence of published prices approaches a continuous price signal.

Block 3: Order Flow Segregation

The Unified Pool and Segregation by Participant Class

In the default design all orders enter one auction. This is the simplest design to understand and regulate. Its drawback is that a liquidity provider who cannot tell informed from uninformed traders must quote a price that reflects the average of the pool, so uninformed participants end up paying for the losses that informed participants cause. Splitting flow by participant class, typically retail against institutional, lets the provider quote a tighter price to the group it believes is uninformed. This is the logic behind retail wholesaling, the purchase of retail order flow, in equity markets (Easley et al. 1996), and it carries over to batches through one auction per group, or through eligibility rules for each group. The difficulty is classification. The split is only as good as the label, labels can be gamed by sending institutional orders through retail channels, and participants in different groups receive different prices for the same asset at the same moment, which raises fairness and best-execution concerns.

Intent-Based Segregation: The Dual Flow Batch Auction

Jump Trading's Dual Flow Batch Auction (DFBA) (Jump Crypto 2025) splits orders by *intent* rather than by who submits them. Every order is labelled either maker flow (supplying liquidity) or taker flow (demanding it, and seeking prompt execution). The proposal deliberately leaves open how the label is assigned and names three ways: the exchange designates certain participants as market makers, whose orders are maker flow; anyone may submit maker orders, but access to taker flow is controlled by the exchange; or each user labels its own orders, with taker as the default. Each batch then runs two separate auctions, maker buys against taker sells and maker sells against taker buys, both at one uniform price. Market makers never trade against each other, which removes trades between two liquidity providers that add volume but give liquidity to nobody who wanted it. A fast participant also cannot use taker orders to pick off maker orders inside the batch, because there is no queue to win.

The design has its own costs. Splitting the pool splits the liquidity, so each auction clears against a thinner book. Intent labels must be enforced and not merely declared, otherwise a participant could label itself a maker in one batch and a taker in the next and collect the benefits of both. And the split assumes that "maker" and "taker" mean something, which in a mechanism that clears everything at the same instant is a rule imposed by the engine, not a fact about who arrived first.

The Maker–Taker Distinction and Fee Design

This last point leads to a problem the literature on batch auctions has not solved. In a continuous book the maker is the resting order and the taker is the order that crosses it, so the distinction depends on arrival order. That definition is not available when all matched orders arrive in the same interval and trade at the same time and price. Yet fees, rebates and a large part of exchange revenue depend on it. This thesis distinguishes four ways to redefine it, none fully satisfactory; the first follows the DFBA, the other three are proposed here:

1. **By declared intent**, as in the DFBA. Simple to operate, but self-declared unless a separate eligibility rule enforces it.

2. **By price aggressiveness.** The label depends on how far the order's limit lies from the clearing price, a matter of degree. Orders whose limit price is far from the clearing price on the passive side are makers; orders that would have crossed a wide range of prices, such as market orders, are takers. This can be observed and is hard to game without changing one's real exposure, but it penalises participants for wanting certainty of execution.

3. **By marginality.** The label depends only on whether the order is at the clearing price of its batch, a yes-or-no question. The orders whose limit equals the clearing price, and which therefore set it, are takers; orders priced strictly better than it are makers. This is attractive in theory, since the marginal order determines the price, but the fee a participant pays then depends on what everyone else does in the batch, so it cannot be known in advance.

4. **By persistence across batches.** The label depends on time, not on price. An order filled in the batch in which it was submitted is a taker; an order that was not filled and has rolled over through one or more batches, resting as it would in a continuous book, is a maker. This rewards liquidity that stays available, which is what the rebate is supposed to pay for, and it is the only one of the four that measures something lasting rather than a single moment.

Criteria 2 and 3 both look at the order's price within a single batch, criterion 2 by how far the limit lies from the clearing price and criterion 3 only by whether it lies at it. Criterion 4 looks at the order's history across batches and is the only one that does not depend on the clearing price.

The choice matters beyond accounting. A rebate is an incentive, so whichever definition the engine adopts is the behaviour it ends up paying for.

Block 4: Matching, Clearing and Rationing

Implementing the Clearing Rule

Prices can only take values on a fixed grid of allowed steps, so the set $\mathscr{P}^*$ of prices that maximise traded volume is a short run of neighbouring steps. The engine finds it in one pass over the submitted limit prices and then picks one of them, $p^* = \sigma(\mathscr{P}^*)$ (2.6), by the four-step tie-break cascade defined in Section 2.3.

The order of those steps matters. Minimising the residual imbalance (step 2) depends on the limit prices participants submit, so it is exactly the lever that a late participant exploits (4.2). The remaining two steps — distance to the previous clearing price p_{prev}, then the lowest price — do not depend on the orders of the current batch. Putting one of those first would remove the lever without changing total surplus (4.1).

Why the Price Is Uniform

Clearing can also be seen as an optimisation problem: choose how much of each order to execute so that total gains from trade are as large as possible, subject to buying and selling the same quantity. The uniform price is then not an extra rule bolted on afterwards; it falls directly out of that one requirement, that total buying equal total selling across the whole batch. Because the requirement applies to the batch as a whole and not order by order, every executed order ends up trading at the same price. Two consequences follow. First, the optimisation decides who trades but not always at what price: several prices can support the same trades, and that range is exactly $\mathscr{P}^*$. The price ambiguity is therefore built into the problem, and σ is what resolves it. Second, the filling rule is simple. Buyers whose limit price is above the clearing price and sellers whose limit price is below it are filled completely, orders on the wrong side are not filled, and only orders priced exactly at the clearing price can be partly filled. Those are the orders that must be rationed. This holds when one price maximises the volume; when a range of prices does and the tie-break picks a price inside it, the long side is rationed in priority order and an order priced better than the clearing price can occasionally be filled only in part as well (Section D.2).

Rationing Rules

At the clearing price the two sides are usually not equal, $D(p^*) \neq S(p^*)$, so the orders on the longer side priced exactly at p^* cannot all be filled and must be rationed. The rule chosen determines what participants do to obtain a larger share. Any rule that gives more to those who increase some variable they control encourages them to inflate that variable (Haynes and Onur 2020).

Price-time priority within the batch. Eligible orders are filled in priority order (market orders first, then the better limit price, then the earlier submission), so among orders at p^* the earlier one is filled first. This keeps continuity with continuous markets but brings back a speed race at the scale of the interval: if being early inside the batch improves the fill, participants pay to be early, which is the incentive the mechanism was built to remove. Under a fixed close it also pulls against the boundary repricing race (Section 4.2.3), which rewards submitting late: a participant must trade queue position against information. This is the rule the simulated engine uses (Table 5.1).

Pro rata. Each order at p^* receives a share proportional to its size, the rule the DFBA uses after price priority. It ignores timing, but because a larger order receives a larger fill, participants submit more than they want to trade. This *order stuffing* makes displayed sizes less informative.

Size priority or maximal fill. Larger orders are filled first, or the number of completely filled orders is maximised. This favours large "block" orders, which may be intended, but turns pro rata's incentive to inflate size into a threshold effect.

Random allocation. Fills are drawn by lottery among the orders at p^*. This removes both the timing and the size incentive and makes submitted size an honest statement of what the participant wants. The cost is an uncertainty about execution that participants cannot manage, which matters to anyone hedging, and the need for a random draw that participants can verify.

Hybrid rules. Several of the CME's matching algorithms give part of the volume by time priority and the rest pro rata, balancing the two distortions (Haynes and Onur 2020). A batch version could give a share κ pro rata and the remaining $1 - \kappa$ by lottery.

No rule is neutral. Every rule ties the allocation to some variable, and that variable becomes what participants compete on. The question is not which rule has no distortion, but which distortion the venue prefers.

Block 5: Settlement and Residual Handling

Gross versus Net Settlement

Gross settlement moves each matched pair separately, which is simple and leaves a clear record of every fill. Net settlement moves only each participant's net change in position, which reduces the number of transfers, sometimes greatly when many participants both buy and sell in the same batch. Where every transfer costs a fee, as on a blockchain, this is a direct saving and one of the main

practical arguments for batching (Wang et al. 2018; Gong et al. 2023). In a centralised venue the saving is smaller, but reconciliation is still easier. The drawback is that netting hides which order led to which transfer, which complicates per-fill reporting and any fee charged per execution.

The Residual

The volume on the longer side that cannot be matched at p^* does not disappear, and it is not filled from any other source. What happens to it is set by the order's time-in-force, and there are three possibilities:

1. **Roll over.** The unmatched remainder moves to the next batch with its original limit price and, where relevant, its original submission time. This is the natural default for resting limit orders and is what this thesis implements.
2. **Cancel.** The remainder is dropped at the end of the batch, the equivalent of an immediate-or-cancel order.
3. **Cancel the whole order unless fully filled**, the equivalent of a fill-or-kill order. This has to be built into the clearing problem itself and cannot be applied afterwards.

The third option shows where the clean structure of the clearing problem breaks down. All-or-nothing conditions turn a problem that a simple search over the price grid can solve into one that must test many combinations of orders. That is why venues supporting them limit order sizes, and why decentralised batch exchanges with rich order types use competitions between solvers instead of one fixed algorithm (Harrison and Pardo-Guerra 2025).

Summary: Design Options and Their Consequences

Table 4.2 collects every option discussed in this chapter, block by block, with the behaviour it is predicted to induce and the cost it carries. The options the simulated engine implements are listed in Table 5.1.

Table 4.2: Every design option of Chapter 4, the behaviour it is predicted to induce, and its main cost.

Option	Predicted consequence	Main cost or risk
Block 1: batch interval (Section 4.2)		
Short τ	prices stay current; a speed difference δ still decides the batch with probability δ/τ	thin, one-sided batches that often do not cross; truthfulness weakens as orders become pivotal
Long τ	speed differences become irrelevant; fuller batches	delay of $\tau/2$ on average; stale orders; market makers cannot withdraw in time

Table 4.2, continued

Option	Predicted consequence	Main cost or risk
Fixed close	predictable boundary; late submission weakly dominates, producing the boundary repricing race (Section 4.2.3)	speed regains value as deadline precision; accumulation phase can empty out
Random close	waiting becomes a gamble; interior optimum between information and risk of exclusion	imprecise timing for everyone; random draw must be verifiable
Volume- or event-triggered close	interval adapts to activity	close can be triggered strategically by submitting volume
Random end after a minimum	schedule predictable to within a short window; the exact last moment cannot be targeted	a short random window gives less protection than a fully random close
Block 2: information disclosure (Section 4.3)		
Fully open book	strong within-batch price discovery; early orders are exploited by later ones	strongest incentive to submit late; amplifies the boundary race
Indicative price	coordination on a common price estimate	some intent leakage through price moves
Aggregate imbalance	attracts offsetting liquidity without revealing limits	weaker price anchoring
Sealed bid	no exploitation of submitted orders; no informational reason to wait	bid shading under price uncertainty, which can reduce cleared volume
Block 3: order flow segregation (Section 4.4)		
Unified pool	all liquidity meets in one auction; simplest to regulate	uninformed flow pays for losses to informed flow
By participant class	tighter prices for flow believed uninformed	labels can be gamed; different prices for the same asset at the same time
By intent (DFBA)	makers never trade with each other; no queue to pick off makers	liquidity split across two thinner auctions; labels must be enforced
Maker–taker by declared intent	simple to operate	self-declared unless enforced
Maker–taker by price aggressiveness	observable and hard to game	penalises demand for certainty of execution
Maker–taker by marginality	ties the fee to price setting	fee unknown before the batch clears
Maker–taker by persistence	rewards liquidity that stays available across batches	needs order age tracked; every new order starts as a taker
Block 4: clearing and rationing (Section 4.5)		

Table 4.2, continued

Option	Predicted consequence	Main cost or risk
Tie-break led by minimum imbalance	price responds to marginal submitted limits	the lever for the last-mover advantage (4.2)
Tie-break led by a reference price	price insensitive to marginal limits; removes the lever	needs a reference (previous clear or external price)
Price-time priority	continuity with continuous markets	rewards being early in the batch: a speed race at the interval scale, pulling against the boundary race
Pro rata	ignores timing	order stuffing; displayed sizes less informative
Size priority or maximal fill	favours block orders	threshold incentive to inflate size
Random allocation	removes timing and size incentives; size becomes an honest statement	unmanageable execution uncertainty; verifiable randomness needed
Hybrid (κ pro rata, $1-\kappa$ lottery)	balances the two distortions	inherits a share of both
Block 5: settlement and residual (Section 4.6)		
Gross settlement	clear record of every fill	one transfer per matched pair
Net settlement	fewer transfers; large saving where each transfer has a fee	hides which order led to which transfer; harder per-fill fees
Roll over residual	natural for resting limit orders; residual keeps its limit price	unmatched orders accumulate across batches
Cancel residual (IOC)	no stale residual	participants must resubmit every batch
Fill-or-kill	all-or-nothing execution	turns clearing into a combinatorial problem; needs size limits or solvers

Three cross-block interactions stand out. A fixed close is costly mainly under an open book: disclosure supplies the information that makes waiting pay, and an imbalance-led tie-break supplies the lever to use it, so the boundary repricing race needs Blocks 1, 2 and 4 to line up and can be broken at any one of them. Price-time rationing and the boundary race pull in opposite directions, early for queue position and late for information, so neither rule removes the speed incentive on its own. And a short τ makes every block's problems sharper at once, because thin batches make individual orders pivotal for the price, the rationing and the residual.

Chapter 5

Simulation Design and Data

The empirical part of the thesis replays one historical stream of orders through two engines at once, a continuous engine and a batch engine. Because both engines receive exactly the same events, everything that could otherwise distort the comparison is held fixed, and the matching mechanism is the only difference between the two runs.

Simulation Environment

The Two Engines and the Replay

Both engines are written in Rust and work on the same order and trade format, so a difference in results cannot come from how orders are represented. Apart from that format they share no code. A shared replay loop reads the event stream once, in time order, and gives every order to *both* engines in the same pass, which is what makes a comparison interval by interval possible (Section B.2 lists its steps). The metrics are computed by one shared piece of code outside the two engines, so both are measured in exactly the same way. The replay follows the timestamps stored in the data, not the computer's clock, so the results do not depend on how fast the machine runs. It is deterministic, because nothing in either engine is random.

The Continuous Engine (CDA)

The continuous engine implements the matching rule of Section 2.2: price-time priority, immediate matching at the resting order's price, and an unfilled limit remainder that stays in the book until it is filled or a cancel record removes it (Section 5.1.5). Section B.3 describes the data structures behind the book. The time-in-force field of an order is read from the data but not enforced. This is a limitation of this implementation, and Section D.5 measures how large it is.

The Batch Engine (FBA)

The batch engine implements the clearing rule of Section 2.3: it collects all orders arriving within an interval τ, clears them at its end at the volume-maximising price chosen by the tie-break cascade, rations the longer side by price-time priority within the batch, and rolls the residual into the next batch. Two implementation details are not part of that description. Candidate prices are taken only from submitted limit prices; a market order counts as demand or supply at every price but never proposes a price itself, so a batch made only of market orders does not trade. And orders priced

strictly better than the clearing price are filled completely, except in the rare case where a range of prices supports the same volume (Section D.2). Section B.4 describes the clearing step in detail.

No outside source of liquidity, such as an automated market maker, is used to fill it, so both engines can only trade against the orders that were actually submitted. The interval is fixed at $\tau = 1$ second, which keeps a replay of about 640 million records computationally feasible while still being a meaningful interval.

The Design Choices Used for the FBA

Chapter 4 describes a space of possible batch auctions in five blocks, summarised in Table 4.2. The engine that was run makes one fixed choice in each block, listed in Table 5.1. None of them can be changed at run time (Chapter B), and every result in this thesis is for exactly this combination.

Table 5.1: The FBA configuration that was run, by design block.

Design block	Choice in the simulated engine
Interval and gate	One fixed interval $\tau = 1$ s. The batch opens at the first record and closes when a record with a timestamp at or after the next boundary arrives; there is no randomised extension of the close.
Disclosure	Not applicable in a replay: nobody reacts to what the engine shows, so the disclosure regime cannot change any result.
Segregation	One pool: all orders, buyers and sellers, market makers and takers, are cleared together in a single auction.
Clearing	One uniform price per batch, chosen to maximise the executed volume from the submitted limit prices; market orders count as demand or supply at every price. Ties in volume are broken by the least leftover imbalance, then the closest price to the previous clearing price, then the lowest price.
Rationing	Price-time priority within the batch: market orders first, then the better price, then the earlier submission; the side with more volume is filled in that order until the smaller side is used up.
Residual and cancels	Volume that finds no counterpart always rolls into the next batch; it is never cancelled by the engine. An order leaves the engine only when it is fully filled or when a cancel record for it arrives (Section 5.1.5); a cancel that arrives before the batch closes removes the order from that batch.
Settlement	Trades are recorded gross, one per matched pair; there is no netting and no fee or rebate, so the maker–taker question of the design chapter does not arise.
Not modelled	Time-in-force, self-trade prevention, and any reaction of participants to the mechanism.

How Cancellations and Other Records Are Treated

Each record of the data is one status change of one order, and the replay turns it into an action for the engines in a fixed way. A record that opens an order (status `open`, or `triggered` for a conditional order) becomes a new order. A record of one of the eight cancel types (cancelled by the trader, scheduled or reduce-only cancel, cancel after a sibling order was filled, self-trade cancel, margin cancel, vault-withdrawal cancel, liquidation cancel) removes the order with the same order number from the engine: from the resting book of the CDA, or from the waiting orders of the FBA, including any part that was carried over from earlier batches. The whole order is removed, whatever size it

had left. If the order is no longer there, because it was already filled or because its opening record never reached the engine (it was opened before the first record of the data, or skipped for its size, Section 5.2.2), the cancellation has no effect and is not an error. Cancellations are processed in file order, so a cancel that carries the same timestamp as a new order is applied in the order the records appear. Rejections, and conditional orders that have not triggered yet, never enter an engine.

The records that the venue wrote when *its own* engine executed an order (status `filled`) are deliberately not replayed. The two engines decide fills themselves, and replaying the venue's fills as well would count them twice. The consequence is that an order which the venue filled completely is removed from a simulated book only if the simulation also fills it; otherwise it stays until it is matched, or cancelled by a later record, which the venue never sends for an order that is already filled. Cancellation records are therefore the only way the simulated books shed orders other than by trading, and the size of the books depends on them.

Venue and Data

The Venue

Hyperliquid is a decentralised exchange that runs on its own blockchain. Its trading layer, HyperCore, holds the order books, margin accounts and positions in the state of the chain, so the limit order book itself is on chain rather than kept by an off-chain operator; the chain is secured by HyperBFT, a variant of the HotStuff consensus protocol (Hyperliquid 2026). Most of its activity is in perpetual futures, among them the SOL contract studied here (Section 2.1). Three features of the venue matter for this thesis.

A continuous book with price-time priority. Orders are matched in price-time priority, at prices on a fixed tick grid and in sizes that are multiples of a lot size (Hyperliquid 2026). The venue is therefore a continuous double auction in the sense of Section 2.2, and the CDA engine of this thesis models its matching rule.

Processing in blocks, cancellations first. Orders do not reach the book one at a time as they leave a participant's machine; they are processed block by block. Within each consensus batch of a block the venue sorts actions into three groups and processes them in this order: actions that place no new order, then cancellations, then actions that place new orders, keeping the block proposer's order within each group (Hyperliquid 2026). A cancellation and an aggressive order that arrive in the same batch are therefore not raced against each other: the cancellation always goes first. Hyperliquid's continuous market thus already contains one element of batch design, protection of resting quotes against being picked off within a block, while trades still execute one order at a time at the resting order's price. The replay processes records in the order the node wrote them, which follows the venue's own processing order, so this rule is reflected in the stream both engines receive. The comparison in this thesis is therefore between batching and a continuous market that already softens the latency race, not a purely

first-come, first-served exchange, and the gain from batching it measures may be smaller than on a traditional venue.

Latency and timestamps. The documented end-to-end latency, from sending a request to receiving the committed response, has a median of 0.2 s and a 99th percentile of 0.9 s for a co-located client (Hyperliquid 2026). Speed differences that matter on this venue are therefore measured in tens to hundreds of milliseconds, not the microseconds of the equity races in Section 3.1, and a batch of $\tau = 1$ s is only about five times the median latency, close to the lower bound discussed for Block 1 (Section 4.2). Because records are written block by block, most consecutive records carry the same timestamp (Section 6.1), and a timestamp shows when the node processed a message, not when the participant sent it.

The Dataset

The simulation uses the public Hyperliquid Level 4 order book dataset of Albers et al. (2026), recorded directly at a Hyperliquid node and published on Zenodo: the full order-by-order feed, with participant attribution, rejected orders, failed cancellations, both sides of every trade and post-trade inventory, all at nanosecond resolution. Because it is recorded at the node and not sampled from snapshots, the book does not have to be reconstructed, which removes an important source of error.

Two conversions are applied as the records are read (Section B.1). Prices are kept exactly, as whole numbers of 10^{-6} USD. Sizes are rounded to whole SOL, and a record whose size rounds to zero, an order smaller than 0.5 SOL, is skipped; this removes 2.65% of the records (Table 6.1). Both engines receive the same rounded stream, so the conversion cannot favour either of them, but the smallest orders of the venue are not part of the comparison.

Since the engines are the author's own, some quantities that are invisible in real markets can be observed directly: every submitted price, including those of orders that never trade, the exact rationing outcome, and the volume left unmatched after each batch. Some quantities remain unknown. The dataset has no oracle or mark price, so neither engine's price can be measured against an external reference second by second within the replay; only at an hourly scale can both be compared with the spot price, and they track it closely (Section D.5.1). Orders that were never submitted visibly cannot be recovered. And timestamps show when the node saw a message, not when the participant chose to send it.

Experimental Design

The treatment is the matching engine, and the design tries to make everything else identical, stating where it could not:

1. **Same input.** Both runs read the same events in the same order: the full December 2025 SOL order-status archive, from its first record.

2. **Same starting state.** Both engines start from an empty book at the same first timestamp, and no lead-in period is excluded from the metrics.

3. **Same rules outside the mechanism.** Neither engine prevents self-trades, so this affects both alike (Section 6.7.3).

4. **Determinism.** The same input always produces the same output, since nothing in either engine depends on randomness or on the clock; the only exceptions are the two columns that measure computing time (Section D.4).

The sample is a single instrument, the SOL perpetual, replayed once through each engine over the whole month covered by the data: 1–31 December 2025, 744 hourly files, none missing (the records themselves contain a few silences, Section D.1). It is the full month, not a sample, so comparisons across instruments or sessions are not possible. Each one-second interval provides one CDA value and one FBA value from the same order flow, so differences are measured as paired differences per interval. This removes variation caused by the market itself and isolates the effect of the mechanism. Because market data in consecutive seconds are correlated, the standard errors are Newey–West errors (Newey and West 1987), which allow for this correlation. The lag truncation is set automatically per metric from the paired sample size n by the rule of Newey and West (1994), $\lfloor 4(n/100)^{2/9} \rfloor$, capped at 60 lags, so no fixed lag count is chosen by hand and every table in Chapter 6 reports a standard error computed the same reproducible way. Kyle's lambda is the one metric that is a slope rather than a mean. For it, the per-second difference between the two engines' price changes is regressed on the common order flow, and the same Newey–West rule gives the standard error of the difference in slopes (Section 2.5.2).

This design is what allows a finding to be attributed to the mechanism: the two engines see the same orders in the same second, so a systematic paired difference can only come from how they match them. A replay holds the order stream fixed, which a field comparison cannot (Section 1.3); what it cannot show is how participants would have changed their orders under the other mechanism (Section 7.2).

Metrics

The results report three groups of metrics, one for each part of RQ2: liquidity and cost, price discovery, and execution and timing, plus two engine controls. Section 2.5 defines every metric formally, and Section 3.5 explains why each is used.

Interpretive Status of the Metrics

The replayed orders were submitted on Hyperliquid's own continuous engine, not in reaction to batching, so every metric shows the mechanical effect of the batch rule on fixed historical orders (Section 7.2). For the same reason, where orders fall relative to a simulated batch boundary carries no information, since that position depends on where the boundary was placed. Throughput and clearing latency are controls, not results (Section 2.5.4).

Validation of the Simulator

A finding is only as reliable as the code that produced it, so the simulator is checked in four ways: structural conditions checked live during the run, which stop it rather than write a wrong row; a suite of automatic tests of financial properties such as conservation of quantity and no trade beyond a limit; a checklist of hand-worked cases, including every tie-breaking step; and two outside references, an independent second implementation of both engines and a comparison with the volume and price the venue recorded. Section 6.2 reports the outcome and Chapter D lists every test.

With the data, the two engines, the paired design and the metrics in place, Chapter 6 reports what the comparison finds.

Chapter 6

Results: Comparing CDA and FBA

This chapter reports the results of one executed run: the SOL perpetual, the full month of 1–31 December 2025, $\tau = 1$ second, and the single fixed engine configuration of Chapter B. Throughout, a *bucket* is one of the 2,678,400 consecutive one-second windows of the month, and every metric is computed per bucket. Each paired difference (FBA−CDA) uses only the buckets in which the metric exists for both engines (the n column) and a Newey–West standard error (Section 5.3). Sections 6.3–6.5 report the results, Section 6.6 summarises them, and Section 6.7 sets out the limits on how far they can be read.

Main findings. The comparison yields five findings, one for each dimension of market quality, each stating which engine performs better and by how much.

F1 Liquidity: the CDA is slightly more liquid. Depth within 10, 50 and 100 bps of the price is higher in the CDA (32,681.93 against 31,515.77 SOL within 10 bps), the price moves less per dollar traded (Amihud illiquidity 0.09 against 0.11), and the quoted spread is practically the same (0.84 against 0.87 bps). The differences are small: batching at one second does not make the market more liquid (Section 6.3).

F2 Transaction costs: resting orders pay an adverse-selection cost in the CDA; the FBA's cost cannot be measured. In the CDA the realized spread is negative and the price impact grows from 1.33 to 1.65 bps between 1 and 30 s. Because FBA trades have no direction, the replay cannot say which engine is cheaper to trade in (Sections 6.3 and 6.7.1).

F3 Volatility: the FBA is slightly more volatile. Its daily volatility is 2.11 against 2.05 bps for the CDA: batching at one second does not dampen price moves (Section 6.4).

F4 Price formation: no difference. Both engines arrive at the same price (median gap 0.00 bps), and five seconds after the venue's net order flow both prices have moved by the same amount (Kyle's lambda 0.638 against 0.642 bps per 1,000 SOL, p 0.221; Section 6.4).

F5 Execution: the same volume trades, but the CDA executes faster. Volume, averaged over all seconds of the month with seconds without trades counted as zero, is the same in both engines (58.08 against 58.37 SOL per second, p 0.215), but orders wait 3.52 s longer for their first fill in the FBA (Section 6.5).

Section 6.6.1 explains how each finding is reached, why it arises and how far it can be relied on. F2, F4 and F5 are well supported; F1 and F3 are consistent in direction but small, and partly reflect how each engine's price is measured.

Sample and Descriptive Statistics

Table 6.1 describes the replayed sample. Both engines receive this event stream unchanged, so differences between their results are not caused by differences in the input.

Table 6.1: Description of the replayed sample. Message counts refer to the raw event stream, identical for both engines; trade counts and notional are each engine's own simulated output.

Characteristic	Value
Instrument	SOL perpetual futures (SOL-PERP), Hyperliquid
Session	1–31 Dec 2025 (76.62% of 1s FBA batches empty)
Input files	744
Total records seen	640,176,812
of which new live orders	283,797,135
of which cancellations	281,778,306
of which other (rejections, fills, non-triggered conditionals)	57,615,417
skipped (zero size after rounding)	16,985,954 (2.65%)
Distinct participants (user ids)	37,027
Median inter-arrival time, raw record stream	0.000 ms
Mean inter-arrival time	4.159 ms
99th pct. inter-arrival time	126.6 ms
CDA trades (simulated)	3,481,516
FBA trades (simulated)	3,530,770
CDA executed notional (simulated)	$20,200,362,171.01
FBA executed notional (simulated)	$20,308,937,215.80

The data have no separate modify event and neither engine implements one, so a change to a resting order appears as some combination of the categories above.

Inter-arrival time is the time between one record and the next. Its median (0.000 ms) is exactly zero because 96.0% of consecutive records have the same nanosecond timestamp, as expected on a venue that processes orders block by block (Section 5.2.1). The mean (4.159 ms) and the 99th percentile (126.6 ms) show the gaps between these bursts, of tens to hundreds of milliseconds. This also explains the empty-batch share: "76.62% of batches empty" means that in that second no buy limit reached any sell limit (a batch of only market orders does not trade either, Section 5.1), not that the second was quiet. The continuous market is no different in this respect: the CDA also trades in only about a quarter of all seconds (Section 6.4), because in most seconds no order arrives that is priced to cross the book.

Code Validation Results

These are preconditions, not findings: the comparison can only be trusted as far as the code behind it. The run triggered no structural check failure and passed every test in both test sets (85/85 automatic tests, 37/37 hand-checked cases). An independent Python reimplementation of both engines, written from the rules with different data structures, agrees with the simulator in every one-second bucket of four full hours (29,446 engine-buckets, 0 differences), and the tests catch 19 of 19 bugs planted on purpose. Convergence of the FBA to the CDA as $\tau \to 0$ was not checked (Section 6.7.3).

RQ2.1: Liquidity and Transaction Costs

Table 6.2 reports the liquidity and cost metrics for both engines. Two findings come from it: the CDA is slightly more liquid than the FBA (F1), and in the CDA resting orders pay an adverse-selection cost, while the FBA's cost cannot be measured (F2).

Table 6.2: Liquidity and transaction costs under both mechanisms. Paired differences per interval (FBA−CDA); Newey–West standard errors robust to autocorrelation in parentheses; n is the number of one-second intervals (of 2,678,400 in the month) in which the metric was computable on both engines. FBA effective spread, realized spread and price impact are not a cost measure (Section 6.7.1) and their differences are not interpreted.

Metric	CDA	FBA	Diff.	p	n
Quoted spread (bps)	0.84	0.87	0.03 (0.00)	<0.001	2,655,207
Depth at best	133.40	414.30	280.90 (1.83)	<0.001	2,655,207
Depth within 10 bps	32,681.93	31,515.77	-1,166.17 (8.03)	<0.001	2,655,207
Depth within 50 bps	158,346.57	156,232.12	-2,114.44 (50.18)	<0.001	2,655,207
Depth within 100 bps	268,437.06	260,936.90	-7,500.16 (143.50)	<0.001	2,655,207
Effective spread (bps)	1.00	0.00	-1.01 (0.01)	<0.001	545,562
Realized spread, $\Delta = 1$s (bps)	-0.32	-0.01	0.32 (0.01)	<0.001	545,562
Realized spread, $\Delta = 5$s (bps)	-0.51	-0.01	0.50 (0.02)	<0.001	545,562
Realized spread, $\Delta = 30$s (bps)	-0.65	-0.07	0.57 (0.05)	<0.001	545,561
Price impact, $\Delta = 1$s (bps)	1.33	0.00	-1.32 (0.01)	<0.001	545,562
Price impact, $\Delta = 5$s (bps)	1.52	0.01	-1.51 (0.02)	<0.001	545,562
Price impact, $\Delta = 30$s (bps)	1.65	0.07	-1.58 (0.05)	<0.001	545,561
Amihud illiquidity ($\times 10^6$)	0.09	0.11	0.02 (0.00)	<0.001	545,572

Quoted spread. The two engines are close: 0.84 bps for the CDA and 0.87 bps for the FBA, a gap of 0.03 bps. The difference is statistically significant (p <0.001) only because n is over two million; it is too small to matter economically.

Depth: the CDA is slightly deeper (F1). Depth within a fixed distance of the reference price is defined the same way for both engines, and on it the FBA is slightly *thinner* at every threshold: 31,515.77 against 32,681.93 SOL within 10 bps, 156,232.12 against 158,346.57 within 50 bps, and 260,936.90 against 268,437.06 within 100 bps (all p <0.001; Figure 6.2). If batching pooled liquidity, the FBA would be deeper. Depth at best is higher for the FBA (414.30 against 133.40 SOL), but the two figures measure different objects and are not compared (Section 6.7.2).

Spread decomposition: resting orders lose in the CDA (F2). Effective spread, realized spread and price impact split the cost of a trade into the total cost, the part the liquidity provider keeps once the price has moved, and the part lost to better-informed traders. The CDA shows the textbook pattern of adverse selection: a positive effective spread (1.00 bps), a *negative* realized spread at every horizon (-0.32/-0.51/-0.65 bps at 1/5/30 s), and a positive price impact that grows with the horizon (1.33/1.52/1.65 bps). The realized spread is the revenue of the *resting* side of a trade, the maker, which in a continuous market is mostly a market maker; the taker's cost is the effective spread. A negative realized spread means that the maker earned the effective spread at the moment of the trade but lost more than that once the price moved: the price impact exceeds the effective spread. On average a resting order therefore loses once the price has absorbed what the trade revealed, and only the makers bear this loss. The FBA's values (Figures 6.1 and 6.3) are close to zero by construction and are reported but not interpreted (Section 6.7.1); the comparison between the engines rests on the measures that need no trade direction, Amihud illiquidity below and Kyle's lambda in Section 6.4.

Amihud illiquidity: lower for the CDA (F1). The price movement per dollar traded is about a fifth higher for the FBA (0.11 against 0.09, $p < 0.001$). Part of this gap is a matter of measurement. The CDA's reference price is the book midpoint, which does not move when trades alternate between the bid and the ask; the FBA's is the clearing price, which lands on the ask side when buyers dominate a batch and on the bid side when sellers do, so it moves by about the spread between such batches even when the midpoint does not. The Amihud gap therefore overstates how much less liquid the FBA is; together with the depth result it shows that the FBA is not more liquid.

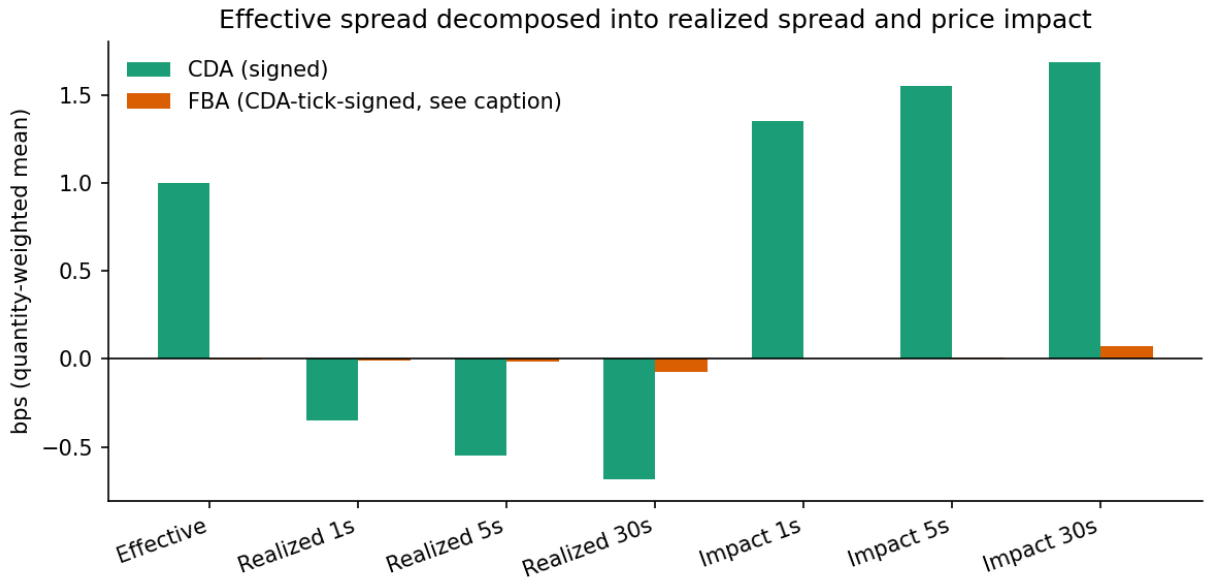


Figure 6.1: Effective spread decomposed into realized spread (at three markout horizons) and price impact, mean per bucket, CDA versus FBA over the full sample. The FBA values are not a cost measure (Section 6.7.1).

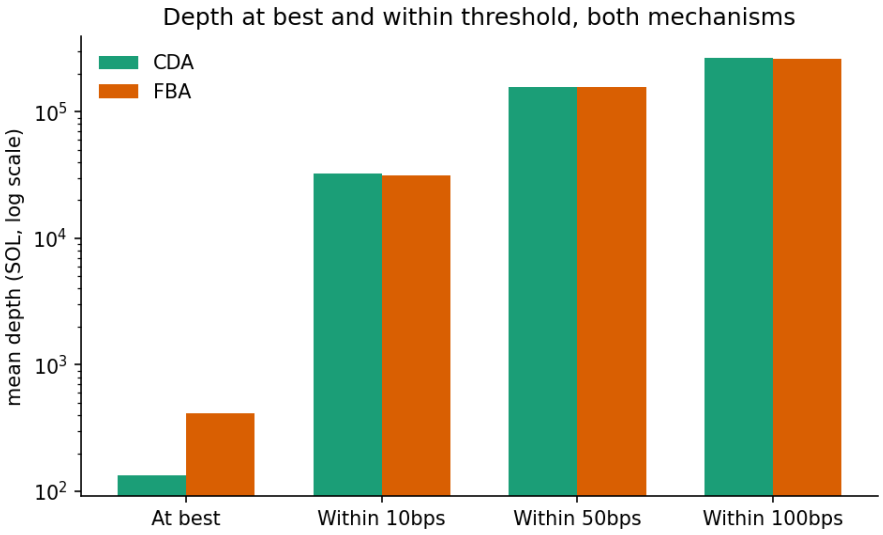


Figure 6.2: Mean depth at best and cumulatively within 10/50/100 bps, CDA versus FBA (log-scale axis: depth at best is two to three orders of magnitude smaller than the cumulative bands). The two "at best" figures are not directly comparable.

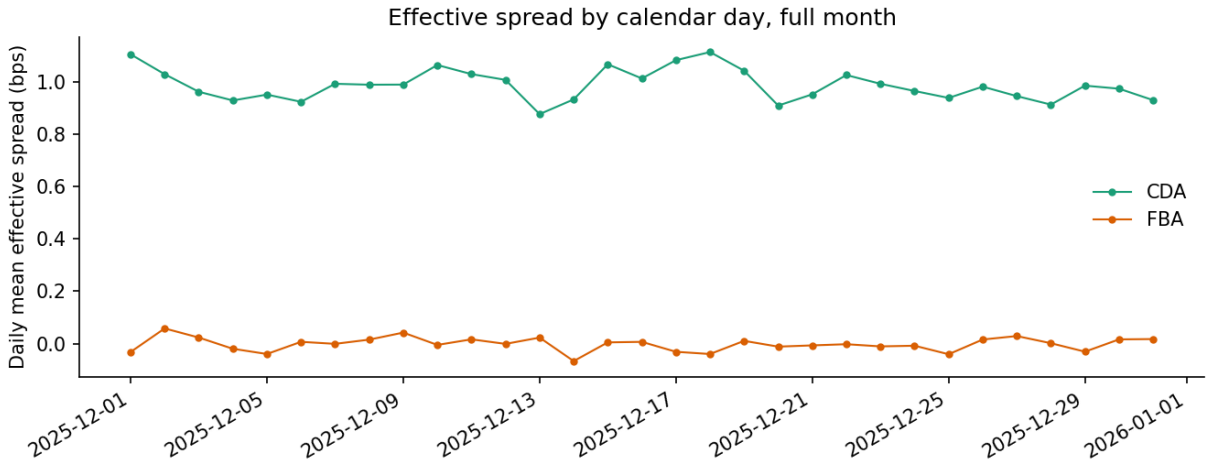


Figure 6.3: Daily mean effective spread, CDA versus FBA, across all 31 days. The FBA values are not a cost measure (Section 6.7.1).

RQ2.2: Price Discovery and Market Quality

Table 6.3 reports the price discovery metrics. Three findings come from this section: the FBA is slightly more volatile than the CDA (F3), and the two engines arrive at the same price and respond to the same order flow by the same amount (F4).

Table 6.3: Price discovery under both mechanisms. Paired differences (FBA−CDA), mean (Newey–West HAC standard error in parentheses), with p-values where both engines report the metric; n as in Table 6.2. For Kyle's lambda the CDA and FBA columns are the two slopes on the same venue flow, the difference is the slope of the engines' difference in price change (Section 2.5.2), and n is the number of seconds with venue flow.

Metric	CDA	FBA	Diff.	p	n
Kyle's lambda, same second (bps per 1,000 SOL)	0.436	0.536	0.099 (0.008)	<0.001	786,990
Kyle's lambda, 5 s (bps per 1,000 SOL)	0.638	0.642	0.005 (0.004)	0.221	786,990
Matched-timestamp volatility, daily RMS (bps)	2.05	2.11	0.06 (0.00)	<0.001	31

Volatility: the FBA is slightly more volatile (F3). Volatility within a bucket cannot be compared, because the FBA almost never has two prices in one bucket ($n = 1$); matched-timestamp volatility replaces it. Both engines traded in 545,573 buckets (20.37%), about four times the 5.53% that two unrelated series with trades in roughly 23% of buckets would share by chance, so both engines react to the same bursts of orders. Price changes between consecutive matched buckets (545,572 changes) cover exactly the same instants for both engines, and their daily root mean square is similar for both, slightly higher for the FBA (2.11 against 2.05 bps, p <0.001 over $n = 31$ days, 0 days excluded; Figure 6.4). No dampening of price moves (Table 3.1) is found at $\tau = 1$ s. The small excess for the FBA is consistent with how the two prices are formed: the CDA's VWAP averages several fills within the second, which smooths it, while the FBA's single clearing price does not. The measure covers the busier seconds in which both engines trade; it is a comparison on common ground, not a volatility estimate for the whole month.

Kyle's lambda: the same response to order flow (F4). Both engines' prices follow the venue's net taker flow (Figure 6.5, left): net buying raises the price and net selling lowers it, as adverse selection predicts. Measured 5 s after the flow, the two engines respond by the same amount, 0.638 against 0.642 bps per 1,000 SOL (p 0.221). Batching therefore neither hides order flow from the price nor makes the price overreact to it; theory predicts that the FBA's price should react *less* to one-sided flow (Table 3.1), and the replay shows no such mechanical effect. In the theory the reduction comes through behaviour, liquidity providers quoting more aggressively once they are protected from sniping, which a replay of fixed orders cannot reproduce. Within the same second the FBA's price moves more (0.536 against 0.436, p <0.001), but the gap comes only from the seconds with the very largest flows, and the same-second value compares a VWAP with a single clearing price over a sample in which a non-trading engine's change is zero (Section 2.5.2); the 5 s value is the cleaner comparison. The CDA's per-sweep estimate, 0.09 bps per SOL of a sweep's executed quantity, is an average of per-second slopes; it describes the CDA alone and is not on the scale of the pooled slopes above (Section 2.5.2).

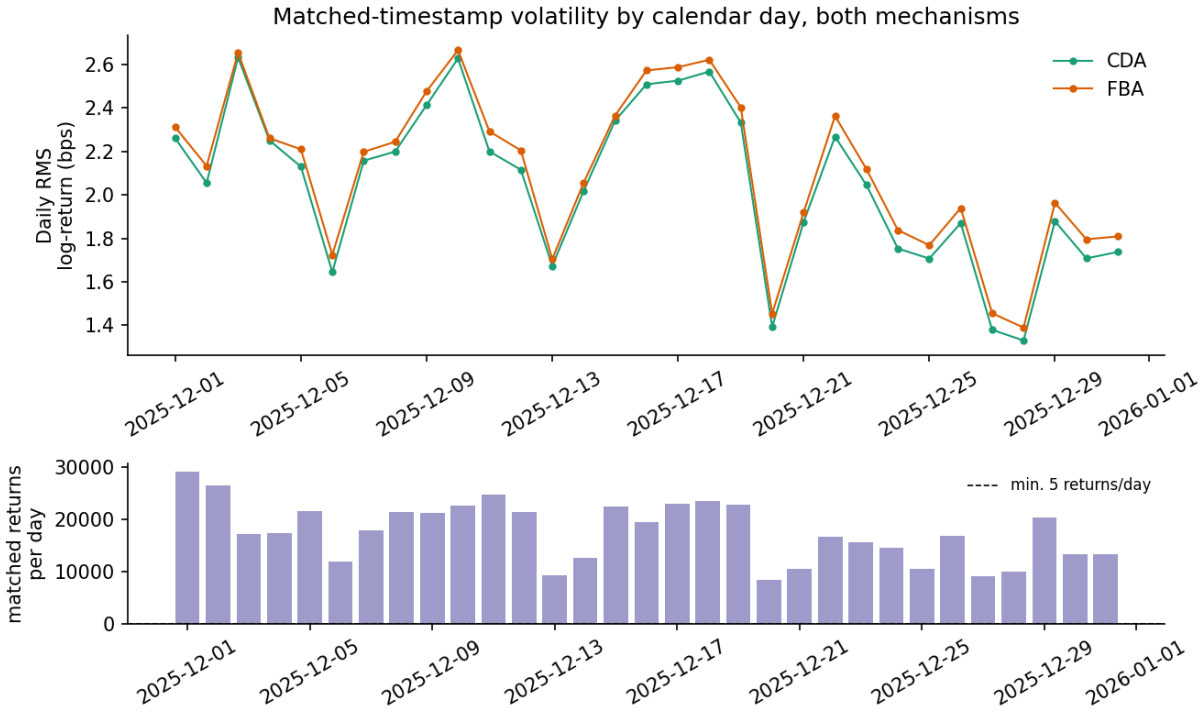


Figure 6.4: Matched-timestamp volatility by calendar day (top), and the number of matched returns each day was built from against the 5-return exclusion floor (bottom).

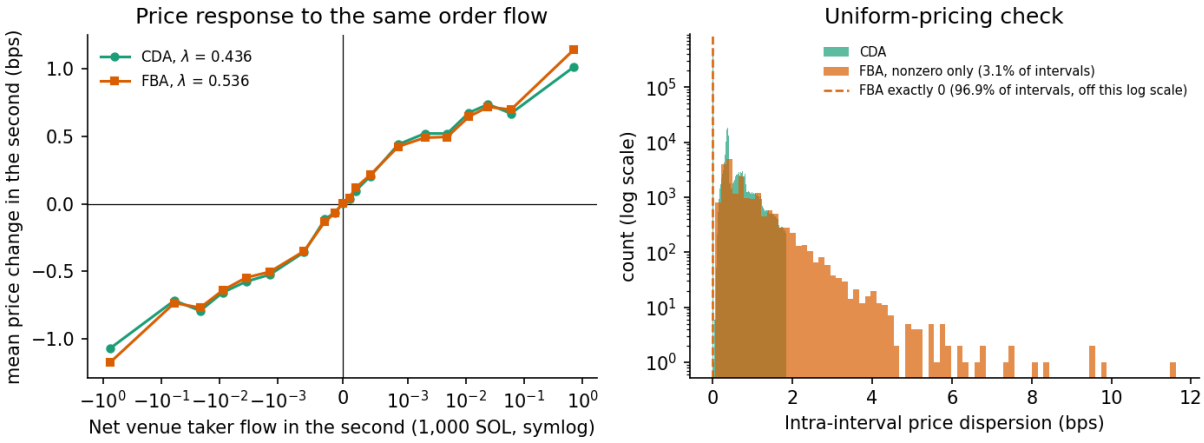


Figure 6.5: Left: mean price change of each engine within the second, against the venue's net taker flow in that second, in 20 bins of equal count (horizontal axis symmetric-logarithmic); the legend gives the pooled same-second slopes. Right: distribution of the spread of trade prices within one second (log-scale count axis), a check that the batch engine trades each batch at a single price, as its rule requires; it is not part of the comparison.

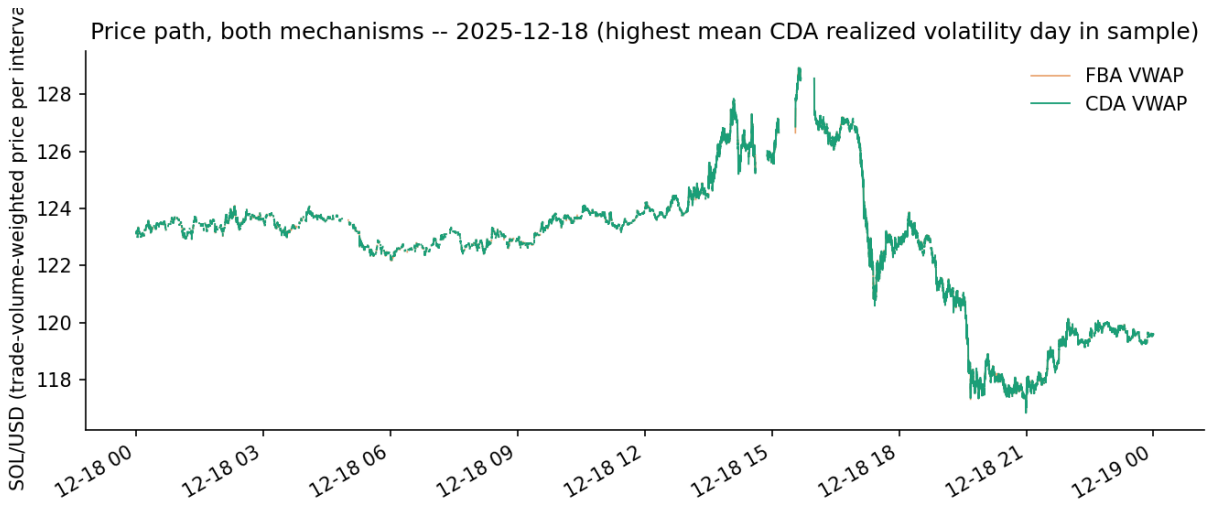


Figure 6.6: CDA and FBA price paths, as the VWAP of each one-second interval, the price series the two engines have in common, over 2025-12-18. This is the day with the highest mean CDA realized volatility among the days on which at least 80% of the one-second buckets have a value, so that a day with little data cannot be selected by one extreme observation. No external reference price is available at one-second resolution to plot alongside; at an hourly scale both prices track the spot price closely (Section D.5.1).

Price paths: the same price (F4). The two engines' prices track each other closely (Figures 6.6 and 6.7). Over 545,573 intervals in which both traded, the FBA-minus-CDA VWAP difference has a median of 0.00 bps, a mean of -0.00 bps and a standard deviation of 0.85 bps. The gap widens only in fast markets: its size correlates 0.36 with CDA realized volatility, against under 0.25 with either engine's trade count. The reason is mechanical. The CDA's VWAP averages every fill as the book is swept within the second, while the FBA clears once at one price, so in a fast market the two definitions of "the price of this second" can differ by tens of bps without either being wrong. A shorter batch interval would narrow this gap, since the FBA would then clear several times within the second; only $\tau = 1$ s was run.

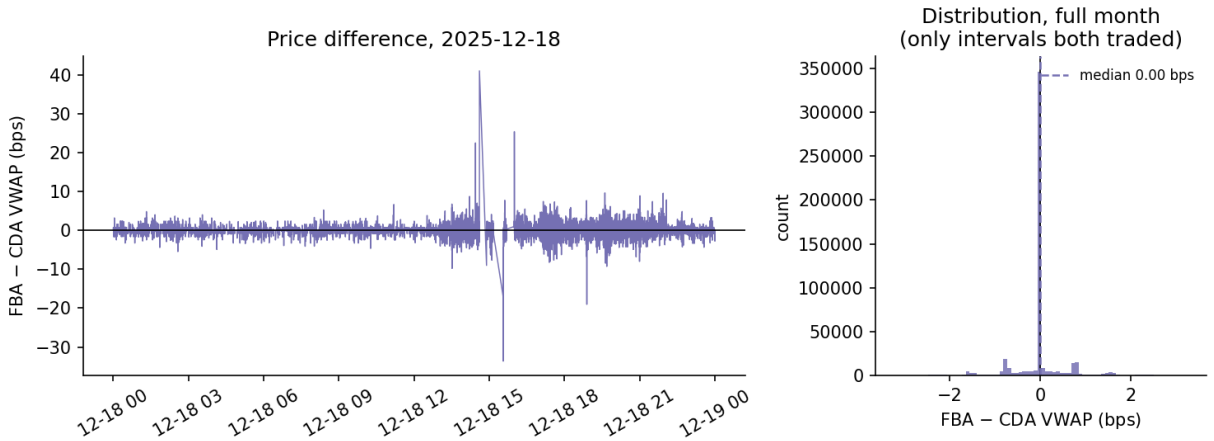


Figure 6.7: Left: the FBA-minus-CDA VWAP difference in bps, same day as Figure 6.6. Right: its distribution pooled over every interval in the month where both engines traded, clipped at the 0.5th/99.5th percentiles for legibility.

RQ2.3: Execution

Table 6.4 reports the execution metrics. Two findings come from this section: both engines trade the same volume, but the CDA executes orders faster (F5). Trader surplus confirms the transfer argument of Section 4.2.3.

Table 6.4: Execution under both mechanisms, plus engine controls. Diff. is mean (Newey–West HAC standard error in parentheses); n as in Table 6.2.

Metric	CDA	FBA	Diff.	p	n
Executed volume (SOL, per interval, mean)	58.08	58.37	0.29 (0.23)	0.215	2,678,400
Trade count (per interval, mean)	1.30	1.32	0.02 (0.00)	<0.001	2,678,400
Avg. time to execution (s)	19.28	22.80	3.52 (0.07)	<0.001	925,551
Trader surplus (USD, per interval)	37.02	37.51	0.48 (0.07)	<0.001	2,678,400
Engine controls					
Throughput (messages/s)	960,048.63	255,139.80	-704,908.83 (1,187.60)	<0.001	2,655,207
Mean clearing/match latency (μs)	1.25	1,918.00	1,916.75 (4.03)	<0.001	2,655,207

Executed volume: the same in both engines (F5). Volume is not statistically different (58.08 against 58.37 SOL per interval, p 0.215); over the month this is $20,200,362,171.01 against $20,308,937,215.80 of notional (Table 6.1). Both engines can only trade the orders that were submitted, so the result means that batching, although it delays trades, does not leave tradable flow unmatched over the month.

Trade count is slightly higher for the FBA (1.32 against 1.30 trades per interval, p <0.001). With the same volume this means slightly smaller trades, consistent with one clearing price splitting a large order among several counterparties.

Time to execution: the CDA is faster (F5). Waiting is the clearest cost of batching: 22.80 s for the FBA against 19.28 s for the CDA, a gap of 3.52 s (p <0.001). Waiting for the next clear explains at most one second of it, half a second on average for orders arriving evenly within the interval. The rest is not explained by the empty batches: 76.62% of one-second batches clear no trade, but the CDA trades nothing in about the same share of seconds (Section 6.1), so the empty batches mostly mirror seconds in which no order priced to cross arrived. The most likely source of the remainder is which resting orders are matched once a batch does cross: the batch fills the orders eligible at its clearing price in price-time order within the batch, while the CDA fills an arriving order level by level from the best resting price, so the two engines can fill different resting orders from the same flow. This explanation is not measured directly here. A longer interval would add to the wait for the clear, up to τ, but would collect more orders per batch; only $\tau = 1$ s was run.

Trader surplus is marginally higher for the FBA (37.51 against 37.02 USD per interval, p <0.001). The difference does not come from uniform pricing as such: on a trade between two limit orders, the buyer's and the seller's improvement add up to the gap between their limits whatever the trade price (Section 4.2.3). It comes from a slightly different set of matched orders, and from trades with a market

order, whose side has no limit and contributes nothing, so that only the limit side's improvement, which does depend on the price, is counted (Section 2.5.3).

Engine controls show whether each engine would keep up with the order flow in real time; they do not bear on any other result (Section 6.7.3). The mean latency (1.25 μs for the CDA against 1,918.00 μs for the FBA) times one order against one whole batch, which also re-evaluates every rolled-over order, so its three-orders-of-magnitude gap reflects what is timed. Throughput, which divides the same message count by computing time, is the fairer comparison: 960,048.63 against 255,139.80 messages per second of computing time, under four times apart. Both engines stay far inside real-time needs: an FBA clear of about 2 ms is three orders of magnitude shorter than the one-second interval it has to fit into (Figure 6.8).

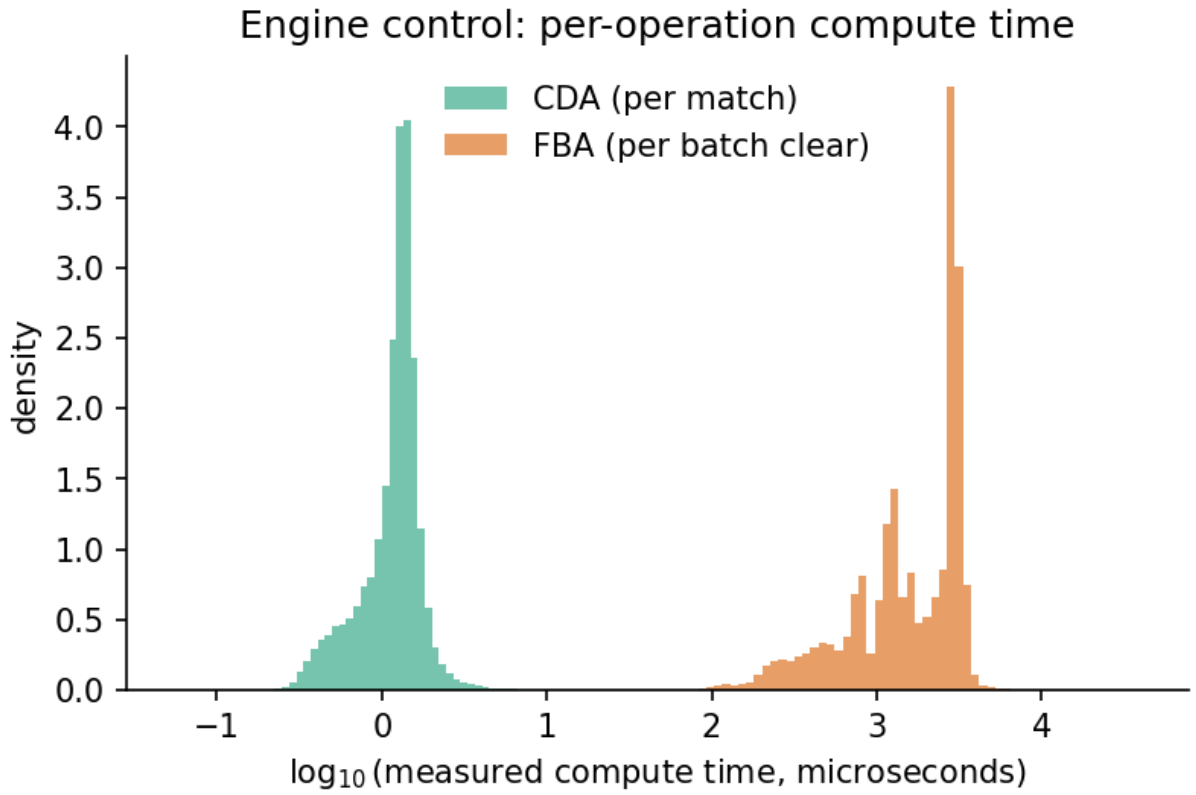


Figure 6.8: Distribution of measured compute time per operation, CDA (per message) versus FBA (per batch clear); log scale. A control, not a market-quality result.

Conclusions from the Simulation

Main conclusion. When the same month of SOL order flow is cleared in one-second batches instead of continuously, the continuous market is slightly more liquid (F1) and slightly less volatile (F3), and it executes orders faster (F5). The two engines do not differ in price formation: they reach the same price and respond to the same order flow by the same amount (F4), and they trade the same volume (F5). In the continuous market resting orders pay a systematic adverse-selection cost (F2); whether batching lowers it cannot be measured in the replay (Section 6.7.1).

How Each Finding Is Reached, Why It Arises, and How Far It Can Be Relied On

All findings rest on the same design: both engines receive exactly the same orders, every metric is compared second by second as a paired difference, and the standard errors allow for correlation between consecutive seconds (Section 5.3). A difference between the engines can therefore come only from the matching rule or from how a metric is measured in each engine, and each finding below is checked for both. With more than half a million paired seconds for most metrics, even very small differences are statistically significant, so each finding is judged on the size of the effect as well as on its p-value. For each finding, *Evidence* gives the numbers, *Reasoning* how the conclusion follows from them, *Why* the mechanism that produces the result, and *Reliability* how far it can be relied on. Table 6.5 summarises the result.

F1 — Liquidity: the CDA is slightly more liquid. *Evidence:* depth within 10, 50 and 100 bps is higher in the CDA by between one and four percent (p <0.001); Amihud illiquidity is 0.09 for the CDA against 0.11 for the FBA (p <0.001); the quoted spread is 0.84 against 0.87 bps. *Reasoning:* depth is the measure defined the same way for both engines, and its only known bias favours the FBA (Section 6.7.2), so the CDA's lead in depth cannot come from that bias. Amihud illiquidity points the same way but is not like-for-like: it uses the book midpoint for the CDA and the clearing price for the FBA, and a clearing price moves between the bid and the ask side with the direction of each batch even when the midpoint does not, which raises the FBA's value (Section 6.3). The quoted spread is practically equal and is weaker evidence, because the FBA's is read from the orders left after a clear. *Why:* the argument for batching is that it pools orders that would otherwise arrive at different moments, so that each auction is deeper than the continuous book. At one second this market gives a batch little to pool: 76.62% of batches clear no trade, about the same share of seconds in which the CDA trades nothing, so a batch that trades usually holds the same burst of crossing flow that the CDA trades in that second, and the resting orders around it are the same orders in both engines. *Reliability: moderate.* The direction is consistent across the measures and robust to the known bias, but the differences are small, partly due to measurement, and significant mainly because of the sample size; the firm conclusion is that batching at one second does not make the market more liquid. The finding holds for $\tau = 1$ s only.

F2 — Transaction costs: an adverse-selection cost in the CDA; the FBA's cost cannot be measured. *Evidence:* effective spread 1.00 bps, realized spread -0.32/-0.51/-0.65 bps and price impact 1.33/1.52/1.65 bps at 1/5/30 s. *Reasoning:* each CDA trade has an observed direction, and a negative realized spread with a price impact that grows with the horizon is the standard signature of adverse selection (Section 3.2.3). *Why:* a resting order is a standing offer that anyone can accept, and it is accepted most often by traders who expect the price to move in their favour. The maker earns the spread at the moment of the trade, but the price then moves against it by more than that, so on average it loses. Hyperliquid's processing of cancellations before new orders (Section 5.2.1) protects quotes against being picked off within a block, but not against better-informed traders. *Reliability: high for the CDA,* where the pattern holds at all three hori-

zons; the magnitudes describe the replay, in which part of the volume comes from post-only orders that would not have traded as takers on the venue (Section D.5). Which engine is cheaper to trade in cannot be said, because FBA trades have no direction (Section 6.7.1).

F3 — Volatility: the FBA is slightly more volatile. *Evidence:* daily volatility of 2.11 bps for the FBA against 2.05 bps for the CDA over 31 days (p <0.001), computed over the same instants for both engines. *Reasoning:* using the same instants removes any difference in when the two prices are observed, so the gap comes from the prices themselves. *Why:* uniform clearing replaces the several prices at which the CDA trades within a second with one, so the price variation the CDA spreads across a second appears in the FBA as a move from one second to the next. In addition, the CDA's VWAP averages several fills and is therefore smoother than a single clearing price, which raises the FBA's measured volatility even with no economic difference. *Reliability: low to moderate.* The gap is about three percent and partly a matter of how each price is formed; the firm conclusion is that batching at one second does not dampen price moves, as theory predicts it should (Table 3.1).

F4 — Price formation: no difference. *Evidence:* a VWAP gap with a median of 0.00 bps and a standard deviation of 0.85 bps over 545,573 seconds; Kyle's lambda at 5 s of 0.638 against 0.642 bps per 1,000 SOL, with p 0.221 for the difference, over $n = 786,990$ seconds. *Reasoning:* if batching moved the price away from the continuous one, the paired price gap would show it; and both engines are regressed on one regressor, the venue's own net taker flow, produced by neither engine, so the difference in slopes is tested directly (Section 2.5.2). *Why:* the price is set by the same limit prices in both engines. When an arriving order meets the resting orders, the CDA trades at each resting price in turn, while the FBA trades all of it at one price, the last level the order reaches; the two prices therefore coincide when an order takes one level and differ, in either direction, when it sweeps several, which is why the gap has a median of zero and widens only in fast markets. Net buying removes the same resting sell orders in both engines, so once a burst of flow is over both prices settle at the same level. Within the same second the FBA's price moves further (0.536 against 0.436), because the clearing price jumps to the last level a large flow reaches while the CDA's VWAP averages the levels on the way. The theory that predicts a smaller response under batching works through behaviour, liquidity providers quoting more aggressively once protected from sniping, which a replay of fixed orders cannot reproduce. *Reliability: high.* The slopes at 5 s differ by under one percent, and a sample of this size would detect a small difference; formally the result is that no difference is found, not a proof of equality.

F5 — Execution: the same volume, faster in the CDA. *Evidence:* executed volume of 58.08 against 58.37 SOL per second (p 0.215); time to first fill of 19.28 s in the CDA against 22.80 s in the FBA (p <0.001). *Reasoning:* both engines can only trade the orders that were submitted, so equal volume means batching leaves no tradable flow unmatched over the month; the longer wait is significant and in the direction batching implies. *Why:* batching reschedules trades rather than preventing them: an order that would cross on arrival in the CDA waits for the next

clear, and unfilled orders roll over instead of being dropped. The wait for the clear is at most one second, half a second on average for orders arriving evenly. The empty batches do not explain the rest, because the CDA has as many seconds without a trade (Section 6.1). The rest most likely comes from which resting orders are matched once a batch crosses: the batch fills the orders eligible at its single price in price-time order within the batch, while the CDA fills an arriving order level by level, so the two engines can fill different resting orders from the same flow; this is not measured directly. *Reliability: high for equal volume and for the direction of the delay, lower for its size.* A fill that comes long after an order's submission falls outside the window the metric is computed on and is not counted, for either engine; because FBA orders wait longer, this more likely understates the gap than overstates it. Absolute volume describes the replay, which trades about twice the venue's volume in both engines (Section 6.7.3).

Trader surplus (37.02 against 37.51 USD per second) is not listed as a finding: it barely changes, as the transfer argument of Section 4.2.3 predicts, and so confirms the theory rather than adding to it.

Table 6.5: The five findings: for each dimension of market quality, the engine that performs better, the key evidence (CDA against FBA) and how far the finding can be relied on (Section 6.6.1).

Dimension	Better	Key evidence (CDA vs. FBA)	Reliability
F1 Liquidity (2.1)	CDA, slightly	depth within 10 bps 32,681.93 vs. 31,515.77 SOL; Amihud 0.09 vs. 0.11; quoted spread 0.84 vs. 0.87 bps	moderate
F2 Transaction costs (2.1)	not measurable	CDA realized spread -0.65 bps, price impact 1.65 bps at 30 s; no FBA counterpart	high (CDA side)
F3 Volatility (2.2)	CDA, slightly	daily volatility 2.05 vs. 2.11 bps	low to moderate
F4 Price formation (2.2)	neither	median VWAP gap 0.00 bps; Kyle's lambda at 5 s 0.638 vs. 0.642 (p 0.221)	high
F5 Execution (2.3)	CDA, faster	volume 58.08 vs. 58.37 SOL/s (p 0.215); time to execution 19.28 vs. 22.80 s	high

Interpretation. On every dimension that can be compared, the batch market either matches the continuous one or falls slightly behind it: the same price, the same response to order flow and the same volume (F4, F5), but slightly less liquidity (F1), slightly more volatility (F3) and slower execution (F5). At one second and on this market, batching largely reproduces the continuous market with a short delay. The gains it promises depend on gathering orders that arrive at different moments into one auction, but trades occur in only about a quarter of all seconds, so a one-second batch usually contains just the orders that would have traded at once in the continuous market. Whether a longer interval would gather enough orders per batch to change this is the main open question this leaves.

What remains open. The simulation does not settle whether batching reduces adverse selection (Section 6.7.1), how participants would behave under batching (Section 7.2), or whether a longer interval would fill the batches, since only $\tau = 1$ s was run.

Limits of the Comparison

This section sets out the limits on comparing the metrics across the two engines: the missing trade direction of the FBA, the metrics that cannot be compared or are compared only with a caveat, and the fidelity of the replay. Table 6.6 closes with how far each metric family can be relied on. Limits of the research design as a whole are discussed in Section 7.2.

The FBA's Trades Have No Direction

FBA trades have no direction (Section 2.3.5), so the FBA spread values are an estimated sign, from a tick rule on the CDA's prices, times the unsigned deviation the simulator records (Section 2.5.1); the sign exists in 545,572 of the 545,573 buckets in which both engines traded. Two points make these values unusable as a cost measure. First, the formulas need the sign times the *signed* deviation $p_k - m_k$. Because both engines' prices move together, the CDA's tick largely follows the FBA's own price move, so the sign times $|p_k - m_k|$ reduces to the plain price change, whose average is the price drift; the near-zero values would appear whatever the FBA's cost. Second, all three FBA measures apply the same sign to the same trades, so their agreement is one result, not three.

The CDA side needs no estimate and stands. The FBA side is reported for completeness but not compared, and no conclusion is drawn from it about whether batching lowers the adverse-selection cost. Measuring that would need the simulator to record the signed deviation of each FBA trade and a direction that does not come from a price, such as the sign of the venue's net taker flow in that second (Section 2.5.2).

Which Metrics Can Be Compared

Most metrics are defined the same way for both engines and are compared directly: executed volume, trade count and VWAP, depth within a threshold, time to execution and trader surplus. Three metrics cannot be compared at all, and five are compared with a caveat.

Not comparable. **Depth at best** compares one CDA price level with the whole eligible side of the FBA schedule at the clearing price, $(D(p^*) + S(p^*))/2$, which adds up all orders priced at or better than p^*. The FBA's higher value is a consequence of this definition, not of more liquidity, so the two are reported but not compared. **Realized volatility within a bucket** needs two prices in one second, which the FBA almost never has; matched-timestamp volatility replaces it. **The spread decomposition** needs a trade direction the FBA does not have (Section 6.7.1).

Compared with a caveat. **Depth within a threshold** carries the depth result. Its one asymmetry, that the FBA's value is read just before the batch clears and so includes orders about to execute, can only make the FBA look deeper, so it does not explain the finding that the FBA is thinner. **Quoted spread** is computed correctly for both engines, but the FBA's is read from the leftover orders just after a clear, which is not the same kind of object as a CDA book that has continuously matched everything it could (Section 2.3.5 and Chapter E). It is therefore weaker evidence about liquidity than depth

within a threshold. **Amihud illiquidity** uses each engine's own reference price, the midpoint for the CDA and the clearing price for the FBA; the clearing price moves between the bid and the ask side with the direction of each batch, which raises the FBA's value (Section 6.3). **Matched-timestamp volatility** compares the CDA's VWAP, an average of several fills, with the FBA's single clearing price, which is less smooth (Section 6.4). **Kyle's lambda** regresses both engines on the same regressor, the venue's own net taker flow per second (Section 2.5.2), so the two slopes are directly comparable. Two limits remain. The regressor is the flow that executed on the venue, not on either simulated engine, and in the FBA the same orders may trade in a different second or not at all. And the same-second slope compares a VWAP with a single clearing price (Section 6.4); the 5 s horizon avoids this.

Fidelity of the Replay

- **Volume level.** The replay does not enforce post-only orders or self-trade prevention, so it trades about twice the venue's recorded volume (Section D.5). Both engines receive this same input, so it does not favour either side of a paired comparison, but absolute volume, trade-count and latency levels describe the replay, not the live market.
- **Rounding.** Sizes are rounded to whole SOL, and orders below 0.5 SOL are skipped for both engines (Section 5.2.2).
- **Implementation.** A difference could in principle come from how well each engine is implemented. This is checked by the shared order representation and metrics code, the independent reference implementation and the test suites (Chapter D); convergence of the FBA to the CDA as $\tau \to 0$, the strongest such check, was not run.
- **Engine controls.** The replay runs on the data's timestamps, so computing speed cannot change any market metric. Throughput and latency are wall-clock measurements and the only two of the 35 output columns that are not reproduced bit for bit across runs (Section D.4).

Table 6.6: How far each metric family can be relied on, and why.

Metric family	Reliance	Reason
Executed volume, trade count, VWAP	high	read directly off executed trades, same definition for both engines
Time to execution, trader surplus	high	same definition for both engines, computed from each engine's own fills
Depth within a threshold	high	same definition for both engines; the one asymmetry favours the FBA
Amihud illiquidity	moderate	midpoint for the CDA, clearing price for the FBA; the asymmetry raises the FBA's value
Intra-interval dispersion	check only	zero by the batch rule; confirms each batch trades at one price, not a comparison
Matched-timestamp volatility	moderate	same instants for both engines, but a VWAP is smoother than a single clearing price
Kyle's lambda, 5 s	high	same regressor (venue flow) and same definition for both engines
Kyle's lambda, same second	moderate	CDA price is a VWAP within the second, the FBA's a single clearing price
Spread decomposition, CDA side	high	observed trade direction
Spread decomposition, FBA side	not comparable	no trade direction; estimated sign times unsigned deviation is near zero by construction
Quoted spread	moderate	FBA value read from leftover orders, not a book
Depth at best	not comparable	different objects in the two engines
Realized volatility within a bucket	not comparable	the FBA has at most one price per second; replaced by matched-timestamp volatility
Throughput, latency	feasibility only	no effect on market metrics in an event-time replay

Chapter 7

Conclusion

Answers to the Research Questions

This thesis asked two questions: how a frequent batch auction engine is designed, and what changes when the same real order flow is cleared in batches instead of continuously. The two paragraphs below answer each in turn.

RQ1. A frequent batch auction engine is fully specified by five design blocks: the batch interval, the disclosure regime during accumulation, the order flow segregation architecture, the clearing and rationing rule, and settlement and residual handling. These are not optional extras layered onto the mechanism but decisions that serial processing settles automatically in a continuous engine, and which become visible and contestable only once it is abandoned. Table 4.2 lists every option with its predicted consequence. Three points carry the most weight. The only unambiguous structural gain is discrete accumulation combined with uniform pricing; every other choice trades one distortion for another, so the question is which distortion the venue prefers. The blocks interact and cannot be optimised independently. And a venue can implement frequent batch auctions exactly as proposed and still have a speed race, through two separate channels: price-time rationing inside the batch rewards being early, and a fixed, predictable close rewards being late — the sharpest practical implication of the analysis. Two questions with no settled answer were developed as contributions: the maker–taker distinction dissolves under simultaneous uniform-price clearing, with persistence across batches the strongest of four candidate redefinitions; and the price improvement uniform clearing produces is a transfer between buyers and sellers, not a gain, whose split is set by the clearing-price selection rule. Under a fixed close and a visible batch, that split can be steered by whoever submits last, which turns the choice of time window into a boundary repricing race (Section 4.2.3).

RQ2. The simulation replayed one month of SOL perpetual order flow through both engines, with one-second batches, and compared them second by second on the same orders. It produced five findings, one per dimension of market quality (Table 6.5); Section 6.6.1 explains how each is reached, why it arises and how far it can be relied on.

F1 Liquidity: the continuous market is slightly more liquid (moderate reliability). It is a few percent deeper near the price, and its price moves less per dollar traded; the quoted spread is practically the same. Batching is meant to add liquidity by gathering orders that arrive at different moments into one auction, but trades occur in only about a quarter of all seconds here,

so a one-second batch usually contains just the orders that would have traded at once in the continuous market.

F2 Transaction costs: resting orders pay an adverse-selection cost in the continuous market (high reliability). The realized spread is negative and the price impact grows with the horizon, because resting orders are accepted most often by traders who expect the price to move their way. Whether the batch market is cheaper cannot be measured.

F3 Volatility: the batch market is slightly more volatile (low to moderate reliability). One clearing price per second moves further between seconds than the continuous market's averaged price; batching does not dampen price moves.

F4 Price formation: no difference (high reliability). Both engines reach the same price, and the same net order flow moves both prices by the same amount within five seconds, because it removes the same resting orders in both.

F5 Execution: the same volume trades, but the continuous market is faster (high reliability). Batching reschedules trades rather than preventing them, and orders wait 3.52 s longer for their first fill in the batch market.

In summary, on every dimension that can be compared, the batch market at one second either matches the continuous market or falls slightly behind it; it largely reproduces the continuous market with a short delay. This agrees with field evidence from the Taiwan Stock Exchange: Lee et al. (2026) find that liquidity and price efficiency improved for mid- and small-cap stocks when the exchange moved from frequent batch auctions to continuous trading. Their design captures the participants' response to the mechanism, this one the mechanism alone, and neither finds a liquidity gain from batching. Whether batching protects resting orders from adverse selection, and whether a longer interval would give the batches enough flow to pool, remain open (Section 6.7.1).

Limitations

Participants cannot respond. A replay holds order flow constant, which is what makes the comparison clean, and by the same token prevents participants from responding to the mechanism. The replayed orders were submitted to a continuous engine, so their sizes, arrival times and limit prices were not chosen against a batch rule. The equilibrium predictions of the FBA literature — narrower quoted spreads, size inflation, orders bunching at the batch close, bid shading (Section 4.2.3) — are therefore out of reach, and the findings are the mechanical baseline against which such responses would be measured. The same holds for cancellations: if latency races were present, a continuous market would show more cancellations than a batch market, but the data contain only the continuous market's cancellations.

One venue, one instrument, one month, one interval. Magnitudes should not be carried over to other markets without re-estimation. Hyperliquid processes cancellations before new orders within

each block (Section 5.2.1), so its continuous market already protects resting quotes to a degree; on a purely first-come, first-served exchange the benefit of batching could be larger. And every finding is for $\tau = 1$ s.

Fidelity of the replay. The replay departs from the venue in ways that affect both engines alike — unenforced post-only and self-trade rules and rounding to whole SOL. None favours either side of the paired comparison, but absolute levels describe the replay, not the live market (Section 6.7.3).

Future Work

Each limitation above points to a concrete extension.

- **Measure adverse selection under batching.** Recording the signed deviation of each FBA trade from its reference price, and signing it by the venue's net taker flow in that second, would give the FBA a spread decomposition comparable with the CDA's (Section 6.7.1) and answer the question F2 leaves open.
- **Vary the interval.** A sweep over τ would show whether longer intervals fill the batches enough to reverse F1 without changing price formation (F4), and the limit $\tau \to 0$ would check that the FBA converges to the CDA.
- **Let participants respond.** Agent-based simulation or laboratory experiments built on the same engines could test the behavioural predictions a replay cannot reach.
- **Other instruments and venues**, to see whether the findings hold beyond one perpetual contract on one exchange.

Bibliography

Albers, J., Cucuringu, M., Howison, S., and Shestopaloff, A. Y. (2026). An open book: Level 4 order book data from the Hyperliquid exchange. *SSRN Working Paper*. Dataset: https://zenodo.org/records/18184441.

Aldrich, E. M. and López Vargas, K. (2020). Experiments in high-frequency trading: Comparing two market institutions. *Experimental Economics*, 23(2):322–352.

Amihud, Y. (2002). Illiquidity and stock returns: Cross-section and time-series effects. *Journal of Financial Markets*, 5(1):31–56.

Aquilina, M., Budish, E., and O'Neill, P. (2022). Quantifying the high-frequency trading "arms race". *The Quarterly Journal of Economics*, 137(1):493–564.

Baldauf, M. and Mollner, J. (2020). High-frequency trading and market performance. *The Journal of Finance*, 75(3):1495–1526.

Budish, E., Cramton, P., and Shim, J. (2015). The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics*, 130(4):1547–1621.

Budish, E., Lee, R. S., and Shim, J. J. (2024). A theory of stock exchange competition and innovation: Will the market fix the market? *Journal of Political Economy*, 132(4):1209–1246.

Burkett, J. and Woodward, K. (2020). Uniform price auctions with a last accepted bid pricing rule. *Journal of Economic Theory*, 185:104954.

Copeland, T. E. and Galai, D. (1983). Information effects on the bid-ask spread. *The Journal of Finance*, 38(5):1457–1469.

Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L., and Juels, A. (2020). Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *2020 IEEE Symposium on Security and Privacy*, pages 910–927.

Daures, L., Moinas, S., and Boussetta, S. (2026). Click first or last? Strategic order submission during the Euronext preopening session. *Management Science*, 72(3):2656–2679.

Deutsche Börse (2026). Xetra: Continuous trading with auctions. Online market model description. Accessed 12 October 2026.

Dutta, A. and Team, C. (2025). A coincidence of wants mechanism for swap trade execution in decentralized exchanges. arXiv preprint arXiv:2507.10149.

Easley, D., Kiefer, N. M., and O'Hara, M. (1996). Cream-skimming or profit-sharing? The curious role of purchased order flow. *The Journal of Finance*, 51(3):811–833.

Eibelshäuser, S. and Smetak, F. (2022). Frequent batch auctions and informed trading. SAFE Working Paper 344, Leibniz Institute for Financial Research SAFE.

Glosten, L. R. and Milgrom, P. R. (1985). Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1):71–100.

Gode, D. K. and Sunder, S. (1993). Allocative efficiency of markets with zero-intelligence traders: Market as a partial substitute for individual rationality. *Journal of Political Economy*, 101(1):119–137.

Gong, T., Liu, Z., and Kate, A. (2023). The case of FBA as a DEX processing model. arXiv preprint arXiv:2302.01177.

Harrison, B. and Pardo-Guerra, J. P. (2025). Hybrid genetic algorithm for optimal user order routing: Multi-objective solver optimization in CoW protocol batch auctions. arXiv preprint arXiv:2510.21647.

Hasbrouck, J. (1993). Assessing the quality of a security market: A new approach to transaction-cost measurement. *The Review of Financial Studies*, 6(1):191–212.

Hasbrouck, J. (1995). One security, many markets: Determining the contributions to price discovery. *The Journal of Finance*, 50(4):1175–1199.

Haynes, R. and Onur, E. (2020). Precedence rules in matching algorithms. *Journal of Commodity Markets*, 19:100106.

Hendershott, T., Jones, C. M., and Menkveld, A. J. (2011). Does algorithmic trading improve liquidity? *The Journal of Finance*, 66(1):1–33.

Hyperliquid (2026). Hyperliquid docs: HyperCore overview, order book and contract specifications. Online documentation. Accessed 12 October 2026.

Jump Crypto (2025). Dual flow batch auction. Jump Crypto research blog, 21 August 2025. Accessed 11 September 2026.

Klemperer, P. (1999). Auction theory: A guide to the literature. *Journal of Economic Surveys*, 13(3):227–286.

Kyle, A. S. (1985). Continuous auctions and insider trading. *Econometrica*, 53(6):1315–1335.

LeBaron, B. (2006). Agent-based computational finance. In Tesfatsion, L. and Judd, K. L., editors, *Handbook of Computational Economics*, volume 2, pages 1187–1233. Elsevier.

Lee, C. M. C. and Ready, M. J. (1991). Inferring trade direction from intraday data. *The Journal of Finance*, 46(2):733–746.

Lee, Y.-T., Wang, K., and Riccò, R. (2026). Frequent batch auctions vs. continuous trading: Evidence from Taiwan. *Journal of Financial Markets*, page 101082.

Madhavan, A. (1996). Market transparency. *Journal of Financial Intermediation*, 5(3):255–283.

Moral Aranda, A. (2026a). market_sim: CDA and FBA matching engines for SOL/USD replay simulation. GitHub repository.

Moral Aranda, A. (2026b). Simulated CDA and FBA output: SOL/USD, december 2025 (replay of the Hyperliquid Level 4 order book).

Newey, W. K. and West, K. D. (1987). A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, 55(3):703–708.

Newey, W. K. and West, K. D. (1994). Automatic lag selection in covariance matrix estimation. *The Review of Economic Studies*, 61(4):631–653.

O'Hara, M. (1995). *Market Microstructure Theory*. Blackwell Publishers, Cambridge, MA.

Ramsay, G., Goel, A., and Mazières, D. (2023). SPEEDEX: A high-throughput, arbitrage-free decentralized exchange. arXiv preprint arXiv:2111.02719.

Schlegel, J. C. (2023). Transaction ordering auctions. arXiv preprint 2312.02055, arXiv.

Vickrey, W. (1961). Counterspeculation, auctions, and competitive sealed tenders. *The Journal of Finance*, 16(1):8–37.

Wah, E. and Wellman, M. P. (2013). Latency arbitrage, market fragmentation, and efficiency: A two-market model. In *Proceedings of the Fourteenth ACM Conference on Electronic Commerce*, pages 855–872.

Walther, T. (2021). Multi-token batch auctions with uniform clearing prices: Features and models. Technical report, Gnosis DEX Research.

Wang, D. N., Zhou, J., Wang, A., and Finestone, M. (2018). Loopring: A decentralized token exchange protocol. Loopring Foundation Whitepaper.

Zoican, M., Haas, M., and Khapko, M. (2021). Speed and learning in high-frequency auctions. *Journal of Financial Markets*, 54:100583.

Appendix A

Batch Clearing Algorithm

The clearing routine implements (2.3) and (2.6) by a search over the limit prices submitted to the batch, followed by the tie-break cascade and the rationing step of Section 4.5.3. Section B.4 describes the same routine in words, together with the data structures behind it.

```
INPUT : bids B and asks A, each order with limit pi (none for a
        market order), remaining quantity q and submission time t;
        rationing rule R; previous clearing price p_prev
OUTPUT: clearing price p*, fills F, residual E

1  candidates <- distinct limit prices in B and A
2  if candidates is empty: return (no clearing, all orders roll over)
3  for each p in candidates:
4      D(p) <- market buy qty + sum of q over buy limits with pi >= p
5      S(p) <- market sell qty + sum of q over sell limits with pi <= p
6      V(p) <- min(D(p), S(p))
7  Vmax  <- max over p of V(p)
8  M     <- { p : V(p) = Vmax }                      // the set P*
9  M     <- argmin over p in M of |D(p) - S(p)|      // tie-break step 2
10 if |M| > 1: M <- argmin over p in M of |p - p_prev|  // step 3
11 if |M| > 1: M <- argmin over p in M of p             // step 4
12 p*    <- unique element of M
13 eligible buys  <- market buys and buys with pi >= p*
14 eligible sells <- market sells and sells with pi <= p*
15 sort each side by R (here: market first, better pi, earlier t)
16 match the two sorted lists front to front at p* until one
       side is used up; record each fill in F
17 E     <- every order with quantity left
18 roll E over into the next batch; if Vmax > 0, p_prev <- p*
19 return (p*, F, E)
```

Listing A.1: Batch clearing.

Steps 1–8 are the price search: D and S only change value at a submitted limit price, so the volume-maximising price is always among the candidates, and since D is non-increasing and S non-decreasing the maximising set is an interval. A batch with no limit order at all has no price of its own and does not trade (step 2). When $V_{\max} = 0$ the remaining steps produce no fill, so every order rolls over as well. Steps 9–11 are the tie-break cascade: minimise the residual imbalance, then the distance from the auction's own previous clearing price p_{prev} (not an external reference, since the dataset has no oracle price), then take the lowest price. Step 15 is the only place where the rationing rule enters,

so a different rule would change the allocation without touching the price computation. Because step 16 walks each side in one priority order rather than first filling every inframarginal order and then rationing the marginal ones, an order priced strictly better than p^* is filled completely in the typical case but not always: when the maximising interval spans more than one price, a better-priced order on the *longer* side can be left only partly filled (Section D.2).

Appendix B

Engine Implementation and Configuration

This appendix explains how the two matching engines are built: how an order is represented, how the replay feeds the engines, which data structures each engine keeps and what it does with every incoming message, and which settings were fixed for the run. It describes the logic of the implementation rather than its source code; the clearing step is also given as pseudocode in Chapter A.

Orders and Trades

Both engines work on the same two record types. An *order* carries its order number, the participant that submitted it, its side, its type, its limit price, its size, its submission time in nanoseconds and the status of the record it came from. To these the engines add one field of their own, the *remaining quantity*, which starts at the order's size and is the only field matching ever changes. A *trade* records the price, the quantity, the order numbers and participants of the buyer and the seller, the time, and which engine produced it.

Three conversions are applied when a record of the dataset is read.

- **Prices are integers.** A price is stored as a whole number of millionths of a dollar (10^{-6} USD). All comparisons between prices, such as whether a limit price equals the clearing price, are therefore exact, with no floating-point rounding.
- **Quantities are whole SOL.** A size is rounded to the nearest whole SOL, with halves rounded up. A record whose size rounds to zero, an order smaller than 0.5 SOL, is skipped; this affects 2.65% of all records (Table 6.1) and both engines identically.
- **Order types are reduced to two kinds.** Limit, stop-limit and take-profit-limit orders become *limit orders* at their limit price. Market, stop-market, take-profit-market and vault-close orders become *market orders*, which have no limit price.

The status of a record decides what the engines do with it (Section 5.1.5): an `open` record of an ordinary order or a `triggered` record of a conditional order becomes a new order, a record with one of the eight cancel statuses removes the order with the same number, and every other record is ignored.

The Replay Loop

The replay reads the hourly files in time order and handles one record at a time. Each record goes through the same four steps.

1. **Start the clock.** The first record of the run sets the origin t_0 of the measurement grid (2.8) and opens the first batch, which is due to close at $t_0 + \tau$.
2. **Close due batches.** While the record's timestamp is at or after the next closing time, the current FBA batch is cleared and the closing time moves forward by τ. The FBA therefore has no clock of its own: a batch closes when the first record at or after its closing time arrives, and after a silence in the data several empty batches are closed one after another. Because the check comes before the record is handed over, a record stamped exactly at a closing time belongs to the next batch, which is the half-open window $[0, \tau)$ of Section 2.3.
3. **Hand the record to both engines.** A new order or a cancellation is given first to the FBA, which only stores it, and then to the CDA, which matches it at once.
4. **Record the state.** The CDA's best bid and ask, their sizes, its total depth and any trades are passed to the metrics code after every record. The FBA's outcome is passed on at every clear.

Computing time is measured around each CDA submission and around each FBA clear. These are the only two wall-clock measurements in the run, and they produce the throughput and clearing-latency controls of Section 2.5.4.

The Continuous Engine

Data structures. Each side of the book is a sorted map from a price to a queue of the orders resting at that price. The best ask is the lowest key of the ask map and the best bid the highest key of the bid map, so both are found without scanning the book. Within a queue, orders are kept in order of submission time, then order number, which is time priority. An index from order number to side and price lets a cancellation go straight to the right queue. Running totals of resting quantity and of the number of orders on each side are updated with every change, so depth is available at any moment without adding up the book.

A new order. For an incoming buy order (a sell is symmetric):

1. Look at the first order in the queue of the lowest ask price. If the incoming order is a limit order whose price is below that ask, stop.
2. Otherwise trade min(remaining of both) at the *resting* order's price, stamped with the incoming order's time, and reduce both remaining quantities by it.

3. If the resting order is now fully filled, remove it from its queue and from the index, and remove the price level if its queue is empty. Repeat from step 1 while the incoming order has quantity left.

4. If a limit order still has quantity left, it joins the back of the queue at its own price. If its timestamp is earlier than that of the last order already there, which happens when the data are slightly out of order, it is inserted at its time-ordered position instead. Whatever remains of a market order is discarded, because a market order never rests.

A cancellation. The index gives the side and price of the order; the order is removed from that queue, together with the level if it becomes empty, and the running totals are reduced by its remaining quantity. A cancellation for an order that is not resting, because it was already filled or never entered, changes nothing.

The book keeps one structural rule at all times: a price level exists in the map only while at least one order rests at it. A violation would stop the run (Section 5.5).

The Batch Engine

Data structures. The FBA keeps a single list of pending orders, which holds the new orders of the current batch together with everything rolled over from earlier batches. It also remembers the last price at which a batch actually traded, p_{prev}, which the tie-break needs.

A new order or a cancellation. A new order is appended to the list and nothing else happens until the batch closes. A cancellation removes the order with that number from the list, including any part of it that was rolled over; as in the CDA, an unknown order number changes nothing.

Clearing a batch. At each close the engine runs the following steps (Listing A.1).

1. **Candidate prices.** The candidates are the distinct limit prices in the list. $D(p)$ and $S(p)$ only change value at these prices, so the volume-maximising price is always one of them. If there is no candidate, because the list is empty or holds only market orders, nothing trades and every order stays in the list.

2. **Demand and supply.** The quantities of market buy and sell orders are added up once, since they count at every price. Bid limits and ask limits are each sorted by price and given a running cumulative sum, so $D(p)$ and $S(p)$ at any candidate are read off with one binary search instead of a pass over the whole list.

3. **Price selection.** The candidates are compared one after another under the cascade of Section 2.3: larger $V(p)$ first, then smaller $|D(p) - S(p)|$, then smaller distance to p_{prev}, then the lower price. The winner is p^*.

4. **Eligible orders.** On each side the orders that accept p^* are selected: every market order and every limit order priced at or better than p^*. They are sorted by priority: market orders first, then the better limit price, then the earlier submission time, then the order number.

5. **Matching.** Two pointers walk down the sorted buy list and the sorted sell list together. At each step the two current orders trade $\min(\text{remaining of both})$ at p^*, and the pointer of whichever order is used up moves on. The walk stops when one side runs out, which happens after exactly $V^* = \min\{D(p^*), S(p^*)\}$ has traded, because every eligible buyer accepts every eligible seller at p^*. All trades of the batch are stamped with the latest submission time among the batch's orders.

6. **Residual.** Every order with quantity left, whether eligible but rationed out or not eligible at all, stays in the list for the next batch with its original limit price and submission time, so it keeps its time priority. p_{prev} is updated only if the batch traded.

The cost of a clear grows with the size of the whole list, not only with the orders that arrived in the last second, because rolled-over orders are evaluated again in every batch. This is why one FBA clear takes far longer than one CDA submission (Section 6.5).

Fixed Configuration of the Run

Table B.1 lists the settings of the run. The batch interval is the only one that can be changed when the simulator is started; every other entry is built into the engines, so there is no switch that would select a different rule.

Table B.1: Configuration of the run reported in this thesis.

Setting	Value
Engines	CDA and FBA, fed in one pass with the same records
Batch interval τ	1 s; also the width of a measurement bucket
Batch close	fixed, triggered by event time; no randomisation
Candidate prices	the submitted limit prices of the batch
Tie-break cascade	volume, then imbalance, then distance to p_{prev}, then lowest price
Rationing	price-time priority within the batch
Residual	always rolled over; never cancelled by the engine
Price unit	10^{-6} USD (exact integer arithmetic)
Quantity unit	1 SOL; sizes rounded half-up, below 0.5 SOL skipped
Not implemented	time-in-force, post-only, self-trade prevention, fees and rebates, netting

The missing self-trade prevention is a genuine limitation, not a policy applied to both engines on purpose (Section 5.3); the effect of the missing time-in-force and post-only rules is measured in Section D.5.

Appendix C

Reproducibility

Every run's checkpoint file records the configuration of Table B.1 together with the input source path, the interval τ, and per-engine running totals, so that a resumed run can be checked against the run it continues. A per-file input hash and a code commit hash embedded in the run's own output — which would let a result be tied unambiguously to the exact code and data that produced it, beyond what the configuration and file list already establish — are not currently written by the checkpoint format; this is listed here as a gap worth closing, not implied to already exist. Given identical input and configuration, an uninterrupted run reproduces identical output in all columns except the two that measure computing time (Section D.4). The analysis pipeline is shared by both engines, so no metric is computed by engine-specific code. It reads the logged CSV output, plus one series taken from the raw archive itself: the venue's net taker flow per second, the common regressor of Kyle's lambda (Section 2.5.2), which the verification script `analysis/verify_raw_data.py` extracts from the order-status records.

Getting the Code

The full source, in Rust, is public (Moral Aranda 2026a). It lives at `https://github.com/alejandromoralwork/tfg`, branch `main`, commit `931dac246eb22e135691707f1fc195c98c8e903c`.

It is a single program, `market_sim`, that needs nothing beyond the standard Rust toolchain. Both matching engines, the decoder for the binary data format, the metrics code and the command-line interface used below are part of it; none of them is an external package. Cloning and building it is the same on any platform:

```
git clone https://github.com/alejandromoralwork/tfg.git
cd tfg/src
cargo build --release
# binary at target/release/market_sim (market_sim.exe on Windows)
```

Listing C.1: Getting and building the simulator.

Started without arguments, the program opens an interactive prompt; started with arguments, it runs that one command and exits, which is the form every run in this thesis used.

Running It Without Downloading Any Data

The repository carries a small, git-tracked sample (one hour of real SOL order statuses, a few hundred thousand records) so the whole pipeline can be exercised in well under a minute, with nothing downloaded first:

```
market_sim simulate data/sample/order_statuses/20251201 1
# writes output/20251201/{fba,cda}_timeseries.csv, summary.txt, checkpoint.
    txt
market_sim test engine all
# 0 exit status only if all 37 hand-built checklist cases pass
```

Listing C.2: A first run, on the bundled sample data.

Figure C.1 shows the same binary's interactive prompt: switching engine, adding orders by hand, reading back the book, and clearing an FBA batch, exactly the behaviour described in Chapters A and B. This is a real captured session; the terminal colours shown are reapplied from the command-line interface's colour calls because the ANSI codes themselves are dropped when the output is piped into a file rather than shown on a real terminal.

Running the Full Month

Reproducing the actual results in Chapter 6 needs the real December 2025 SOL archive (about 6 GB compressed), fetched from the Zenodo record cited as Albers et al. (2026), at `https://zenodo.org/records/18184441`:

```
market_sim download sol            # fetches + extracts into data/
    order_statuses/sol
market_sim simulate sol 1          # the exact run behind this thesis; ~45h
    on one machine
```

Listing C.3: Fetching the real dataset and replaying the whole month.

A single machine is slow only because the replay is single-threaded by design: it feeds every record to both engines one at a time, in timestamp order (Section 5.1), so it cannot be split across processor cores; the run can be stopped and resumed at any point, since it checkpoints after every hourly file. For a faster turnaround, the repository includes a complete, copy-paste walkthrough for running the identical container on Google Cloud Batch, including the exact job specification used by this thesis's own cloud run. It takes a reader who has never used Google Cloud before from installing `gcloud` to a finished run in their own project, typically a few dollars of compute for the full month.

This thesis does not, and deliberately does not, publish a shared key or a running virtual machine that lets a reader execute the simulation on the author's own cloud account: a thesis PDF is effectively public forever, and a live credential embedded in it cannot be safely revoked once distributed, while leaving compute running for anonymous use is an open invitation to abuse and an open-ended cost. The repository's own walkthrough gives every step needed to stand up the same environment, in a reader's own project, in its place.

```
market_sim.exe -- interactive prompt

======================================================================
TFG Simulator
======================================================================

sim [FBA]> engine cda
[OK] Switched simulation matching engine mode to: Continuous

sim [CDA]> add sell 127.50 10 maker1
[OK] Continuous order processed. Instant trades cleared: 0

sim [CDA]> add buy 127.50 4 taker1
[OK] Continuous order processed. Instant trades cleared: 1
   [TRADE] Qty: 4 at Price: 127.500000

sim [CDA]> orderbook
---  Current Continuous Order Book State ---
Best Bid: n/a  |  Best Ask: 127.500000  |  Spread: n/a
Bids (0):
Asks (1):
  ID: 1   | User: maker1   | Qty: 6    | Price: 127.500000
--- CDA ---
  quoted_spread_bps        : n/a
  depth_at_best            : 6
  trade_count              : 1
  executed_volume          : 4
  executed_notional        : 510
  fill_rate                : 0.5714
  book_imbalance           : -1.0000

sim [CDA]> engine batch
[OK] Switched simulation matching engine mode to: Batch

sim [FBA]> add buy 127.40 6 Alejandro
[OK] Queued order successfully in FBA discrete window buffer [ID: 3]

sim [FBA]> add sell 127.40 9 Jesus
[OK] Queued order successfully in FBA discrete window buffer [ID: 4]

sim [FBA]> batch
---  Current Pending FBA Window Accumulation Buffer ---
Best Unfilled Buy: 127.400000  |  Best Unfilled Sell: 127.400000
Buy orders (1):
  ID: 3   | User: Alejandro | Qty: 6    | Max Limit: 127.400000 USDT
Sell orders (1):
  ID: 4   | User: Jesus     | Qty: 9    | Max Limit: 127.400000 USDT

sim [FBA]> clear
[FBA Window Closed] Computing Uniform Market Clearing...
============================================================
[OK] Uniform Clearing Calculated Successfully!
   Execution Rate (Uniform Price) : 127.400000 USDT
   Total Executed Asset Mass      : 6 units

Detailed Execution Trade Log:
   Match ID #1   | Alejandro (Order #3) bought 6 units from Jesus (Order #4) @ 127.400000 USDT
[INFO] 1 order(s) left unexecuted at this clearing price -- rolled over to the next window.

sim [FBA]> log
======================================================================
                    SYSTEM HISTORICAL EXECUTION LOG
======================================================================
  Trade ID   | Eng  | Buyer       | Seller      | Quantity   | Price
  ----------------------------------------------------------------------
  #1         | CDA  | taker1      | maker1      | 4          | 127.500000
  #1         | FBA  | Alejandro   | Jesus       | 6          | 127.400000
======================================================================

sim [FBA]> exit
Terminating simulator core workspace...
```

Downloading the Exact Simulated Output Used Here

Running the full month, whether locally or on Google Cloud Batch, is the only way to reproduce these result files from the raw order-status archive, but it is not the only way to obtain them: the two time-series CSVs and the summary report the run above actually produced — the same files Section C.1's repository turns into every number and figure in Chapter 6 — are archived on Zenodo with their own permanent DOI (Moral Aranda 2026b), at `https://zenodo.org/records/22845443` (DOI `https://doi.org/10.5281/zenodo.22845443`). This is deliberately not a personal file share: the two CSVs are, at roughly 640 MB and 630 MB, far over GitHub's 100 MB file limit and so cannot be committed to the repository itself (Section C.1), and a long-lived, independently citable record is the right place for data a result in this thesis depends on, not an ad hoc link that can move or expire. The record contains:

- **FBA timeseries** (`fba_timeseries.csv`, about 630 MB);
- **CDA timeseries** (`cda_timeseries.csv`, about 640 MB);
- **Summary report** (`summary.txt`, a few kilobytes, with per-column minimum, mean and maximum and the run's running totals).

Each file is exactly what `market_sim simulate sol 1` writes to the results directory, byte for byte, from the run behind this thesis. The analysis pipeline recomputes every macro, table and figure in Chapter 6 directly from them, without needing to replay the month at all, with one exception: Kyle's lambda also needs the venue's net taker flow per second, which comes from the raw archive rather than from the simulation. Running `analysis/verify_raw_data.py` once over the downloaded archive (Section C.3) writes it; this reads the records but does not replay them through either engine.

Appendix D

Testing and Verification

This appendix summarises how the data and the simulator were checked. No amount of testing proves a program has no errors; it can only make specific errors hard to miss. Two independent test systems back the engines and the metrics: a suite of 85 automatic Rust tests, all 85 passing (4 of them need real data or system tools and are run separately), and a checklist of 37 hand-worked scenarios (15 CDA, 14 FBA, 8 metrics) compiled into the simulator itself, so it runs on the finished executable with no development tools. On top of these, three further checks were built specifically for this appendix: an independent second implementation of both engines, deliberately planted bugs to confirm the tests can catch mistakes, and a comparison of the simulated output with what happened on the venue. All numbers below come from the scripts listed in Section D.7, run on the full December 2025 data and result files.

The Input Data

The 54-byte binary format is decoded by dedicated unit tests, including one that decodes the literal bytes of a real record and checks every field against the value printed in the dataset's own preview file. Because unit tests on a few records cannot show that all 640,176,812 are read correctly, the whole archive was decoded a second time by an independent program that shares no code with the simulator. Table D.1 compares its counts with the simulator's own `scan` command and its running totals: all three agree exactly, and the four record categories add up to the total with no remainder.

Table D.1: Record counts for the December 2025 SOL archive from three independent programs.

Quantity	Scan command	Simulation	Second decoder
Hourly files read	744	744	744
Total records	640,176,812	640,176,812	640,176,812
New live orders	283,797,135	—	283,797,135
Cancellations	281,778,306	—	281,778,306
Other records (fills, rejections, . . .)	57,615,417	—	57,615,417
Size rounds to zero (skipped)	16,985,954	16,985,954	16,985,954

The second decoder also checked ten structural conditions across every file and record — file sizes, boolean fields, status/type/time-in-force codes, size ordering, and timestamp order — and found none violated. Three smaller imperfections in the data itself are worth noting, since they affect the results in small ways discussed later: 18 records have an earlier timestamp than the one before them (by up to 581 s, counted by the replay as late events, 2,075 of them for the CDA); 14 silences longer than a minute occur, the longest 68 minutes; and 0.17 percent of records (1,101,027) are byte-for-byte

repeats of an earlier record in the same file, concentrated in a few hours. A repeated `open` record puts a second copy of the same order into the replay, which is the reason a handful of one-second buckets later show more quantity filled than submitted.

Sorting every record by order number and replaying each order's own history checks that it is opened once, never grows in size, and (with one exception in 60,378 orders, all but one of them conditional orders) has one consistent submission time throughout. A short section of the dataset's own preview file, with real trades from the first three seconds of the month, was traced back into the order-status records by hand and confirms how fills are recorded: 28.14 of 28.26 SOL of real trades are recovered this way (the rest is one order that never appears in the archive at all).

The Engines

Both engines were checked three ways. First, a randomised property test: a seeded stream of orders and cancellations runs through the real engine and, in parallel, through a deliberately naive reference (a full scan of every resting order for the CDA; a brute-force scan of every possible price for the FBA), compared after every event or clearing. This covered 360,000 CDA events across 3,000 scenarios (book never crossed, every trade at the resting order's price, no order filled beyond its size) and 36,000 FBA clearings (the price chosen always the true volume-maximiser among *every* possible price, with the allocation following price-time priority exactly on both sides). That priority walk is also what confirms a claim made in Section 4.5: a better-priced order is only ever left partly filled on the longer side, and only when the volume-maximising price range spans more than one price — never on the shorter side, and never when a single price maximises volume.

Second, 15 and 14 scenarios worked out by hand cover the ordinary cases (a plain cross, a multi-level sweep, ties with and without price history) and the edge cases that are easy to implement incorrectly (an empty book or batch, a market order with no liquidity, a cancel of an unknown order, roll-over of a leftover); three are regression tests for real defects found while building the simulator.

Third, an independent Python re-implementation of both engines (sharing no code or data structures with the Rust one, decoding the raw records itself) was run against four full hours of real data: the sample hour, a quiet night, the busiest hour of the month, and an hour with many repeated records. Table D.2 shows every prediction it made — trades, volumes, prices, depths, and for the FBA the unexecuted share — matched the Rust output in all 29,446 one-second buckets checked.

Table D.2: Agreement between an independently written reference implementation and the Rust simulator, four full hours. 0 disagreements across 29,446 engine-buckets.

Hour	Records	Agreement
1 Dec, 12:00–13:00 (sample hour)	1,219,826	every bucket, both engines
25 Dec, 04:00–05:00 (quiet night)	294,268	every bucket, both engines
15 Dec, 14:00–15:00 (busiest hour)	2,623,879	every bucket, both engines
20 Dec, 09:00–10:00 (many repeated records)	387,906	every bucket, both engines

Finally, to check the tests themselves are strict enough to matter, 19 realistic bugs were planted one at a time in a copy of the source (never the real engines) — e.g. changing the CDA's crossing rule from "at or above" to "strictly above", or the FBA's tie-break to prefer the larger imbalance instead of the smaller one. Of the 19 planted bugs, 19 are detected by at least one layer. 2 of them (F1, F8) are detected only by the property tests, which is where the new tests add most. Every bug in the FBA's choice and rationing of prices is caught by the property test, and every bug in the CDA's matching is caught by it as well.

The Metrics and the Result Files

17 unit tests check individual metric formulas against numbers worked out by hand, including recovering an artificially planted Kyle's-lambda slope to nine decimal places, and confirming that streaming the output file in pieces (as the real run does) gives byte-identical results to writing it all at once. A further eight checklist scenarios drive the metrics code with synthetic events and compare every output column with a hand-computed value. The independent reference implementation above adds a third, larger-scale check: it recomputes quoted spread, depth, effective spread and the unexecuted share from scratch for every second of the same four hours, and matches (Table D.2).

The two month-long result files were then checked for internal consistency: every relation that follows from a metric's own definition (for example, that price impact equals effective spread minus realized spread, that a spread is never negative, that VWAP times volume equals notional) was tested on all 2,678,400 rows of both files. Of dozens of such relations, only two are ever violated, and both are understood rather than being errors: the FBA's price dispersion, which should be exactly zero, is not in 19,468 buckets where two separate batches happened to contribute a trade to the same second; and the filled share of submitted quantity, which should never exceed one, does so in six rows out of 2,654,250, all traceable to the repeated input records noted above. Recomputing eleven columns independently from the raw CSV files, rather than trusting the simulator's own running summary, matched to a relative error of at most 10^{-8} in every case but one (the simulator's own per-batch FBA Kyle's lambda, 2.4×10^{-4}, a sum of near-cancelling values; the comparison in Chapter 6 does not use this value, Section 2.5.2). Every number quoted anywhere in this thesis is generated the same way, by a script that also checks that no macro is left undefined and no cross-reference is left dangling; this process caught and corrected one real error, a wrongly based percentage of skipped records.

Determinism

The engines contain no random numbers and no clock, so the same input should always give the same output. Replaying the sample hour twice from scratch, and separately replaying 500 random flows through the property tests twice each, confirms this: of the 35 output columns, 33 are byte-identical between the two runs. The two that are not, throughput and clearing latency, are measured wall-clock timings and are labelled as controls for that reason, not as results.

Does the Replay Match the Real Venue?

Everything above shows the engines do what their rules say. A different question is whether those rules, applied to the real order flow, give a realistic picture of what happened on the venue — and the answer is only partly yes. The archive itself contains the venue's own record of every taker fill (an order filled the moment it was submitted), so the venue's true traded volume can be measured directly from data the simulator never even reads. Over the month this comes to 76.81 million SOL, while the simulated volume is 155.57 million for the CDA and 156.35 million for the FBA — about twice as much. The two track each other closely in time (correlated at 0.93 per day) but not in level.

The reason is that the replay does not enforce two rules the real venue does: an order marked "post-only" can never trade as a taker on the real venue, and self-trades are prevented there but not here. Measuring these directly on a random sample of 24 whole hours shows that 43 percent of the simulated volume comes from post-only takers and 1.7 percent from a participant trading with themselves; removing the post-only volume brings the simulated total to within a few percent of the venue's (0.95 times it, against 1.67 times before). Both engines receive this same imperfect input, so it does not favour either side of the paired comparison in Chapter 6, but the absolute levels of volume and everything computed from it describe the replayed flow, not the venue exactly. This is the largest limitation this verification found.

Does the Price Still Track the Real Market?

The volume mismatch above is about *how much* trades; it says nothing about *at what price*. Comparing each engine's hourly volume-weighted price against real hourly SOL-USD spot candles from Yahoo Finance (Figure D.1; the perpetual is held close to the spot price by funding, so the two are comparable at an hourly scale) shows the simulated price tracks the real one closely all month: a mean absolute difference of about 12.4 bps for the CDA and 12.5 bps for the FBA, roughly 0.1% of the price, with the simulated price falling inside that hour's real trading range 97.4/97.3 percent of the time. The two engines differ from each other by only 0.6 bps on average, so this small gap to the real market is a property of the replay, not a difference between the mechanisms being compared.

Was the Month One Continuous Run?

The simulator can save a checkpoint and resume a stopped run, but the code that resumes creates both engines again from empty, losing every resting or pending order. This leaves a visible trace: the CDA's book depth briefly collapses at the start of an hour, and FBA batches can afterwards straddle the timing grid. Looking for this trace shows 10 such restarts, all in the first five days of the month and none afterward.

Because both engines restart at the same record, the paired comparison between them is affected equally; but book-state measures, especially CDA depth, are understated for some hours afterward. The FBA's batch clock is also not realigned with the metrics grid after a restart. Before the restart of 2 December at 19:00 no bucket holds trades from two batch clears; from then until the end of the

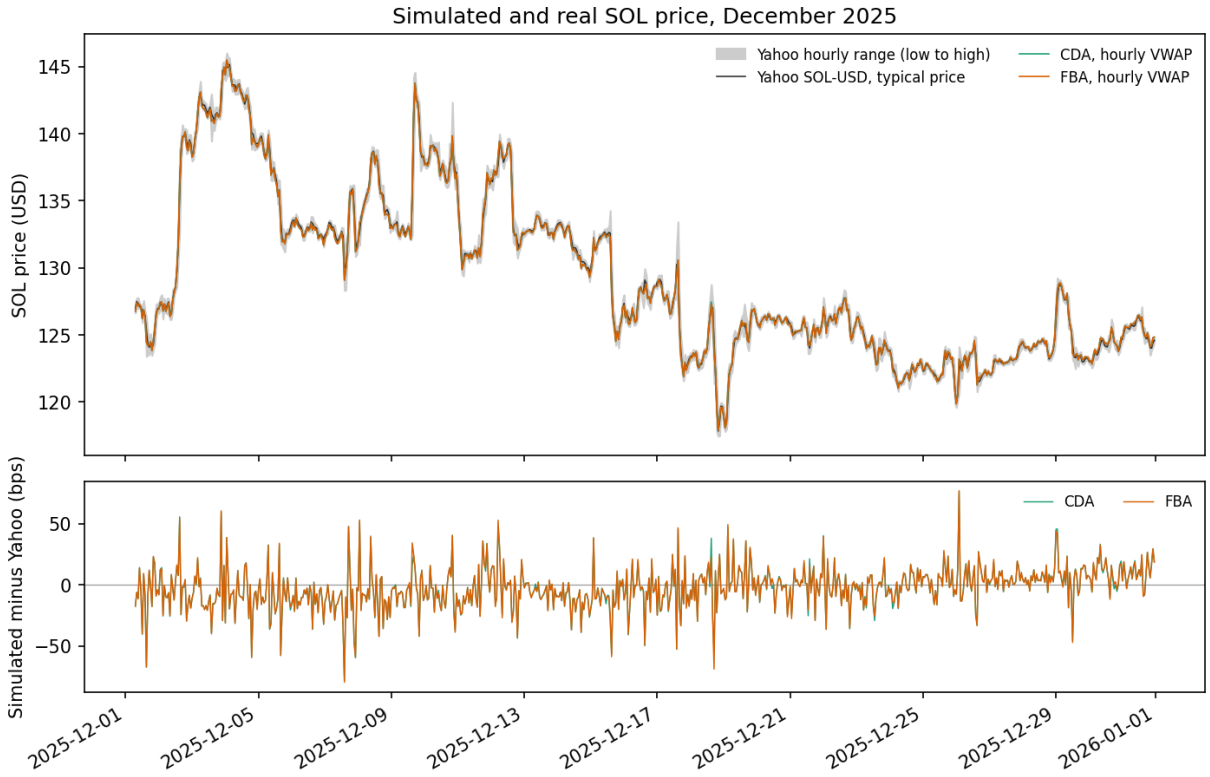


Figure D.1: Hourly volume-weighted price of each engine against the real SOL-USD price (Yahoo Finance), whole month. Bottom: simulated minus real price, in basis points.

month about 3.4% of the buckets in which the FBA traded do, because a batch then straddles two one-second buckets and its trades are stamped with the latest submission time in the batch (Section B.4). This is why the FBA's price dispersion is not exactly zero in 19,468 buckets. An uninterrupted replay of the month would remove both effects; this work did not run one.

What Was Not Checked

Three checks fall outside this verification. The convergence of the FBA to the CDA as $\tau \to 0$ was not tested, since it needs a sweep over several interval widths that this thesis does not include. The full month was not replayed by the independent Python reference, only four hours in detail and 24 more for volume, because it is far slower than the Rust simulator. And the repeated records and data silences noted above were measured but not corrected in the results.

Everything here can be reproduced: `cargo test` runs the automatic tests, `market_sim test engine all` the checklist, and the verification scripts described above reproduce every number in this appendix.

Appendix E

Comparability of the FBA's Unmatched Orders and the CDA Book

A natural first intuition is to treat the FBA's set of not-yet-matched orders at time t as a stand-in for "the CDA order book at t," and then compute the usual book-based metrics (quoted spread, depth, book imbalance) on it the same way they are computed on the CDA. This appendix explains why that intuition does not hold, and what the metrics defined in Section 2.5 do instead.

The Question

A CDA order book at t is a *settled* object: every order that could trade against the book already has, continuously, one order at a time. Nothing sits in it by accident of timing. The FBA has no such object between clears — only a buffer of orders accumulating toward the next batch, and, once that batch clears, whatever is left over. Two candidate readings of "the FBA's book at t" exist, and they fail for different reasons.

Pre-Clearing: Settlement Mismatch and Inflated Depth

Read immediately *before* the batch containing t clears, the pending set mixes two kinds of order that must be kept apart. Using the notation of Section 2.3: some pending orders are inframarginal at the price the batch is about to clear at (every buy with $\pi_k > p^*$, every sell with $\pi_k < p^*$) and will execute the instant the gate closes — they are not resting liquidity in any economic sense, only flow that batching has temporarily delayed. Others are genuinely marginal or out of the money and will remain unmatched. Counting the whole pending set as if it were book depth overstates FBA liquidity purely as an artefact of the batching delay, not because more liquidity is available.

The deeper problem is one of kind, not convention. The CDA book at t reflects a mechanism that has already finished matching everything it can as of t. The FBA's pending set at the same t is, by construction, *unsettled*: matching for the interval containing t has not happened yet. Comparing the two is comparing a post-matching state to a pre-matching one, not two readings of the same kind of object taken at different conventions.

Post-Clearing: Lopsidedness and a Different Kind of "Still There"

Read immediately *after* the batch containing t clears, the residual — what rolls into the next batch — fixes the settlement problem: both objects are now "what is left once everything that could match, did." Two problems survive even so.

First, the residual is lopsided at the price that matters. Orders priced away from p^* on both sides roll over, so the leftover set usually has a best buy and a best sell. At p^* itself, however, it is one-sided: a batch that trades fills its shorter side completely, so whatever is left at p^* belongs to the longer side only, and most batches do not cross at all (Section 6.5). An imbalance read off this set describes which side the last batch could not absorb, not the standing two-sided pressure that a CDA book's imbalance describes.

Second, even where the residual is two-sided, the *reason* an order is still there differs between the two mechanisms. A CDA resting order has been passed over continuously by every unit of opposing flow that has arrived since it was posted — its survival is a standing, cumulative judgment by the market. An FBA residual order is unmatched because of how much opposing volume happened to land in that one specific batch window; it rolls forward and may clear in the very next one. "Still unmatched" means something more persistent in a CDA book than it does in a one-second FBA buffer.

What the Metric Formulas Actually Use

None of the above is a problem this thesis's metrics inherited silently; each book-like FBA quantity in Section 2.5 was already defined to sidestep one of the two issues above, for a reason now made explicit.

Depth at best uses $D(p^*)/S(p^*)$ — the *total* volume eligible to trade at the clearing price, matched and unmatched together, not the unmatched residual. This is the closest analogue of a CDA book's touch size ("how much was there at the best price"), and deliberately not an answer to "how much is left over." It is read off the one price that cleared, not a provisional one, but it still counts the volume about to execute at that price, so it carries the Pre-Clearing section's mixing problem; this is one reason Section 6.7.2 treats it as not comparable with the CDA's touch size.

Depth within a threshold is also read from the batch just before it clears, so it too includes orders about to execute. That can only make the FBA look deeper than its standing liquidity (Section 6.7.2).

Quoted spread uses the post-clearing residual, and only for batches where both a best unfilled buy *and* a best unfilled sell exist. Because unmatched orders on both sides roll over, this holds in almost every second, so the gate rarely binds; what remains is the second problem above, that the leftover set is not the same kind of object as a CDA book, which is why quoted spread is read with caution in Section 6.7.

Book imbalance is not computed for the FBA at all. Unlike quoted spread, it has no gated, partial version here: a book-imbalance figure needs a resting two-sided book to read continuously, which, as argued above, the FBA does not have between clears, so this thesis does not build a proxy for an object that does not exist.

Almost everything else — executed volume, trade count, VWAP, effective and realized spread, price impact — is built from the *executed trade set*, not the unmatched set at all. A trade is timing-unambiguous for both mechanisms, since it only exists post-clearing either way (a CDA trade the instant of crossing, an FBA trade the instant the batch clears), which is exactly why trade-based metrics sidestep the pre/post-clearing dilemma above entirely rather than resolving it. Effective and realized spread and price impact also need the direction of each trade, which FBA trades do not have, so their FBA values are reported but not compared (Section 6.7.1).

In summary, the FBA's unmatched orders — the leftover residual this appendix has been discussing — appear in exactly one metric, quoted spread, and only there under the two-sided gate above; the two depth measures read the batch before it clears instead, and book imbalance is not attempted. Nowhere in this thesis is the unmatched set read, on its own, as a general stand-in for "the FBA order book."